sherpa.

Product Delivery

от плана до релиза и дальше

Практическое руководство

Содержание

01 Что такое Product Delivery и чем отличается от Discovery 5

02 Роль PM в Delivery-процессе 10

03 Цели и ценность системного подхода 16

04 Связь с бизнес-целями и стратегией компании 22

05 Определение North Star и outcome-метрик 27

06 Треугольник ограничений: время, бюджет, качество 32

07 Roadmap: outcome vs output 37

08 Backlog: структура и уровни детализации 42

09 Приоритизация: RICE, WSJF, ICE, MoSCoW 47

10 Планирование релизов и итераций 53

11 Декомпозиция фич и задач 58

12 User stories, Job Stories и Acceptance Criteria 63

13 Спецификации: когда нужен PRD, а когда достаточно Miro 68

14 Техническое уточнение и grooming 74

15 Методологии: Scrum, Kanban, Scrumban 79

16 Циклы разработки: sprint и flow 84

17 Управление зависимостями 89

18 Роль PM и взаимодействие с командой разработки 94

19 Подходы к QA: shift-left, автоматизация, ручные проверки 99

20 Типы тестов: unit, integration, end-to-end, нагрузочные 104

21 Beta-тесты и тестирование на реальных пользователях 109

22 Критерии качества 114

23 Подготовка к релизу: чек-листы, freeze, релиз-ноты 119

24 Feature toggles и стратегии rollout 124

25 A/B-тестирование и постепенный rollout 129

26 Go-to-market: координация с маркетингом и продажами 134

27 Настройка аналитики и метрик здоровья продукта 139

28 Мониторинг ошибок и инцидентов: Sentry, Datadog и другие 144

29 Поддержка пользователей и работа с обратной связью 150

30 SLA, SLO и работа с техническим долгом 155

31 Оценка достижения целей релиза 160

32 Сравнение план/факт по метрикам и срокам 165

33 Retrospective команды 170

34 Выявление инсайтов для будущих итераций 175

35 Автоматизация CI/CD и тестирования 180

36 Налаживание процессов Continuous Delivery 185

37 Управление несколькими командами и продуктами 190

38 Баланс discovery и delivery в долгосрочной перспективе 195

ПРИЛОЖЕНИЯ

A Шаблоны документов 200

B Глоссарий 206

Б Чек-листы по этапам 203

КАК ЧИТАТЬ

- правило или ключевое число
- важно: типичная ошибка
- мировой кейс или пример из практики
- классика: идея, автор, год
- f формула: сначала общий вид, затем значения символов

ПРАКТИКА задание после главы

СКВОЗНОЙ КЕЙС проходит через все 38 глав

«Чек»

Облачная касса и учёт для кафе и небольших магазинов: кассовое приложение на планшете, личный кабинет и интеграции с эквайрингом и ОФД. Клиентов много, но релизы выходят раз в две недели, каждый третий ломает что-то у части точек, а большая фича идёт от идеи до прода два с половиной месяца. Продакт-менеджер Ника отвечает за доставку фичи «Остатки» и за то, чтобы процесс перестал зависеть от героизма. Мы проходим её путь от дорожной карты до непрерывной поставки.

ПРОДАКТ-МЕНЕДЖЕР
Ника
хочет предсказуемых релизов с измеримым эффектом

ТЕХЛИД
Егор
хочет меньше пожаров и больше автоматизации

РУКОВОДИТЕЛЬ QA
Алина
хочет, чтобы качество закладывали до тестирования

Подписка 2 490 ₽ в месяц за точку **Клиенты** кафе, кофейни, магазины у дома
Платформы касса на Android, веб-кабинет **Команда** 4 продуктовые команды, 32 инженера

ВОСЕМЬ ВВОДНЫХ И РЕЗУЛЬТАТ

18 000 точек-клиентов	**44,8 млн ₽** выручка в месяц (MRR)	**2,4%** отток точек в месяц
раз в 2 недели частота релизов	**74 дня** от идеи до прода	**18%** релизов с откатом или хотфиксом
9 критичных инцидентов в квартал	**99,5%** доступность кассы	**отток 2,4 → 1,8% · 74 → 30 дней** цель года: удержание и скорость поставки

Компания вымышлена, но собрана из типичных ситуаций B2B SaaS. Числа согласованы: выручка, отток, частота релизов, инциденты и эффект фич выводятся друг из друга.

01 Что такое Product Delivery и чем отличается от Discovery

После неё вы различаете исследование возможностей и поставку решений, понимаете, где проходит граница между ними и почему продукт проваливается, когда одно подменяют другим.

1.1 Два потока · 1.2 Где граница · 1.3 Кейс «Чека»: два года без различий · 1.4 Стоимость поставки ненужного · 1.5 Бенчмарки · 1.6 Анти-паттерны · 1.7 Вопросы для самопроверки

Продуктовая работа состоит из двух потоков. Discovery отвечает на вопрос «что стоит делать»: какие проблемы пользователей решать, какие решения ценны, реализуемы и выгодны. Delivery отвечает на вопрос «как надёжно довести это до пользователей»: спланировать, разработать, проверить качество, выпустить, измерить эффект и поддерживать. Без discovery команда быстро поставляет ненужное. Без delivery — бесконечно исследует и ничего не выпускает.

1.1 Два потока

	DISCOVERY	DELIVERY
Вопрос	Что делать и зачем?	Как доставить надёжно и быстро?
Результат	Проверенная идея, понимание проблемы	Работающая функция у пользователей
Неопределённость	Высокая: ценность неизвестна	Ниже: ценность подтверждена, риски — в исполнении
Артефакты	Интервью, прототипы, эксперименты	Бэклог, спецификации, код, релизы, метрики
Ошибка	Сделать не то	Сделать то, но поздно, дорого или с дефектами
Метрики	Доля подтверждённых гипотез	Время поставки, качество, эффект на outcome

1.2 Где граница

Граница не жёсткая. Хорошая команда ведёт оба потока параллельно: пока одна фича в разработке, следующая проходит проверку. Но для каждой фичи есть момент передачи — когда ценность достаточно подтверждена, чтобы вложить в неё недели инженерной работы. Если передача происходит слишком рано, delivery превращается в дорогой эксперимент. Слишком поздно — команда теряет время на перепроверку очевидного.

КЛАССИКА Джефф Паттон и Марти Каган, «Dual-Track Agile», 2012–2017

Двойной поток

Паттон и Каган описали продуктовую разработку как два параллельных трека: discovery быстро проверяет идеи прототипами и экспериментами, delivery превращает проверенные идеи в продукт производственного качества.

Ключевой принцип — обе дорожки ведёт одна команда. Если discovery делает отдельный отдел, а delivery — разработка, контекст теряется при передаче.

1.3 Кейс «Чека»: два года без различий

Ника пришла в «Чек», когда компания выпускала фичи «по запросам крупных клиентов». Она разобрала 24 фичи за последние два года: сколько стоили, сколько точек ими пользуются и были ли они проверены до разработки.

ТИП ФИЧИ	ФИЧ	ПРОВЕРЕНЫ ДО РАЗРАБОТКИ	ИСПОЛЬЗУЮТ ≥ 20% ТОЧЕК
По запросу крупного клиента	11	1	2
Идея руководства	7	0	2
Из обращений поддержки	4	2	3
Из исследований	2	2	2
Итого	**24**	**5**	**9**

Из 24 фич массово используются 9, и почти все они прошли хотя бы минимальную проверку. Остальные 15 съели, по оценке Егора, около 60% инженерного времени за два года. Вывод Ники: проблема «Чека» не только в медленной поставке, но и в том, что поставляется. Поэтому она начала с разделения потоков: ни одна крупная фича не идёт в разработку без короткой проверки ценности.

1.4 Стоимость поставки ненужного

ФОРМУЛА 1.1 · ПОТЕРИ НА НЕПРОВЕРЕННЫХ ФИЧАХ

$$W = N_{ф} \cdot (1 - u) \cdot C_{ф} \tag{1.1}$$

где

$N_{ф}$ — число выпущенных фич за период — шт.

u — доля фич, которые дали эффект — доля

$C_{ф}$ — средняя стоимость фичи — ₽

W — потери на фичах без эффекта — ₽

«Чек» за 2 года: 24 фичи, u = 9 / 24 ≈ 0,375, средняя фича — 6 недель команды из 8 человек ≈ 9,6 млн ₽ → W ≈ 24 · 0,625 · 9,6 ≈ 144 млн ₽

ПРАВИЛО

Прежде чем брать фичу в delivery, задайте два вопроса: какая проблема пользователей доказана и как мы поймём, что фича её решила. Если ответов нет, фича идёт в discovery, а не в спринт.

МИРОВОЙ КЕЙС Google Glass, 2013–2015

Провал быстрой поставки

Google быстро разработала и выпустила очки дополненной реальности для широкой публики — с впечатляющим инженерным исполнением. Но вопросы о том, кому и зачем нужны очки, как к ним отнесутся окружающие и сколько люди готовы платить, остались без ответа.

Потребительскую версию свернули, а продукт позже переориентировали на промышленное применение — туда, где ценность нашлась. Хорошая поставка не спасает непроверенную ценность.

ПЛОХО

Всё в delivery

«Крупный клиент попросил — ставим в спринт».

Фича для одного клиента, 6 недель команды, 2% точек пользуются.

ХОРОШО

Сначала проверка

«Сколько точек с такой проблемой? Проверим прототипом на 10 клиентах».

В delivery идут фичи с доказанной ценностью.

1.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Доля фич с положительным эффектом в зрелых продуктах	около трети
Время на проверку перед крупной фичей	1–4 недели
Доля времени команды на discovery	10–20%
Фичи по запросу одного клиента	проверять на других клиентах до разработки

1.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Delivery без discovery	Дорогие ненужные фичи	Проверка ценности до разработки
Discovery без delivery	Исследования без релизов	Срок передачи в разработку
Discovery в отдельном отделе	Потеря контекста	Одна команда, два трека
Фичи «под клиента»	Продукт для одного	Проверка на сегменте

ПРИМЕР

Едтех

Онлайн-школа год выпускала по две фичи в месяц и гордилась скоростью. Анализ показал, что 70% новых функций открывали менее 5% студентов. Команда ввела двухнедельный цикл проверки перед разработкой: число фич упало вдвое, а доля используемых выросла до 60%.

1.7 Вопросы для самопроверки

- На какие вопросы отвечают discovery и delivery?
- Чем отличаются ошибки в каждом потоке?
- Что такое двойной поток по Паттону и Кагану?
- Как оценить потери на непроверенных фичах?
- Когда фича готова к передаче в delivery?

ПРАКТИКА 01 после главы 01

Проведите аудит фич за последний год.

ЧТО СДЕЛАТЬ

01	Выпишите все фичи, выпущенные за 12 месяцев.	☐
02	Отметьте источник каждой и была ли проверка ценности.	☐
03	Посчитайте долю используемых фич по сегментам.	☐
04	Оцените потери по формуле (1.1).	☐
05	Сформулируйте правило передачи фичи в delivery.	☐

РЕЗУЛЬТАТ Покажите результат руководству — это самый сильный аргумент за discovery.

02 Роль PM в Delivery-процессе

После неё вы понимаете, за что продакт отвечает в поставке, а за что — команда, техлид и проектный менеджер, где продакт добавляет ценность, а где мешает, и как распределить своё время между стратегией, командой и релизами.

2.1 Зоны ответственности · 2.2 Продакт, проджект и владелец продукта · 2.3 Где продакт добавляет ценность · 2.4 Кейс «Чека»: неделя Ники · 2.5 Бенчмарки · 2.6 Анти-паттерны · 2.7 Вопросы для самопроверки

В delivery продакт-менеджер не пишет код, не тестирует и не управляет задачами разработчиков. Его вклад — в решениях: что входит в релиз, что можно упростить, когда выпускать, как измерять успех, что делать, если план разошёлся с реальностью. Команда отвечает за то, как сделать; продакт — за то, что и зачем, и за то, чтобы результат дошёл до пользователей и дал эффект.

2.1 Зоны ответственности

ВОПРОС	PM	ТЕХЛИД	КОМАНДА	QA
Что делаем и зачем	решает	влияет	влияет	—
Приоритеты бэклога	решает	влияет	влияет	влияет
Как реализовать	влияет	решает	решает	влияет
Оценка сроков	влияет	решает	решает	влияет
Объём релиза и компромиссы	решает	влияет	влияет	влияет
Качество к релизу	критерии	решает	делает	подтверждает
Измерение эффекта	решает	—	влияет	—

2.2 Продакт, проджект и владелец продукта

Продакт-менеджер	Отвечает за ценность и результат: проблема, решение, метрики, приоритеты.
Владелец продукта (Product Owner) в Scrum	Роль внутри фреймворка: управляет бэклогом и принимает инкремент. Часто совмещается с продактом.
Проджект- или деливери-менеджер	Отвечает за сроки, ресурсы, зависимости и риски проекта. В небольших компаниях эту работу делает продакт или техлид.

2.3 Где продакт добавляет ценность

ПЯТЬ ВКЛАДОВ PM В DELIVERY

1. **Ясная цель**
   Каждый в команде знает, какую проблему решает релиз и какая метрика должна измениться.
2. **Быстрые решения**
   Ответ на вопрос разработки в течение дня, а не через неделю.
3. **Срезание объёма**
   Предлагает, что выпустить раньше и проще, когда сроки горят.
4. **Связь с бизнесом**
   Координирует маркетинг, продажи, поддержку вокруг релиза.
5. **Замыкание цикла**
   Проверяет эффект после релиза и приносит выводы в следующую итерацию.

2.4 Кейс «Чека»: неделя Ники

В первый месяц Ника записывала, на что уходит её время. Результат её расстроил: половина недели — статусные встречи и ручные отчёты, а на работу с метриками и пользователями оставались часы.

ДЕЯТЕЛЬНОСТЬ	БЫЛО, Ч/НЕД	СТАЛО ЧЕРЕЗ 3 МЕСЯЦА
Статусные встречи и отчёты	16	5 (автоматический дашборд)
Ответы на вопросы разработки	6	7
Груминг и планирование	5	5
Работа с метриками и экспериментами	3	8
Клиенты и исследования	2	6
Координация продаж, маркетинга, поддержки	8	7
Прочее	4	2

ФОРМУЛА 2.1 · ДОЛЯ ВРЕМЕНИ НА ЦЕННОСТЬ

$$V = \frac{t_{\text{цен}}}{t_{\text{всего}}} \cdot 100\% \qquad (2.1)$$

где

$t_{\text{цен}}$ — часы на работу, которая влияет на решения: метрики, клиенты, приоритеты, ответы команде — ч

$t_{\text{всего}}$ — всего рабочих часов — ч

```
Ника: было (6 + 5 + 3 + 2) / 44 ≈ 36%; стало (7 + 5 + 8 + 6) / 40 = 65%
```

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Становиться «менеджером задач»: раздавать тикеты, спрашивать статусы, двигать карточки. Это забирает время у продуктовой работы и снимает с команды ответственность за процесс. Статус должен быть виден на доске и дашборде без вашего участия.

КЛАССИКА Марти Каган, «Inspired», 2008, 2017; «Empowered», 2020

Продакт как владелец результата

Каган различает «команды-фабрики», которым спускают список фич, и «расширенные в полномочиях» команды, которым дают проблемы и метрики. Во втором случае продакт отвечает за ценность и жизнеспособность, а не за выполнение плана.

Для delivery это означает: продакт оценивается не по числу выпущенных фич, а по изменению outcome-метрик.

МИРОВОЙ КЕЙС Spotify, модель «отрядов», 2012

Один продакт на продукт

В описании Spotify каждый «отряд» — автономная команда до 8 человек с продакт-оунером, который определяет приоритеты, но не говорит, как делать. Сам Spotify позже признавал, что модель на практике работала не так гладко, как в презентациях.

Но принцип прижился: продакт — часть команды, а не внешний заказчик, и отвечает за «что», а команда — за «как».

ПЛОХО

PM — диспетчер

Спрашивает статусы, назначает задачи, пишет отчёты вручную.

На продукт остаётся треть времени.

ХОРОШО

PM — владелец результата

Цель понятна команде, статусы видны на доске, решения — за день.

Две трети времени — на метрики, клиентов, приоритеты.

2.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Доля времени PM на ценность	больше половины
Время ответа на вопрос разработки	в течение рабочего дня
Продактов на команду	1 на 5–9 инженеров
Статусные встречи	заменять дашбордом и асинхронными апдейтами

2.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
PM-диспетчер	Нет времени на продукт	Статус на доске
PM решает, как делать	Конфликт с командой	Зоны ответственности
PM пропадает после планирования	Команда решает сама	Ответ в течение дня
Оценка по числу фич	Фабрика фич	Оценка по outcome

ПРИМЕР

Финтех

В платёжной компании продакт каждый день проводил 40-минутные статусные встречи с командой. Их заменили доской с автоматическими статусами и 10-минутным стендапом команды без продакта. Освободившиеся 3 часа в неделю продакт тратил на звонки с клиентами — через квартал два его открытия стали приоритетами года.

2.7 Вопросы для самопроверки

- [ ] За какие решения отвечает продакт в delivery?
- [ ] Чем продакт отличается от проджект-менеджера?
- [ ] Какие пять вкладов PM в delivery вы знаете?
- [ ] Как посчитать долю времени на ценность?
- [ ] Почему PM не должен быть менеджером задач?

ПРАКТИКА 02 после главы 02

Проверьте, на что уходит ваше время.

ЧТО СДЕЛАТЬ

01	Запишите все рабочие активности за неделю с часами.	☐
02	Посчитайте долю времени на ценность по формуле (2.1).	☐
03	Заполните с командой таблицу зон ответственности.	☐
04	Найдите 2 статусные активности, которые можно автоматизировать.	☐
05	Освободившееся время запланируйте на метрики и клиентов.	☐

РЕЗУЛЬТАТ Повторите замер через месяц.

03 Цели и ценность системного подхода

После неё вы объясняете руководству, зачем выстраивать delivery как систему, измеряете её состояние метриками DORA и потока и ставите цели улучшения, которые связаны с деньгами, а не только со скоростью.

3.1 Четыре метрики DORA · 3.2 Метрики потока · 3.3 Кейс «Чека»: где теряются 74 дня · 3.4 Ценность в деньгах · 3.5 Бенчмарки · 3.6 Анти-паттерны · 3.7 Вопросы для самопроверки

Поставка, которая держится на героях, работает до первого отпуска ключевого разработчика. Системный подход означает, что процесс предсказуем: понятно, как идея становится релизом, сколько это занимает, где узкие места и что происходит, когда что-то ломается. Предсказуемость важна бизнесу больше скорости: обещание клиенту, бюджет и маркетинговый план строятся на сроках.

3.1 Четыре метрики DORA

МЕТРИКА	ЧТО ИЗМЕРЯЕТ	«ЧЕК» СЕЙЧАС
Частота деплоев	Как часто изменения попадают в прод	раз в 2 недели
Время поставки изменения (lead time for changes)	От коммита до прода	9 дней
Доля неудачных изменений (change failure rate)	Релизы с откатом, хотфиксом или инцидентом	18%
Время восстановления (MTTR)	От начала инцидента до восстановления	4 часа

КЛАССИКА Николь Форсгрен, Джез Хамбл, Джин Ким, «Accelerate», 2018

Ускоряйся

По данным многолетних исследований State of DevOps авторы показали, что четыре метрики поставки отличают высокоэффективные команды от остальных и связаны с результатами бизнеса: прибыльностью, долей рынка, продуктивностью.

Главное открытие — скорость и стабильность не противоречат друг другу. Лучшие команды деплоят чаще и при этом ломают реже, потому что изменения маленькие и проверяются автоматически.

3.2 Метрики потока

DORA описывает техническую поставку от коммита. Продакту нужно видеть весь путь — от идеи до пользователя. Для этого используют метрики потока: время от взятия идеи в работу до релиза, долю времени ожидания и пропускную способность.

ФОРМУЛА 3.1 · ЭФФЕКТИВНОСТЬ ПОТОКА

$$FE = \frac{t_{\text{раб}}}{t_{\text{раб}} + t_{\text{ож}}} \cdot 100\% \tag{3.1}$$

где

$t_{\text{раб}}$ — время, когда над элементом активно работают — дн.

$t_{\text{ож}}$ — время ожидания: в очереди, на ревью, на согласовании, на тестировании — дн.

FE — эффективность потока — %

«Чек», средняя фича: 74 дня от идеи до прода, активная работа 17 дней → FE ≈ 23%. Три четверти времени фича ждёт

3.3 Кейс «Чека»: где теряются 74 дня

Ника и Егор разобрали путь пяти последних фич по датам из трекера и переписки. Картина была типичной: работа шла быстро, а между этапами фичи неделями лежали в очередях.

ЭТАП	РАБОТА, ДН.	ОЖИДАНИЕ, ДН.	ПРИЧИНА ОЖИДАНИЯ
Идея → решение о старте	1	14	Ежемесячный комитет
Спецификация	3	6	Согласования с продажами

ЭТАП	РАБОТА, ДН.	ОЖИДАНИЕ, ДН.	ПРИЧИНА ОЖИДАНИЯ
Разработка	9	8	Переключения, зависимости от другой команды
Тестирование	3	10	Ручной регресс перед релизом
Ожидание релиза	—	7	Релизное окно раз в 2 недели
Выкатка	1	12	Поэтапно по партнёрам, ручная проверка
Итого	**17**	**57**	**—**

3.4 Ценность в деньгах

ФОРМУЛА 3.2 · СТОИМОСТЬ ЗАДЕРЖКИ

$$CoD = \Delta V \cdot t_{зад} \tag{3.2}$$

где

ΔV — ценность фичи в единицу времени после релиза ₽/нед.

$t_{зад}$ — задержка релиза нед.

«Остатки»: ожидаемое снижение оттока 0,4 п.п. → 72 точки в месяц × 2 490 ₽ ≈ 180 тыс. ₽ MRR в месяц накопительно; грубо ≈ 40 тыс. ₽ новой MRR за каждую неделю. 8 недель ожидания ≈ 320 тыс. ₽ MRR, которые не начали накапливаться

ПРАВИЛО

Измеряйте поставку от идеи до пользователя, а не только от коммита до прода. Самые большие потери обычно не в разработке, а в очередях между этапами.

МИРОВОЙ КЕЙС Отчёты State of DevOps, DORA, 2014-2024

Быстрее и надёжнее

Ежегодные исследования DORA на десятках тысяч инженеров показывали: элитные команды деплоят по требованию, несколько раз в день, с временем поставки меньше дня и восстановлением меньше часа, при этом доля неудачных изменений у них не выше, чем у медленных команд.

Разрыв между лучшими и худшими — сотни раз по частоте и времени поставки. Этот разрыв почти полностью объясняется практиками: автоматизация, маленькие изменения, непрерывная интеграция.

ПЛОХО

«Разработка медленная»

Давление на разработчиков работать быстрее.

Выгорание, а 74 дня остаются: ожидания не тронуты.

ХОРОШО

Карта потока

Работа — 17 дней, ожидание — 57.

Цели на очереди: комитет, ручной регресс, релизное окно.

3.5 Бенчмарки

МЕТРИКА	ВЫСОКИЙ УРОВЕНЬ ПО DORA
Частота деплоев	от раза в день до раза в неделю
Время поставки изменения	от дня до недели
Доля неудачных изменений	до 15%
Время восстановления	меньше дня
Эффективность потока	40% и выше — хороший уровень

3.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Только скорость разработки	Очереди не видны	Поток от идеи до пользователя
Метрики для наказания	Подгонка цифр	Метрики для улучшения процесса
Скорость против качества	Ложный выбор	DORA: оба вместе

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Нет денежной оценки	Улучшения не приоритет	Стоимость задержки

ПРИМЕР

Электронная коммерция

Маркетплейс увидел, что фичи проводят по 3 недели в очереди на ручное тестирование перед ежемесячным релизом. Автоматизация регресса и переход на еженедельные релизы сократили время от готовности кода до прода с 25 до 4 дней, а доля откатов снизилась с 12 до 6%.

3.7 Вопросы для самопроверки

- Почему предсказуемость важнее скорости?
- Какие четыре метрики DORA вы знаете?
- Как посчитать эффективность потока?
- Что такое стоимость задержки?
- Почему скорость и стабильность не противоречат друг другу?

ПРАКТИКА 03 после главы 03

Постройте карту потока для 3–5 последних фич.

ЧТО СДЕЛАТЬ

01	Восстановите даты этапов по трекеру и переписке.	☐
02	Разделите время на работу и ожидание.	☐
03	Посчитайте эффективность потока по формуле (3.1).	☐
04	Оцените стоимость задержки одной фичи по формуле (3.2).	☐
05	Найдите две самые длинные очереди.	☐

РЕЗУЛЬТАТ Снимите текущие значения DORA — это ваша точка отсчёта.

04 Связь с бизнес-целями и стратегией компании

После неё вы связываете каждую фичу в поставке с целью компании через дерево целей, объясняете команде, зачем она делает то, что делает, и отказываетесь от работы, которая не ведёт к цели.

4.1 Дерево целей
4.2 Как устроено дерево
4.3 Декомпозиция цели
4.4 Кейс «Чека»: зачем команде дерево
4.5 Бенчмарки
4.6 Анти-паттерны
4.7 Вопросы для самопроверки

Команда, которая не знает, зачем делает фичу, принимает решения по-своему: полирует то, что не важно, и упрощает то, что важно. Связь delivery со стратегией — не формальность для презентаций. Это ответ на десятки ежедневных вопросов: стоит ли ради сроков урезать отчёт, можно ли отложить интеграцию, какой баг чинить первым.

4.1 Дерево целей

```
Цель компании: MRR 44,8 → 58 млн ₽ за год
 Рычаг 1: новые точки — +7 200 за год (продажи и маркетинг)
 Рычаг 2: отток 2,4% → 1,8% в месяц (продукт)
  Причина ухода №1: нет учёта остатков — 31% ушедших
   Инициатива: фича «Остатки» — ожидаем -0,4 п.п. оттока
  Причина ухода №2: сбои кассы — 19% ушедших
   Инициатива: надёжность и релизы — ожидаем -0,2 п.п.
 Рычаг 3: тариф «Склад+» — 15% точек переходят, +900 ₽ в месяц
```

4.2 Как устроено дерево

Цель компании	Измеримый результат бизнеса на год: выручка, прибыль, доля рынка.
Рычаги	Части формулы цели, на которые можно влиять: новые клиенты, удержание, средний чек.

Причины	Почему рычаг сейчас не там, где нужно, — из данных и исследований.
Инициативы	Что делаем, чтобы повлиять на причину; у каждой ожидаемый эффект в числах.

4.3 Декомпозиция цели

ФОРМУЛА 4.1 · MRR ЧЕРЕЗ РЫЧАГИ

$$MRR_1 = (N_0 + A - C) \cdot ARPA \tag{4.1}$$

где

N_0	—	точек на начало года	шт.
A	—	новых точек за год	шт.
C	—	ушедших точек за год	шт.
$ARPA$	—	средний платёж точки в месяц	₽

«Чек»: при оттоке 1,8% в месяц за год уходит ≈ 3 900 точек; N_1 = 18 000 + 7 200 - 3 900 = 21 300; ARPA с учётом «Склад+» 2 490 + 0,15 · 900 ≈ 2 625 ₽ → MRR ≈ 55,9 млн ₽. Для 58 млн нужно ещё ≈ 800 точек или выше доля «Склад+» — на это нацелены продажи

Расчёт показал команде две вещи. Без снижения оттока цель недостижима даже при агрессивных продажах: при 2,4% за год уйдёт около 5 100 точек. И «Остатки» влияют сразу на два рычага — удержание и переход на «Склад+».

4.4 Кейс «Чека»: зачем команде дерево

До дерева команда «Остатков» воспринимала фичу как «заказ продаж». После разбора дерева на общей встрече вопросы команды изменились. Разработчик спросил, нужен ли в первой версии импорт из Excel, если точки, которые уходят, — это маленькие кофейни с 200 позициями. Ответ из данных: нет, достаточно ручного ввода и сканера; импорт перенесли на вторую версию и сэкономили две недели.

ВОПРОС КОМАНДЫ	ОТВЕТ ЧЕРЕЗ ДЕРЕВО
Нужен ли импорт из Excel в v1?	Нет: уходят малые точки с 100–300 позициями

ВОПРОС КОМАНДЫ	ОТВЕТ ЧЕРЕЗ ДЕРЕВО
Что важнее: отчёт или списание при продаже?	Списание: без него остатки неверны, и точка уходит
Чинить баг печати чеков или делать фичу?	Баг: сбои кассы — причина №2 ухода
Делать перемещения между точками?	Позже: у 82% клиентов одна точка

ПРАВИЛО

Для каждой инициативы в дорожной карте должен быть путь вверх по дереву до цели компании и ожидаемый эффект в числах. Если пути нет — это кандидат на вычёркивание.

КЛАССИКА — Ричард Румельт, «Good Strategy / Bad Strategy», 2011

Стратегия как набор выборов

Румельт описывает хорошую стратегию как диагноз проблемы, руководящий принцип и согласованные действия. Плохая стратегия — набор амбициозных целей без диагноза и выбора.

Дерево целей — инструмент перевода стратегии в delivery: диагноз (почему уходят точки), принцип (удержание важнее новых функций для крупных клиентов), действия (инициативы с эффектом).

МИРОВОЙ КЕЙС — Amazon, «флайвил», 2001

Одна метрика для всей компании

Джефф Безос и руководство Amazon описали «маховик»: низкие цены → больше покупателей → больше продавцов → больше ассортимента → ниже издержки → ещё ниже цены. Каждая инициатива оценивалась по тому, какое звено маховика она ускоряет.

Модель давала командам простой фильтр решений, похожий на дерево целей: если работа не ускоряет маховик, её стоит пересмотреть.

ПЛОХО

Фича «потому что надо»

«Продажи просят остатки, делаем всё, что у конкурентов».

Импорт, перемещения, отчёты — 14 недель, срок сорван.

ХОРОШО

Фича с путём к цели

«Остатки снижают отток малых точек; в v1 — то, без чего они уходят».

9 недель, импорт и перемещения — во второй версии.

4.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Рычагов у цели компании	3–5
Инициатив на рычаг в квартал	1–3
Инициатив без пути к цели	0
Пересмотр дерева	раз в квартал

4.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Цель без рычагов	Непонятно, на что влиять	Формула цели
Инициативы без эффекта в числах	Нельзя сравнить	Ожидаемый эффект
Дерево только у руководства	Команда не знает зачем	Разбор с командой
Всё «стратегическое»	Нет выбора	Вычёркивать без пути к цели

ПРИМЕР

Логистика

Сервис грузоперевозок держал в плане 22 инициативы. Построение дерева от цели «маржа +3 п.п.» показало, что 9 из них не влияют ни на один рычаг. Их убрали, освободив две команды на автоматизацию подбора машин, которая дала +2,1 п.п. маржи за полгода.

4.7 Вопросы для самопроверки

- Из каких уровней состоит дерево целей?
- Как разложить цель по выручке на рычаги?
- Почему команде важно видеть путь фичи к цели?
- Что такое хорошая стратегия по Румельту?
- Что делать с инициативой без пути к цели?

ПРАКТИКА 04 после главы 04

Постройте дерево целей для своей команды.

ЧТО СДЕЛАТЬ

01	Запишите цель компании на год в числах.	☐
02	Разложите её на рычаги по формуле, похожей на (4.1).	☐
03	Для своего рычага найдите 2–3 причины в данных.	☐
04	Привяжите текущие инициативы к причинам и оцените эффект.	☐
05	Отметьте инициативы без пути к цели.	☐

РЕЗУЛЬТАТ Разберите дерево с командой на 30-минутной встрече.

05 Определение North Star и outcome-метрик

После неё вы выбираете North Star для продукта, формулируете outcome-метрики для каждой фичи до начала разработки и отличаете результат для пользователя и бизнеса от выпущенного объёма работ.

5.1 Output, outcome, impact · 5.2 North Star · 5.3 Outcome-метрики фичи · 5.4 Кейс «Чека»: от output к outcome · 5.5 Бенчмарки · 5.6 Анти-паттерны · 5.7 Вопросы для самопроверки

Output — то, что команда выпустила: фича, экран, интеграция. Outcome — изменение поведения пользователей, которое привело к результату бизнеса: точки стали вести остатки, реже уходят, чаще переходят на дорогой тариф. Delivery, измеряемый output, превращается в фабрику фич. Delivery, измеряемый outcome, отвечает за то, чтобы фичи работали.

5.1 Output, outcome, impact

УРОВЕНЬ	ВОПРОС	«ОСТАТКИ»
Output	Что выпустили?	Модуль остатков в кассе и кабинете
Outcome	Как изменилось поведение?	40% точек ведут остатки через 3 месяца
Impact	Что получил бизнес?	Отток -0,4 п.п., переходы на «Склад+»

5.2 North Star

North Star — метрика, которая отражает ценность для клиента и предсказывает выручку. Для кассы ценность — работающие продажи. «Чек» выбрал метрику «активные точки»: провели не меньше 1 000 чеков за последние 4 недели. Сейчас их 14 200 из 18 000 — 79%.

ФОРМУЛА 5.1 · СВЯЗЬ NORTH STAR С ОТТОКОМ

$$q = s \cdot q_{\text{акт}} + (1 - s) \cdot q_{\text{неакт}} \tag{5.1}$$

где

q	—	средний месячный отток	доля
s	—	доля активных точек	доля
$q_{\text{акт}}$	—	отток активных	доля
$q_{\text{неакт}}$	—	отток неактивных	доля

```
«Чек»: s = 0,79, q_акт = 1,3%, q_неакт = 6,5% → q ≈ 0,79 · 1,3 + 0,21 · 6,5 ≈ 2,4%.
Если доля активных вырастет до 85%: q ≈ 2,1%
```

5.3 Outcome-метрики фичи

Для каждой фичи до начала разработки записывают: целевую outcome-метрику, её базовое значение, ожидаемое значение и срок, а также защитные метрики — что не должно ухудшиться. Без этого после релиза невозможно сказать, сработала ли фича.

МЕТРИКА	ТИП	БАЗА	ЦЕЛЬ	СРОК
Доля точек, ведущих остатки	Outcome (освоение)	0%	40%	3 месяца после релиза
Отток точек, ведущих остатки	Outcome (удержание)	—	≤ 1,6% в месяц	6 месяцев
Переход на «Склад+»	Impact	0%	15% точек	12 месяцев
Время проведения чека	Защитная	1,9 с	≤ 2,1 с	постоянно
Обращения в поддержку на 1 000 точек	Защитная	42	≤ 48	первые 2 месяца

5.4 Кейс «Чека»: от output к outcome

Дорожная карта «Чека» прошлого года состояла из названий фич. Ника переписала цели квартала через outcome — и это сразу изменило разговор с командой о «готовности». Фича считалась сделанной не тогда, когда её выкатили, а когда метрика освоения достигла цели или команда поняла, почему нет.

ПЛОХО

Цель-output

«Выпустить модуль остатков в Q1».

Выпустили, через полгода им пользуются 6% точек.

ХОРОШО

Цель-outcome

«40% точек ведут остатки к концу Q2, отток у них ≤ 1,6%».

Команда отвечает за освоение, а не только за релиз.

ПРАВИЛО

Запишите outcome-метрику, базу, цель, срок и защитные метрики в спецификацию до первой строки кода. Если команда не может договориться о метрике, она не понимает, какую проблему решает.

КЛАССИКА — Джош Седен, «Outcomes Over Output», 2019

Outcome вместо output

Седен определяет outcome как изменение поведения людей, которое ведёт к результату. Он предлагает формулировать задачи команды вопросом: какое поведение должно измениться, у кого и насколько?

Такой подход даёт команде свободу в выборе решения и ответственность за результат: если первое решение не изменило поведение, работа не закончена.

МИРОВОЙ КЕЙС — Medium, 2014

Время, а не клики

Платформа Medium выбрала главной метрикой не просмотры и не клики, а «общее время чтения» — сколько минут люди проводят, читая материалы. Метрика отражала ценность для читателя и авторов лучше, чем трафик.

Выбор North Star определил решения: Medium не оптимизировал кликбейтные заголовки, а продвигал длинные материалы, которые дочитывают.

5.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
North Star	одна на продукт
Outcome-метрик на фичу	1–2 целевых + 2–3 защитных

ПАРАМЕТР	ОРИЕНТИР
Срок оценки освоения	1–3 месяца после релиза
Срок оценки удержания	3–6 месяцев

5.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Цели-output	Фабрика фич	Outcome с базой и целью
North Star = выручка	Не реагирует на продукт	Метрика ценности для клиента
Нет защитных метрик	Скрытый вред	Скорость, обращения, ошибки
Метрика после релиза	Нечего сравнивать	База до разработки

ПРИМЕР

Saas

CRM-сервис годами измерял успех фич числом созданных сделок. Переход на North Star «сделки, закрытые с использованием автоматизаций» показал, что половина новых функций не влияла на закрытие сделок. Следующие два квартала команда работала только над автоматизациями, и удержание клиентов на 12 месяцев выросло на 7 п.п.

5.7 Вопросы для самопроверки

- [ ] Чем output отличается от outcome и impact?
- [ ] Какие свойства должны быть у North Star?
- [ ] Как связать North Star с оттоком?
- [ ] Что нужно записать об outcome-метрике до разработки?
- [ ] Зачем нужны защитные метрики?

ПРАКТИКА 05 после главы 05

Сформулируйте outcome-метрики для ближайшей фичи.

ЧТО СДЕЛАТЬ

01	Опишите output, outcome и impact фичи.	☐
02	Выберите целевую outcome-метрику и измерьте базу.	☐
03	Задайте цель и срок.	☐
04	Добавьте 2–3 защитные метрики.	☐
05	Проверьте связь outcome с North Star по формуле, похожей на (5.1).	☐

РЕЗУЛЬТАТ Вставьте таблицу метрик в спецификацию и покажите команде.

06 Треугольник ограничений: время, бюджет, качество

После неё вы управляете компромиссами между сроком, ресурсами, объёмом и качеством, предлагаете срезать объём вместо качества и аргументированно отвечаете на «а можно быстрее?».

6.1 Четыре переменные · 6.2 Кейс «Чека»: выставка 1 марта · 6.3 Цена «срезать качество» · 6.4 Бенчмарки · 6.5 Анти-паттерны · 6.6 Вопросы для самопроверки

Классический треугольник ограничений говорит: нельзя одновременно зафиксировать срок, бюджет и объём — что-то одно придётся менять, иначе пострадает качество. В продуктовой разработке ресурсы обычно фиксированы (команда есть), срок часто привязан к событию, поэтому главный рычаг продакта — объём. Качество — не рычаг: снижение качества ради срока почти всегда стоит дороже позже.

6.1 Четыре переменные

ПЕРЕМЕННАЯ	МОЖНО ЛИ МЕНЯТЬ	ЦЕНА ИЗМЕНЕНИЯ
Срок	Иногда: если не привязан к событию	Стоимость задержки, потерянные обещания
Ресурсы	Редко быстро: новые люди замедляют на старте	Закон Брукса, онбординг
Объём	Почти всегда: лучший рычаг	Меньше функций в первой версии
Качество	Не стоит	Инциденты, отток, техдолг

КЛАССИКА Фредерик Брукс, «Мифический человеко-месяц», 1975

Закон Брукса

Брукс, руководивший разработкой операционной системы IBM OS/360, сформулировал: добавление людей в опаздывающий проект делает его ещё более опаздывающим. Новичкам нужно время на вход, а старым участникам — на их обучение и координацию.

Вывод для продакта: «давайте добавим людей» — плохой ответ на сорванный срок в середине проекта. Срезание объёма работает быстрее.

6.2 Кейс «Чека»: выставка 1 марта

Продажи пообещали показать «Остатки» на отраслевой выставке 1 марта и начать продажи «Склад+». Оценка полного объёма — 14 недель, до выставки оставалось 9. Добавить людей Егор отказался: «Через месяц новые люди только начнут приносить пользу». Ника разложила объём по ценности для главной цели — удержания малых точек.

ЧАСТЬ ФИЧИ	НЕДЕЛЬ	ВАЖНО ДЛЯ УДЕРЖАНИЯ	РЕШЕНИЕ
Справочник товаров и ручной ввод	2	да	v1
Приёмка со сканером	2	да	v1
Списание при продаже на кассе	3	критично	v1
Отчёт по остаткам и уведомление «заканчивается»	2	да	v1
Импорт из Excel	2	нет (малые точки)	v2
Инвентаризация	2	средне	v1.1 через месяц
Перемещения между точками	1	нет (82% — одна точка)	v2
Итого v1	**9**	**—**	**к выставке**

Продажам Ника показала, что на выставке будет полноценный сценарий «принял товар — продал — увидел остаток — получил уведомление», а инвентаризация выйдет через месяц после. Импорт и перемещения отложили до сигнала от сетевых клиентов. Качество — автотесты на списание и нагрузочное тестирование кассы — не трогали.

6.3 Цена «срезать качество»

ФОРМУЛА 6.1 · ОЖИДАЕМАЯ СТОИМОСТЬ ПРОПУСКА ТЕСТОВ

$$E = p \cdot (C_{\text{инц}} + C_{\text{отток}}) - S \tag{6.1}$$

где

p	—	вероятность критичного дефекта без полного тестирования	доля
$C_{\text{инц}}$	—	стоимость инцидента: поддержка, хотфикс, компенсации	₽
$C_{\text{отток}}$	—	потери MRR от ушедших из-за инцидента	₽
S	—	сэкономленная стоимость тестирования	₽

```
Пропустить нагрузочный тест списания: p ≈ 0,25; инцидент 1,5 млн ₽; отток 150 точек × 2 490 × 12 ≈ 4,5 млн ₽; экономия 1 неделя QA ≈ 0,3 млн → E ≈ 0,25 · 6,0 - 0,3 = 1,2 млн ₽ ожидаемых потерь
```

ПРАВИЛО

Когда спрашивают «можно быстрее?», предлагайте вариант с меньшим объёмом, а не с меньшим качеством. Покажите, что войдёт к сроку, что позже и почему эта граница не вредит цели.

МИРОВОЙ КЕЙС Healthcare.gov, США, 2013

Срок без компромиссов

Портал медицинского страхования должен был запуститься 1 октября 2013 года — дата была закреплена политически. Объём не сокращали, интеграционное тестирование провели за недели до запуска.

В первый день сайт смогли использовать для оформления лишь несколько человек. Исправление заняло месяцы и потребовало привлечения внешней команды. Фиксированные срок и объём ударили по качеству — ровно как предсказывает треугольник.

ПЛОХО

Всё к сроку

14 недель работы за 9, тесты урезаны.

Инцидент на кассах в первый месяц, отток вместо удержания.

ХОРОШО

Срезать объём

Ядро сценария к выставке, инвентаризация через месяц.

Качество сохранено, обещание продажам выполнено.

6.4 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Буфер в оценке сроков	15–25%
Доля объёма, которую обычно можно перенести без потери ценности	30–40% первой версии
Добавление людей в проект	только в начале, не в середине
Тесты критичных сценариев	не сокращаются

6.5 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Зафиксировать всё	Страдает качество	Объём — рычаг
Добавить людей в середине	Ещё дольше	Срезать объём
Срезать тесты	Инциденты	Тесты критичного не трогать
«Да» без компромиссов	Сорванные обещания	Варианты с объёмом

ПРИМЕР

Геймдев

Студия к фиксированной дате выхода на консолях сократила число уровней с 30 до 22 и отложила кооперативный режим до бесплатного обновления. Игра вышла вовремя с высоким рейтингом, а обновление через три месяца вернуло игроков. Конкурент, выпустивший игру в срок с полным объёмом, но с ошибками, получил волну возвратов.

6.6 Вопросы для самопроверки

- Какие переменные есть в треугольнике ограничений?
- Почему объём — главный рычаг продакта?
- Что говорит закон Брукса?
- Как оценить ожидаемую стоимость пропуска тестов?
- Как ответить на «а можно быстрее?»

ПРАКТИКА 06 после главы 06

Разложите объём ближайшей фичи под срок.

ЧТО СДЕЛАТЬ

01	Разбейте фичу на части с оценками.	☐
02	Отметьте важность каждой части для цели.	☐
03	Сформируйте v1 под срок и список переносов.	☐
04	Оцените риск пропуска тестов по формуле (6.1), если его предлагают.	☐
05	Подготовьте разговор со стейкхолдерами: что будет к сроку и почему.	☐

РЕЗУЛЬТАТ Зафиксируйте решение письменно — через месяц его будут вспоминать по-разному.

07 Roadmap: outcome vs output

После неё вы строите дорожную карту вокруг проблем и результатов, а не списка фич с датами, выбираете формат под аудиторию и честно показываете неопределённость дальних горизонтов.

7.1 Два типа карт
7.2 Формат «сейчас — дальше — позже»
7.3 Уверенность в дорожной карте
7.4 Кейс «Чека»: разговор с продажами
7.5 Бенчмарки
7.6 Анти-паттерны
7.7 Вопросы для самопроверки

Дорожная карта — главный документ, через который бизнес видит будущее продукта. Классическая карта — диаграмма Ганта с фичами и датами — создаёт ложную точность: через три месяца половина дат сдвинута, а фичи оказались не теми. Карта, построенная на результатах, обещает не функции, а изменения, и оставляет команде свободу в выборе решений.

7.1 Два типа карт

	OUTPUT-КАРТА	OUTCOME-КАРТА
Элементы	Фичи с датами	Проблемы и целевые метрики
Обещание	«Выпустим X к 15 марта»	«Снизим отток малых точек к концу Q2»
Гибкость	Изменение = нарушение плана	Решение можно менять, цель остаётся
Ответственность команды	Выпустить	Добиться результата
Риск	Ложная точность, фабрика фич	Размытость, если метрики не заданы

7.2 Формат «сейчас — дальше — позже»

Популярный формат outcome-карты делит работу на три горизонта по уровню

уверенности. «Сейчас» — то, над чем работают, с высокой детализацией. «Дальше» — проблемы, к которым перейдут, с ориентировочным решением. «Позже» — направления без решений и дат. Чем дальше горизонт, тем меньше деталей.

ГОРИЗОНТ	«ЧЕК»: ПРОБЛЕМА	ЦЕЛЕВАЯ МЕТРИКА	УВЕРЕННОСТЬ
Сейчас (Q1)	Малые точки уходят без учёта остатков	40% ведут остатки, отток у них ≤ 1,6%	высокая: решение спроектировано
Сейчас (Q1)	Сбои кассы после релизов	Доля неудачных релизов 18% → 10%	высокая
Дальше (Q2)	Сетевые клиенты не видят остатки всех точек	Отток сетей -0,3 п.п.	средняя: проверяем
Дальше (Q2)	Долгое подключение новых точек	Время до первого чека 3 дня → 1 день	средняя
Позже (H2)	Владельцы не понимают прибыльность	Гипотеза: аналитика маржи	низкая: исследуем

7.3 Уверенность в дорожной карте

ФОРМУЛА 7.1 · ОЖИДАЕМАЯ ТОЧНОСТЬ ПЛАНА

$$A(h) \approx a^h \tag{7.1}$$

где

- a — доля пунктов, которые сохраняются при пересмотре через квартал — доля
- h — горизонт в кварталах — кв.
- A — ожидаемая доля пунктов, которые будут выполнены как запланировано — доля

```
«Чек», история прошлого года: из пунктов квартального плана сохранялись 70% → через 1 квартал 0,7, через 2 — 0,49, через 4 — 0,24. Детальный план на год сбудется на четверть
```

Расчёт стал главным аргументом Ники в разговоре с руководством: годовой Гант с фичами — не план, а иллюзия. Вместо него — квартальные цели с деталями и годовые направления с метриками.

7.4 Кейс «Чека»: разговор с продажами

Отдел продаж требовал карту с датами: «клиенты спрашивают, когда будет импорт из Excel». Ника договорилась о двух версиях одной карты: для клиентов и продаж — проблемы и горизонты без точных дат, кроме «Сейчас»; для команды — детальный план квартала. Продажи получили формулировку для клиентов: «Импорт — в планах на второй квартал, точную дату сообщим за месяц».

ПРАВИЛО

Точные даты — только для горизонта «Сейчас» и только после оценки командой. Для дальних горизонтов показывайте проблемы и метрики. Обещание фичи с датой через полгода — обещание, которое с вероятностью больше половины придётся нарушить.

КЛАССИКА Брюс Макарти и др., «Product Roadmaps Relaunched», 2017

Дорожная карта как стратегическое общение

Авторы предлагают строить карту из тем — проблем клиентов или бизнес-целей — с указанием горизонтов и оговоркой о неопределённости. Карта — инструмент договорённости о направлении, а не план работ.

Отдельно они советуют включать в карту видение и цели продукта, чтобы читатель понимал, почему темы стоят в таком порядке.

МИРОВОЙ КЕЙС GitHub, публичная дорожная карта, с 2020

Публичная карта

GitHub публикует дорожную карту в открытом репозитории: элементы сгруппированы по кварталам и стадиям, у каждого есть описание проблемы и уровень готовности. Пользователи видят направления, но компания явно оговаривает, что сроки могут меняться.

Публичная карта показывает, что прозрачность и отказ от жёстких дат совместимы: доверие строится на честности о неопределённости.

ПЛОХО

Годовой Гант

40 фич с датами на 12 месяцев.

Через квартал треть дат сдвинута, продажи обещают клиентам несуществующее.

ХОРОШО

Сейчас — дальше — позже

Проблемы и метрики, детали — только в ближайшем квартале.

Продажи говорят клиентам честно, команда выбирает решения.

7.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Тем в горизонте «Сейчас»	2–4 на команду
Пересмотр карты	раз в квартал, обновление «Сейчас» — ежемесячно
Точные даты	только для «Сейчас» после оценки
Версии карты	для команды, для руководства, для клиентов

7.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Фичи с датами на год	Ложная точность	Горизонты
Карта без метрик	Размытость	Целевая метрика у каждой темы
Одна карта для всех	Не те детали не тем людям	Версии под аудиторию
Карту не обновляют	Расходится с реальностью	Ежемесячный пересмотр «Сейчас»

ПРИМЕР

B2b

HR-платформа перешла с карты фич на карту проблем. Через квартал команда отказалась от запланированного модуля опросов: исследование показало, что проблему вовлечённости лучше решают уведомления руководителям. Цель «рост ежемесячных активных руководителей на 20%» выполнили, хотя фича в плане так и не появилась.

7.7 Вопросы для самопроверки

- Чем outcome-карта отличается от output-карты?
- Что такое формат «сейчас — дальше — позже»?
- Как оценить точность плана на разных горизонтах?
- Как показывать карту продажам и клиентам?
- Почему детали нужны только для ближайшего горизонта?

ПРАКТИКА 07 после главы 07

Переведите текущую дорожную карту в формат результатов.

ЧТО СДЕЛАТЬ

01	Для каждой фичи в карте сформулируйте проблему, которую она решает.	☐
02	Сгруппируйте проблемы по горизонтам.	☐
03	Задайте целевую метрику для каждой темы.	☐
04	Посчитайте по истории, сколько пунктов сохраняется за квартал, и формулу (7.1).	☐
05	Подготовьте версию карты для продаж.	☐

РЕЗУЛЬТАТ Согласуйте с руководством, какие даты вы обещаете, а какие — нет.

08 Backlog: структура и уровни детализации

После неё вы организуете бэклог по уровням — от тем и эпиков до историй и задач, — держите детальной только ближайшую работу и регулярно чистите бэклог, чтобы он оставался инструментом, а не свалкой.

8.1 Уровни бэклога · 8.2 Принцип айсберга · 8.3 Критерии готовности истории · 8.4 Кейс «Чека»: чистка бэклога · 8.5 Бенчмарки · 8.6 Анти-паттерны · 8.7 Вопросы для самопроверки

Бэклог — упорядоченный список работы, которую команда может сделать. Здоровый бэклог отвечает на вопрос «что делаем следующим и почему». Нездоровый — 800 карточек, половина из которых старше года, без приоритета и с размытыми названиями. Такой бэклог не помогает планировать, а создаёт иллюзию, что все идеи «учтены».

8.1 Уровни бэклога

УРОВЕНЬ	ОПИСАНИЕ	РАЗМЕР	ПРИМЕР В «ЧЕКЕ»
Тема	Проблема или цель из дорожной карты	квартал и больше	Удержание малых точек
Эпик	Крупная часть решения	4–12 недель	Модуль «Остатки» v1
Фича	Ценная для пользователя возможность	1–4 недели	Списание товаров при продаже
История	Небольшой срез ценности	1–5 дней	Кассир видит остаток товара при пробитии
Задача	Техническая работа внутри истории	часы — 2 дня	API остатков в кассе

8.2 Принцип айсберга

Детализация бэклога должна убывать с расстоянием: истории на ближайшие 2–3 спринта проработаны и оценены, следующие — на уровне фич, дальние — на уровне эпиков и тем.

Детализировать работу на полгода вперёд — тратить время: к моменту реализации требования изменятся.

ФОРМУЛА 8.1 · ГЛУБИНА ГОТОВОГО БЭКЛОГА

$$D = \frac{S_{\text{гот}}}{v} \tag{8.1}$$

где

$S_{\text{гот}}$	—	сумма оценок историй, готовых к разработке	SP
v	—	средняя скорость команды за спринт	SP/спринт
D	—	на сколько спринтов хватит готовых историй	спринтов

Команда «Остатков»: готовых историй на 68 SP при скорости 34 SP → D = 2 спринта — норма. До наведения порядка было 230 SP «готовых», из которых половина устарела

8.3 Критерии готовности истории

- [ ] Понятно, какой пользователь, что хочет и зачем.
- [ ] Есть критерии приёмки.
- [ ] Оценена командой.
- [ ] Зависимости известны и сняты или запланированы.
- [ ] Есть ссылка на макет или он не нужен.
- [ ] Влезает в спринт.

8.4 Кейс «Чека»: чистка бэклога

Бэклог команды «Остатков» содержал 412 карточек. Ника и Егор провели двухчасовую сессию чистки по простым правилам и оставили в работе меньше четверти.

КАТЕГОРИЯ	КАРТОЧЕК	РЕШЕНИЕ
Старше 12 месяцев без движения	163	Закрыты с комментарием «вернём, если появится сигнал»

КАТЕГОРИЯ	КАРТОЧЕК	РЕШЕНИЕ
Дубликаты	41	Объединены
Без связи с темами карты	74	В архив идей
Баги	38	Отдельный список, приоритизация по влиянию
Связаны с текущими темами	96	Остались, разложены по эпикам

Через три месяца ни одну из закрытых карточек не пришлось восстанавливать. Зато планирование спринта сократилось с трёх часов до полутора: команда больше не листала сотни неактуальных задач.

ПРАВИЛО

Бэклог — не хранилище всех идей. Если карточка не двигалась год и не связана с текущими целями, закройте её. Важная идея вернётся сама — через клиентов, данные или команду.

КЛАССИКА Роман Пихлер, 2010

DEEP-бэклог

Пихлер описал свойства хорошего бэклога аббревиатурой DEEP: Detailed appropriately (детализирован по мере приближения), Estimated (оценён), Emergent (меняется по мере обучения), Prioritized (упорядочен).

Свойство «emergent» особенно важно: бэклог не план, который выполняют, а живой список, который постоянно пересматривают.

МИРОВОЙ КЕЙС Basecamp, «Shape Up», 2019

Бэклог с нулевым ростом

Компания Basecamp отказалась от бэклога в привычном смысле. Идеи не накапливаются в очереди: на каждый цикл из шести недель команда заново выбирает, что делать, из проработанных предложений. Если идея важна, её снова предложат.

Подход радикален, но показывает: длинный бэклог создаёт долг внимания, а не ценность.

ПЛОХО

Бэклог-свалка

412 карточек, половина старше года.

Планирование 3 часа, ничего не найти.

ХОРОШО

Бэклог-айсберг

96 карточек по эпикам, готовые истории на 2 спринта.

Планирование 1,5 часа, приоритеты ясны.

8.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Готовых историй	на 1,5–3 спринта
Размер истории	не больше половины спринта одного человека
Чистка бэклога	раз в квартал
Карточек без движения больше года	0

8.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Бэклог-свалка	Потеря фокуса	Регулярная чистка
Детализация на полгода	Устаревшие требования	Айсберг
Баги вперемешку с фичами	Неясные приоритеты	Отдельная приоритизация
Истории на весь спринт	Риск не успеть	Дробить

ПРИМЕР

Финтех

Платёжный сервис закрыл 1 200 карточек старше 9 месяцев одним действием с автоматическим комментарием. За полгода клиенты и команда «вернули» 14 из них. Остальные 1 186, по словам продакта, «никогда не были работой — только тревогой».

8.7 Вопросы для самопроверки

- Какие уровни бывают в бэклоге?

- Что такое принцип айсберга?
- Как посчитать глубину готового бэклога?
- Какие критерии готовности истории к разработке?
- Почему старые карточки стоит закрывать?

ПРАКТИКА 08 после главы 08

Наведите порядок в бэклоге команды.

ЧТО СДЕЛАТЬ

01	Разложите карточки по уровням: тема, эпик, фича, история.	☐
02	Закройте карточки старше года без связи с целями.	☐
03	Проверьте истории ближайших 2 спринтов по критериям готовности.	☐
04	Посчитайте глубину готового бэклога по формуле (8.1).	☐
05	Договоритесь о квартальной чистке.	☐

РЕЗУЛЬТАТ Сравните время планирования до и после.

09 Приоритизация: RICE, WSJF, ICE, MoSCoW

После неё вы выбираете метод приоритизации под задачу, считаете RICE, WSJF и ICE, используете MoSCoW для объёма релиза и понимаете, когда методы помогают, а когда создают иллюзию объективности.

9.1 Четыре метода · 9.2 WSJF · 9.3 Кейс «Чека»: квартальная приоритизация · 9.4 MoSCoW для релиза · 9.5 Бенчмарки · 9.6 Анти-паттерны · 9.7 Вопросы для самопроверки

Приоритизация — главная работа продакта в delivery: ресурсы команды всегда меньше списка идей. Методы приоритизации не принимают решение за вас, но делают допущения видимыми: о них можно спорить, их можно проверить. Разные методы отвечают на разные вопросы, и выбор метода зависит от того, что именно вы сравниваете.

9.1 Четыре метода

МЕТОД	ФОРМУЛА / СУТЬ	КОГДА
RICE	Охват × Влияние × Уверенность / Усилия	Сравнение продуктовых инициатив с разным охватом
ICE	Влияние × Уверенность × Лёгкость	Быстрая сортировка гипотез и экспериментов
WSJF	Стоимость задержки / Длительность	Когда важна срочность и есть сроки
MoSCoW	Must / Should / Could / Won't	Объём конкретного релиза

9.2 WSJF

ФОРМУЛА 9.1 · WEIGHTED SHORTEST JOB FIRST

$$WSJF = \frac{BV + TC + RR}{JS} \tag{9.1}$$

где

BV — ценность для бизнеса и пользователей — баллы 1–20

TC — критичность по времени: падает ли ценность при задержке — баллы 1–20

RR — снижение риска или открытие возможностей — баллы 1–20

JS — размер работы — баллы 1–20

```
«Остатки v1»: (20 + 13 + 8) / 13 ≈ 3,2. Импорт из Excel: (5 + 3 + 1) / 5 = 1,8.
Автотесты кассы: (8 + 8 + 20) / 8 = 4,5 — первыми, хотя пользователи их не видят
```

WSJF из фреймворка SAFe основан на идее стоимости задержки: сначала делать то, что даёт наибольшую ценность в единицу времени. Сильная сторона — учёт срочности и снижения риска, которые RICE не видит.

9.3 Кейс «Чека»: квартальная приоритизация

На квартал у «Чека» было 9 кандидатов при ёмкости около 30 командо-недель. Ника посчитала RICE и WSJF и обсудила результаты с техлидами и продажами.

ИНИЦИАТИВА	RICE	WSJF	НЕДЕЛЬ	РЕШЕНИЕ
Остатки v1	3 500	3,2	9	да
Автотесты и стабильность кассы	2 900	4,5	8	да
Ускорение подключения точек	2 100	2,6	5	да
Инвентаризация (v1.1)	1 800	2,4	2	да
Импорт из Excel	600	1,8	2	нет
Остатки для сетей	900	1,5	6	следующий квартал
Новый дизайн кабинета	400	0,8	10	нет
Интеграция с маркетплейсом	700	1,2	6	нет

ИНИЦИАТИВА	RICE	WSJF	НЕДЕЛЬ	РЕШЕНИЕ
Тёмная тема кассы	150	0,6	2	нет

Методы почти совпали в порядке, кроме автотестов: RICE их недооценивал, потому что пользователи «не видят» тесты. WSJF учёл снижение риска — и автотесты встали вровень с главной фичей. Итог: 24 недели из 30, остальное — буфер на баги и инциденты.

9.4 MoSCoW для релиза

КАТЕГОРИЯ	«ОСТАТКИ V1»
Must — без этого релиз бессмыслен	Справочник, приёмка, списание при продаже, отчёт
Should — важно, но можно сдвинуть на неделю	Уведомление «заканчивается»
Could — если останется время	Фото товара в справочнике
Won't — точно не в этом релизе	Импорт, перемещения, сети

ПРАВИЛО

Не больше 60% объёма релиза в категории Must. Если всё «обязательно», у команды нет пространства для манёвра, и при первой проблеме срок сорвётся.

КЛАССИКА Дон Рейнертсен, «The Principles of Product Development Flow», 2009

Стоимость задержки

Рейнертсен показал, что большинство компаний не знают стоимости задержки своих проектов и поэтому плохо выбирают порядок работ. Он предложил делить стоимость задержки на длительность и делать первым то, что даёт наибольший результат на единицу времени.

Принцип лёг в основу WSJF и стал одной из главных идей экономики потока в разработке.

МИРОВОЙ КЕЙС практика продуктовых команд

Иллюзия объективности

Многие команды обнаруживали, что RICE «подгоняют»: продакт, которому нравится идея, ставит уверенность 100% и влияние 3. Метод превращается в обоснование уже принятого решения.

Противоядие — договориться о шкалах заранее, требовать данные для высокой уверенности и сравнивать оценки разных людей. Расхождение оценок — повод обсудить допущения, а не усреднить.

ПЛОХО

Метод как автомат

«По RICE автотесты внизу — не делаем».

Через месяц инцидент на кассах.

ХОРОШО

Методы как язык

RICE и WSJF рядом, расхождение обсуждено.

Автотесты в приоритете, решение понятно всем.

9.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Must в релизе	до 60% объёма
Буфер в квартальном плане	15–25%
Кандидатов на приоритизацию	в 2–3 раза больше ёмкости
Независимых оценщиков	2–3 для снижения смещения

9.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Подгонка оценок	Ложная объективность	Шкалы и данные заранее
Один метод для всего	Слепые зоны	RICE + WSJF
Всё Must	Нет манёвра	До 60%

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Технический долг вне приоритизации	Накопление рисков	WSJF учитывает снижение риска

ПРИМЕР

Медиа

Новостное приложение приоритизировало по ICE эксперименты с лентой. Команда заметила, что «лёгкость» систематически переоценивали на 40%. Ввели правило: лёгкость оценивает разработчик, влияние — аналитик, уверенность — по данным. Доля экспериментов, уложившихся в срок, выросла с 45 до 80%.

9.7 Вопросы для самопроверки

- Какой метод выбрать для сравнения инициатив, а какой — для объёма релиза?
- Как считается WSJF?
- Почему WSJF лучше учитывает техническую работу, чем RICE?
- Сколько Must должно быть в релизе?
- Как защититься от подгонки оценок?

ПРАКТИКА 09 после главы 09

Проведите квартальную приоритизацию двумя методами.

ЧТО СДЕЛАТЬ

01	Выпишите 8–12 кандидатов с оценкой усилий.	☐
02	Посчитайте RICE и WSJF по формуле (9.1).	☐
03	Сравните порядки и обсудите расхождения.	☐
04	Сформируйте план с буфером 15–25%.	☐
05	Для главного релиза разложите объём по MoSCoW.	☐

РЕЗУЛЬТАТ Запишите допущения по оценкам — через квартал проверьте их.

10 Планирование релизов и итераций

После неё вы планируете релизы от цели и ёмкости команды, используете историю скорости для прогноза, делаете план на итерацию реалистичным и пересчитываете прогноз, когда реальность расходится с планом.

10.1 Уровни планирования · 10.2 Прогноз по скорости · 10.3 Вероятностный прогноз · 10.4 Кейс «Чека»: план к выставке · 10.5 План итерации · 10.6 Бенчмарки · 10.7 Анти-паттерны · 10.8 Вопросы для самопроверки

Планирование релиза отвечает на вопросы: что войдёт, когда выйдет и с какой уверенностью. Планирование итерации — что команда сделает в ближайшие одну-две недели. Ошибка — строить план от желаемой даты и подгонять объём под неё. Правильно — от ёмкости команды и её истории, с вероятностным прогнозом вместо одной даты.

10.1 Уровни планирования

УРОВЕНЬ	ГОРИЗОНТ	ЧТО ПЛАНИРУЮТ	КТО
Дорожная карта	квартал — год	Темы и цели	PM, руководство
Релиз	4–12 недель	Объём эпика, дата с уверенностью	PM, техлид, команда
Итерация	1–2 недели	Конкретные истории	Команда, PM
День	день	Задачи	Команда

10.2 Прогноз по скорости

Скорость команды — сколько работы она завершает за итерацию. Одна средняя цифра обманчива: скорость колеблется. Честный прогноз использует диапазон: по худшим и лучшим итерациям за последние 6–8 спринтов.

ФОРМУЛА 10.1 · ПРОГНОЗ ЧИСЛА СПРИНТОВ

$$n = \frac{S}{v} \qquad (10.1)$$

где

S — оставшийся объём релиза — SP

v — скорость: пессимистичная, средняя и оптимистичная — SP/спринт

n — спринтов до релиза — спринтов

```
«Остатки v1»: S = 150 SP; скорость за 8 спринтов: мин. 26, среднее 34, макс. 40 → 5,8 / 4,4 / 3,75 спринта. При двухнедельных спринтах — от 7,5 до 11,5 недели; 9 недель — уверенность около 60%
```

10.3 Вероятностный прогноз

Более точный способ — моделирование Монте-Карло по истории пропускной способности: многократно случайно выбирать скорость из прошлых спринтов и смотреть, сколько спринтов нужно в каждой симуляции. Результат — распределение: «с вероятностью 50% к 8 неделям, с вероятностью 85% к 10».

10.4 Кейс «Чека»: план к выставке

Срок — выставка через 9 недель. Прогноз показал, что 150 SP с вероятностью около 60% успеют. Ника не стала давить на команду, а применила MoSCoW: Must — 108 SP, Should — 24, Could — 18. Must успевал с вероятностью 90%.

СПРИНТ	ПЛАН, SP	СДЕЛАНО	ОСТАЛОСЬ MUST	РЕШЕНИЕ
1	34	30	78	—
2	34	36	42	—
3	34	27	15	Зависимость от команды платежей задержала списание
4	34	38	0	Must готов, начаты Should
5 (неделя 9)	—	1 Should + 10 Could	—	Уведомления успели, фото товара — в v1.1

ПРАВИЛО

Давайте дату с уверенностью, а не одну дату: «с вероятностью 85% к 10 неделям». Пересчитывайте прогноз каждый спринт и сообщайте изменения сразу, а не за неделю до срока.

10.5 План итерации

ПЛАНИРОВАНИЕ СПРИНТА ЗА ЧАС

1. **Цель спринта**
   Одна фраза: «кассир видит остаток при продаже и он списывается».
2. **Ёмкость**
   Отпуска, дежурства, праздники: реальная ёмкость часто 70–80% от номинала.
3. **Выбор историй**
   Сверху бэклога под цель, до 80% ёмкости.
4. **Риски и зависимости**
   Что может помешать, кто снимает.
5. **Обязательство команды**
   Команда соглашается с целью, а не со списком.

КЛАССИКА Даниэль Ваканти, «Actionable Agile Metrics for Predictability», 2015

Не оценивайте — считайте

Ваканти предлагает прогнозировать сроки не по экспертным оценкам, а по исторической пропускной способности и времени цикла с помощью Монте-Карло. Такой прогноз учитывает реальную изменчивость работы.

Главный вывод — предсказуемость важнее скорости, а прогноз с вероятностью честнее обещания одной даты.

МИРОВОЙ КЕЙС Даниэль Канеман и Амос Тверски, 1979

Ошибка планирования

Психологи описали «ошибку планирования»: люди систематически недооценивают сроки собственных задач, даже зная, что прошлые похожие задачи заняли больше времени. Канеман позже приводил пример учебника, который его команда планировала написать за два года, а закончила через восемь.

Противоядие — «взгляд извне»: опираться на историю похожих проектов, а не на ощущение. Для команды это прошлая скорость.

ПЛОХО

Одна дата

«Выпустим 1 марта» без запаса.

На третьем спринте зависимость — срок сорван, доверие потеряно.

ХОРОШО

Диапазон и MoSCoW

«Must к выставке с вероятностью 90%, Should — 60%».

Зависимость поглощена, Must готов к сроку.

10.6 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
История скорости для прогноза	6–8 последних итераций
Загрузка спринта	до 80% ёмкости
Реальная ёмкость от номинала	70–80%
Пересчёт прогноза релиза	каждый спринт

10.7 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
План от желаемой даты	Срыв сроков	План от ёмкости
Одна дата без вероятности	Ложная уверенность	Диапазон
Загрузка 100%	Нет места для неожиданностей	До 80%
Новости о срыве в конце	Потеря доверия	Сообщать сразу

ПРИМЕР

Госсектор

Подрядчик госсервиса перешёл с обещаний конкретных дат на прогноз с вероятностью и еженедельным обновлением. Заказчик сначала сопротивлялся, но через два квартала признал, что «85% к дате» сбывались, а прежние «точные даты» — в трети случаев.

10.8 Вопросы для самопроверки

- Чем планирование релиза отличается от планирования итерации?
- Как сделать прогноз по скорости с диапазоном?
- Что такое прогноз Монте-Карло?
- Почему загрузка спринта не должна быть 100%?
- Что такое ошибка планирования?

ПРАКТИКА 10 после главы 10

Сделайте прогноз релиза с диапазоном.

ЧТО СДЕЛАТЬ

01	Соберите скорость за 6–8 последних спринтов.	☐
02	Посчитайте пессимистичный, средний и оптимистичный прогноз по формуле (10.1).	☐
03	Разложите объём по MoSCoW и посчитайте прогноз для Must.	☐
04	Сообщите стейкхолдерам дату с уверенностью.	☐
05	Пересчитывайте прогноз каждый спринт.	☐

РЕЗУЛЬТАТ Через релиз сравните прогноз с фактом и уточните метод.

11 Декомпозиция фич и задач

После неё вы разбиваете крупные фичи на вертикальные срезы, которые приносят ценность и могут выйти отдельно, используете шаблоны разбиения и понимаете, почему маленькие поставки быстрее и надёжнее больших.

11.1 Вертикальные и горизонтальные срезы
11.2 Шаблоны разбиения
11.3 Почему маленькие поставки лучше
11.4 Кейс «Чека»: «Остатки» по срезам
11.5 Бенчмарки
11.6 Анти-паттерны
11.7 Вопросы для самопроверки

Большая фича — большой риск: долго до первой обратной связи, трудно оценить, сложно протестировать, страшно выкатывать. Декомпозиция превращает её в последовательность маленьких поставок, каждая из которых работает сама по себе. Главный принцип — резать вертикально: каждый срез проходит через все слои (интерфейс, логика, данные) и даёт пользователю что-то законченное.

11.1 Вертикальные и горизонтальные срезы

	ГОРИЗОНТАЛЬНЫЙ	ВЕРТИКАЛЬНЫЙ
Как режут	По слоям: база, API, интерфейс	По ценности: маленький сценарий целиком
Пример	«Сделать таблицу остатков в БД»	«Кассир видит остаток товара при продаже»
Когда видна ценность	Когда готовы все слои	После каждого среза
Обратная связь	Поздно	Рано
Риск интеграции	В конце	Каждый срез

11.2 Шаблоны разбиения

По шагам сценария	Приёмка → продажа → отчёт: каждый шаг — отдельная поставка.
По вариантам данных	Сначала штучные товары, потом весовые, потом с модификаторами.
По правилам	Сначала списание без проверки отрицательного остатка, потом с запретом продажи.
По интерфейсам	Сначала в веб-кабинете, потом на кассе.
Простое → сложное	Ручной ввод до сканера, сканер до импорта.
Отложить производительность	Сначала работает, потом быстро на 10 000 позиций.
Спайк	Отдельное исследование технического вопроса с ограниченным временем.

11.3 Почему маленькие поставки лучше

ФОРМУЛА 11.1 · РИСК БОЛЬШОЙ ПОСТАВКИ

$$P_{\text{сбой}} = 1 - (1 - p)^k \tag{11.1}$$

где

- p — вероятность, что одно изменение содержит проблему — доля
- k — число изменений в поставке — шт.
- $P_{\text{сбой}}$ — вероятность, что поставка содержит хотя бы одну проблему — доля

«Чек»: p ≈ 0,02 на изменение. Релиз раз в 2 недели с 10 изменениями: 1 - $0{,}98^{10}$ ≈ 18% — ровно доля неудачных релизов. Ежедневная поставка 1–2 изменений: ≈ 2–4%, и найти причину легко

11.4 Кейс «Чека»: «Остатки» по срезам

Первый план команды был горизонтальным: две недели на схему данных, три на API, три на интерфейсы, неделя на интеграцию. Ника предложила разрезать фичу вертикально, чтобы уже через две недели показать работающий сценарий пилотным точкам.

СРЕЗ	ЦЕННОСТЬ	ПИЛОТ
1. Справочник и ручной ввод остатка в кабинете	Видно, что есть на складе	10 точек, неделя 2
2. Списание штучных товаров при продаже	Остаток меняется сам	10 точек, неделя 4
3. Приёмка со сканером	Быстрый ввод поставок	30 точек, неделя 5
4. Отчёт и уведомление «заканчивается»	Вовремя дозаказать	30 точек, неделя 7
5. Весовые товары и модификаторы	Кафе с напитками и выпечкой	100 точек, неделя 8
6. Нагрузка 10 000 позиций, полировка	Готовность к широкому запуску	все, неделя 9

Срез 2 у пилотных точек выявил то, чего не нашли бы до конца проекта: кофейни продают «капучино 300 мл», а списывать нужно молоко и зёрна по рецептуре. Срез 5 расширили рецептурами — и сделали это до выставки, а не после жалоб клиентов.

ПРАВИЛО

Каждый срез должен быть выпускаемым и проверяемым пользователем. Если после среза нечего показать клиенту, это не срез, а слой.

КЛАССИКА Билл Уэйк, 2003

INVEST

Уэйк сформулировал свойства хорошей пользовательской истории: Independent (независимая), Negotiable (обсуждаемая), Valuable (ценная), Estimable (оцениваемая), Small (маленькая), Testable (проверяемая).

Свойства V и S тянут в разные стороны: маленькое трудно сделать ценным. Вертикальная декомпозиция — способ получить и то и другое.

МИРОВОЙ КЕЙС Amazon, «двухпиццевые команды»

Двухнедельные срезы

В Amazon практикуют небольшие автономные команды и частые маленькие поставки: изменения выкатываются тысячи раз в день по всей компании. По описаниям инженеров, это стало возможным, когда крупные изменения начали резать на независимые части за флагами.

Маленькие изменения проще проверить, откатить и понять, что сломалось, — поэтому частота поставки растёт вместе с надёжностью.

ПЛОХО

По слоям

9 недель — и только потом первый работающий сценарий.

Рецептуры обнаружены после релиза, жалобы кофеен.

ХОРОШО

По срезам

Работающий сценарий через 2 недели у 10 точек.

Рецептуры найдены на неделе 4 и сделаны до выставки.

11.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Размер среза	1–2 недели работы команды
Размер истории	1–5 дней
Первый показ пользователям	через 1–2 недели после старта
Спайк	до 2–3 дней с явным вопросом

11.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Горизонтальные слои	Поздняя ценность и интеграция	Вертикальные срезы
Срез без пользователя	Нечего проверить	Пилот на каждом срезе
Сразу всё сложное	Долго, рискованно	Простое → сложное
Спайк без срока	Бесконечное исследование	Ограничение по времени

ПРИМЕР

Доставка еды

Сервис делал «групповой заказ» в офис. Вместо полной функции за два месяца первым срезом выпустили ссылку, по которой коллеги добавляют блюда в общую корзину, а оплачивает один. Через две недели 6% офисных заказов были групповыми — и команда узнала, что разделение оплаты нужно лишь в каждом пятом.

11.7 Вопросы для самопроверки

- Чем вертикальный срез отличается от горизонтального?
- Какие шаблоны разбиения вы знаете?
- Почему маленькие поставки надёжнее — что показывает формула (11.1)?
- Что такое INVEST?
- Что такое спайк и как его ограничить?

ПРАКТИКА 11 после главы 11

Разрежьте ближайшую крупную фичу на вертикальные срезы.

ЧТО СДЕЛАТЬ

01	Опишите фичу как сценарий пользователя.	☐
02	Примените 2–3 шаблона разбиения.	☐
03	Сформируйте 4–7 срезов, каждый с ценностью и способом проверки.	☐
04	Запланируйте пилот на первом срезе.	☐
05	Посчитайте риск поставки по формуле (11.1) для текущего и нового подхода.	☐

РЕЗУЛЬТАТ Покажите план команде и спросите, какой срез можно сделать ещё меньше.

12 User stories, Job Stories и Acceptance Criteria

После неё вы пишете пользовательские и job-истории, которые передают контекст и цель, формулируете критерии приёмки, по которым можно проверить результат, и используете формат «Дано — Когда — Тогда» для сложных правил.

12.1 Форматы историй · 12.2 Критерии приёмки · 12.3 Формат «Дано — Когда — Тогда» · 12.4 Кейс «Чека»: истории, которые вернули на доработку · 12.5 Бенчмарки · 12.6 Анти-паттерны · 12.7 Вопросы для самопроверки

История — короткое описание ценности с точки зрения пользователя, повод для разговора команды. Её задача — не заменить спецификацию, а зафиксировать, кому и зачем нужна работа. Критерии приёмки дополняют историю: они описывают, как проверить, что работа сделана. Вместе история и критерии позволяют команде договориться до начала разработки, а QA — проверить результат без догадок.

12.1 Форматы историй

ФОРМАТ	ШАБЛОН	ПРИМЕР
User story	Как [роль], я хочу [действие], чтобы [цель]	Как кассир, я хочу видеть остаток товара при пробитии, чтобы не продать то, чего нет
Job story	Когда [ситуация], я хочу [мотивация], чтобы [результат]	Когда покупатель просит товар, которого может не быть, я хочу сразу видеть остаток, чтобы не идти на склад
Техническая история	Чтобы [польза], нужно [изменение]	Чтобы касса работала с 10 000 позиций, нужно кэшировать справочник

Job story полезна, когда важна ситуация, а не роль: одна и та же роль ведёт себя по-разному в разных контекстах. Владелец кофейни утром при приёмке и вечером при закрытии смены — разные «работы».

КЛАССИКА Рон Джеффрис, 2001

Карточка, разговор, подтверждение

Джеффрис описал историю тремя C: Card (короткая запись на карточке), Conversation (разговор, в котором уточняются детали), Confirmation (критерии, по которым подтверждают выполнение).

Самая частая ошибка — считать карточку требованием. Карточка — напоминание о разговоре, а не замена ему.

12.2 Критерии приёмки

Проверяемые	Можно однозначно сказать «выполнено» или «нет».
О поведении, не о реализации	«Остаток уменьшается после продажи», а не «вызывается метод decrement».
С крайними случаями	Нулевой остаток, отрицательный, весовой товар, офлайн.
Короткие	3–7 критериев на историю; больше — признак, что историю надо делить.

12.3 Формат «Дано — Когда — Тогда»

```
История: списание при продаже
Сценарий 1: штучный товар
 Дано: остаток «Вода 0,5» = 12 шт.
 Когда: кассир продаёт 2 шт.
 Тогда: остаток = 10 шт. в кабинете не позже чем через 10 с
Сценарий 2: продажа в офлайне
 Дано: касса без интернета, остаток 10 шт.
 Когда: продано 3 шт., связь восстановилась
 Тогда: остаток = 7 шт., чеки синхронизированы без дублей
Сценарий 3: остаток закончился
 Дано: остаток = 0, настройка «разрешить продажу в минус» выключена
 Когда: кассир пробивает товар
 Тогда: предупреждение «Нет в наличии» и возможность продать с подтверждением
```

12.4 Кейс «Чека»: истории, которые вернули на доработку

Алина посчитала, сколько историй за два спринта QA вернул разработке из-за неясных требований. Из 46 историй вернули 11 — почти четверть. Ника и Алина разобрали причины и договорились о правиле: у каждой истории есть сценарии крайних случаев, и QA участвует в груминге.

ФОРМУЛА 12.1 · ДОЛЯ ВОЗВРАТОВ ПО ТРЕБОВАНИЯМ

$$R = \frac{n_{\text{возвр}}}{n_{\text{ист}}} \cdot 100\% \qquad (12.1)$$

где

$n_{\text{возвр}}$ — историй, возвращённых из-за неясных или неполных требований — шт.

$n_{\text{ист}}$ — всего историй за период — шт.

«Чек»: до правила 11 из 46 → 24%. Через два спринта после — 3 из 44 → 7%

ПРИЧИНА ВОЗВРАТА	СЛУЧАЕВ	ЧТО ИЗМЕНИЛИ
Не описан офлайн-режим	4	Офлайн — обязательный сценарий для кассы
Нет поведения при нуле и минусе	3	Крайние значения в шаблоне
Разное понимание «сразу»	2	Числа: «не позже 10 с»
Весовые товары	2	Тип товара — обязательный вопрос на груминге

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Писать критерии «работает корректно», «удобно», «быстро». Их нельзя проверить. Замените числами и наблюдаемым поведением: «за 2 с», «одним нажатием», «остаток в кабинете совпадает с кассой».

МИРОВОЙ КЕЙС Гойко Аджич, «Specification by Example», 2011

Спецификация примерами

Аджич собрал опыт десятков команд, которые описывают требования конкретными примерами, а примеры превращают в автоматические тесты. Такие «живые спецификации» не устаревают: если поведение меняется, тест падает.

Формат «Дано — Когда — Тогда» — самый распространённый способ записи таких примеров.

ПЛОХО

История без критериев

«Как кассир, хочу видеть остатки».

Офлайн, нули и весовые — догадки разработчика; возврат из QA.

ХОРОШО

История + сценарии

Цель, 3–5 сценариев с крайними случаями, числа.

Разработка и QA понимают одинаково.

12.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Критериев на историю	3–7
Возвраты из QA по требованиям	до 10%
Участие QA в груминге	обязательно
Сценарии крайних случаев	для каждой истории с данными

12.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Карточка как требование	Нет разговора	Три С
Непроверяемые критерии	Споры о готовности	Числа и поведение
Критерии о реализации	Ограничение решения	О поведении
Нет крайних случаев	Возвраты из QA	Ноль, минус, офлайн

ПРИМЕР

Финтех

В банке истории о переводах писали без критериев о лимитах и выходных. После введения шаблона «Дано — Когда — Тогда» с обязательными сценариями «лимит», «выходной», «недостаточно средств» число дефектов, найденных после релиза, сократилось на 40%, а сценарии стали автотестами.

12.7 Вопросы для самопроверки

- Чем user story отличается от job story?
- Что такое три С истории?
- Какими должны быть критерии приёмки?
- Как записать сценарий в формате «Дано — Когда — Тогда»?
- Как посчитать долю возвратов по требованиям?

ПРАКТИКА 12 после главы 12

Перепишите истории ближайшего спринта.

ЧТО СДЕЛАТЬ

01	Выберите 5 историй и проверьте, понятны ли роль, действие и цель.	☐
02	Для одной истории попробуйте формат job story.	☐
03	Допишите 3–7 проверяемых критериев.	☐
04	Для историй с данными — сценарии крайних случаев в формате «Дано — Когда — Тогда».	☐
05	Посчитайте долю возвратов по формуле (12.1) за последние спринты.	☐

РЕЗУЛЬТАТ Пригласите QA на следующий груминг.

13 Спецификации: когда нужен PRD, а когда достаточно Miro

После неё вы выбираете уровень документации под размер и риск задачи, пишете PRD, который читают, и не тратите дни на документ там, где хватит схемы на доске и разговора.

13.1 Уровни документации · 13.2 Выбор уровня · 13.3 Структура PRD · 13.4 Кейс «Чека»: PRD на шесть страниц · 13.5 Бенчмарки · 13.6 Анти-паттерны · 13.7 Вопросы для самопроверки

Документация в delivery нужна для одного: чтобы все участники одинаково понимали, что и зачем делается, и могли вернуться к этому позже. Слишком мало документации — каждый понимает по-своему. Слишком много — документ никто не читает, а через неделю он устаревает. Уровень документации выбирают по риску ошибки и числу участников, а не по привычке.

13.1 Уровни документации

УРОВЕНЬ	ФОРМАТ	КОГДА
Карточка	История с критериями в трекере	Небольшое изменение одной команды, понятный контекст
Доска	Схема флоу, стикеры, макеты на доске	Обсуждение вариантов, синхронизация в одной команде
Одностраничник	Проблема, цель, метрики, рамки, решение кратко	Фича на 1–3 недели
PRD	Полный документ требований	Крупная фича, несколько команд, внешние участники, высокая цена ошибки
Техдизайн	Архитектура, API, данные, риски	Сложные технические изменения; пишет техлид

13.2 Выбор уровня

ФОРМУЛА 13.1 · ИНДЕКС ПОТРЕБНОСТИ В ДОКУМЕНТАЦИИ

$$D = T \cdot R \cdot L \tag{13.1}$$

где

T — число вовлечённых команд и внешних участников: 1, 2, 3+ 1–3

R — цена ошибки: низкая 1, средняя 2, высокая 3 1–3

L — длительность: до 2 недель 1, до 6 недель 2, дольше 3 1–3

D — 1–3 карточка, 4–8 одностраничник, 9+ PRD –

«Остатки v1»: T = 3 (касса, кабинет, платежи + продажи), R = 3 (списание денег и товара), L = 3 → D = 27 — PRD. Кнопка «повторить чек»: 1 · 1 · 1 = 1 — карточка

13.3 Структура PRD

PRD, КОТОРЫЙ ЧИТАЮТ

1. **Проблема и почему сейчас**
   Полстраницы с данными: «31% ушедших точек называют причиной отсутствие учёта остатков».
2. **Цели и метрики**
   Outcome, база, цель, срок, защитные метрики.
3. **Пользователи и сценарии**
   Кто, в каких ситуациях, ключевые сценарии.
4. **Объём**
   Что входит, что нет и почему; MoSCoW.
5. **Решение**
   Флоу, ссылки на макеты, правила, крайние случаи.
6. **Нефункциональные требования**
   Скорость, офлайн, нагрузка, безопасность.

PRD, КОТОРЫЙ ЧИТАЮТ

7 **Запуск**
Пилот, раскатка, GTM, поддержка.

8 **Открытые вопросы и решения**
Журнал решений с датами.

13.4 Кейс «Чека»: PRD на шесть страниц

Прошлые PRD в «Чеке» были по 25–30 страниц, их писали неделями и не читали. Для «Остатков» Ника ограничила себя шестью страницами и вынесла детали в связанные артефакты: флоу — на доску, правила — в истории, архитектуру — в техдизайн Егора.

АРТЕФАКТ	ГДЕ	КТО ВЕДЁТ	ОБЪЁМ
PRD	Вики	Ника	6 страниц
Флоу и экраны	Доска + макеты	Дизайнер	3 сценария, 22 экрана
Истории и сценарии	Трекер	Команда + QA	54 истории
Техдизайн	Вики	Егор	4 страницы
Журнал решений	Раздел PRD	Ника	17 решений за проект

Журнал решений оказался самой полезной частью. Когда на шестой неделе продажи спросили, почему нет импорта, ответ был в записи от второй недели: «Импорт — v2: малые точки, 100–300 позиций; решение Ника + продажи, 14.01».

ПРАВИЛО

PRD — документ решений, а не описания каждого экрана. Детали живут в макетах и историях со ссылками из PRD. Всё, что можно увидеть в макете, в PRD не дублируется.

КЛАССИКА Amazon, «six-pager», 2004

Документ из шести страниц

Джефф Безос запретил презентации на совещаниях руководства и ввёл повествовательные документы до шести страниц, которые участники читают в тишине первые 20–30 минут встречи.

Связный текст заставляет автора продумать логику, а читателей — понять её до обсуждения. Ограничение объёма отсекает лишнее.

МИРОВОЙ КЕЙС Agile-манифест, 2001

Минимум документов

Одна из ценностей манифеста — «работающий продукт важнее исчерпывающей документации». Её часто читают как «документация не нужна», хотя авторы явно писали, что ценят и то и другое, но первое — больше.

На практике команды, полностью отказавшиеся от документов, теряют решения и контекст. Баланс — документировать решения и правила, а не пересказывать интерфейс.

ПЛОХО

PRD на 30 страниц

Описан каждый экран, неделя на написание.

Никто не дочитал, через месяц устарел.

ХОРОШО

PRD на 6 страниц + артефакты

Проблема, метрики, объём, решения; детали — по ссылкам.

Читают все, журнал решений отвечает на вопросы.

13.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Объём PRD	2–6 страниц
Время на PRD	1–3 дня
Обновление	по мере решений, журнал с датами
Читали PRD до старта	вся команда, ключевые стейкхолдеры

13.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
PRD для каждой мелочи	Потеря времени	Индекс потребности
Нет документа для крупной фичи	Разное понимание	PRD при высоком риске
Дублирование макетов текстом	Устаревание	Ссылки
Нет журнала решений	Вопросы повторяются	Решения с датами

ПРИМЕР

Логистика

Команда сервиса доставки договорилась: изменения до двух недель — только доска и истории, крупные — одностраничник, межкомандные — PRD. Время от идеи до старта разработки для малых изменений сократилось с 9 до 3 дней, а для крупных качество не пострадало.

13.7 Вопросы для самопроверки

- Какие уровни документации бывают и когда какой нужен?
- Как оценить потребность в документации?
- Из каких разделов состоит PRD?
- Зачем нужен журнал решений?
- Почему PRD не должен дублировать макеты?

ПРАКТИКА 13 после главы 13

Выберите уровень документации для трёх ближайших задач.

ЧТО СДЕЛАТЬ

01	Оцените каждую задачу по формуле (13.1).	☐
02	Для задачи с высоким индексом напишите PRD до 6 страниц.	☐
03	Вынесите детали в макеты и истории со ссылками.	☐
04	Заведите журнал решений.	☐
05	Попросите двух коллег прочитать PRD и пересказать цель.	☐

РЕЗУЛЬТАТ Если пересказы расходятся — перепишите раздел о проблеме и целях.

14 Техническое уточнение и grooming

После неё вы проводите регулярное уточнение бэклога так, чтобы к планированию истории были готовы, вовлекаете техлида и QA до начала работы и превращаете груминг из долгого совещания в короткую рабочую сессию.

14.1 Что происходит на груминге · 14.2 Техническое уточнение · 14.3 Ритм · 14.4 Кейс «Чека»: груминг на час вместо трёх · 14.5 Бенчмарки · 14.6 Анти-паттерны · 14.7 Вопросы для самопроверки

Груминг, или уточнение бэклога, — регулярная работа над историями до того, как они попадут в спринт: прояснить требования, найти риски, разбить крупное, оценить. Без него планирование превращается в трёхчасовое обсуждение того, что вообще нужно сделать, а разработка — в серию вопросов к продакту посреди спринта.

14.1 Что происходит на груминге

ДЕЙСТВИЕ	КТО ВЕДЁТ	РЕЗУЛЬТАТ
Объяснить цель и контекст истории	PM	Команда понимает зачем
Задать вопросы и найти крайние случаи	Команда, QA	Дополненные критерии
Оценить технические риски и зависимости	Техлид	Список рисков, спайки
Разбить крупное	Команда	Истории по размеру
Оценить	Команда	Оценка в SP или размерах
Отметить готовность	PM	История готова к спринту

14.2 Техническое уточнение

Для крупных фич до груминга нужна отдельная сессия с техлидом: какие компоненты затронуты, какие есть варианты реализации и их цена, где неизвестность. Её результат —

техдизайн и список спайков. Если пропустить этот шаг, команда узнаёт о сложности в середине спринта.

14.3 Ритм

НЕДЕЛЯ УТОЧНЕНИЯ

Пн **PM готовит**
Истории на следующий спринт: цель, черновик критериев, макеты.

Вт **Техуточнение**
PM + техлид + QA по крупным темам, 45 минут.

Ср **Асинхронно**
Команда читает истории и оставляет вопросы в трекере.

Чт **Груминг**
Вся команда, 60 минут, только вопросы и оценка.

Пт **Готово**
Истории с отметкой «готово к спринту».

14.4 Кейс «Чека»: груминг на час вместо трёх

Раньше команда «Остатков» встречалась раз в спринт на три часа и успевала разобрать 6–8 историй. Ника ввела недельный ритм с асинхронным этапом: вопросы задавались заранее в трекере, а на встрече обсуждали только спорное.

ПОКАЗАТЕЛЬ	БЫЛО	СТАЛО
Длительность груминга	3 ч раз в 2 недели	1 ч раз в неделю
Историй за сессию	6–8	10–14
Вопросов к PM посреди спринта	~25 за спринт	~6
Историй, взятых в спринт неготовыми	30%	5%
Длительность планирования	3 ч	1,5 ч

ФОРМУЛА 14.1 · ЭФФЕКТИВНОСТЬ ГРУМИНГА

$$E = \frac{n_{\text{гот}}}{h \cdot p} \qquad (14.1)$$

где

$n_{\text{гот}}$ — историй, доведённых до готовности — шт.

h — длительность встреч — ч

p — участников — чел.

E — готовых историй на человеко-час — шт./чел.-ч

Было: 7 историй / (3 ч · 9 чел.) ≈ 0,26. Стало: 2 сессии × 12 = 24 истории / (2 ч · 9 + 1,5 ч техуточнения · 3) ≈ 24 / 22,5 ≈ 1,07

ПРАВИЛО

На груминг приходят подготовленными: истории разосланы, вопросы заданы заранее. Встреча — для обсуждения спорного и оценки, а не для первого чтения требований.

КЛАССИКА Scrum Guide, 2020

Уточнение бэклога в Scrum

Руководство по Scrum описывает уточнение бэклога как постоянную деятельность, в которой элементы делятся, дополняются деталями и оцениваются. Конкретный формат не задан — команда выбирает сама.

Элементы, которые можно сделать за спринт, считаются готовыми к выбору на планировании. Готовность — договорённость команды, часто оформленная как Definition of Ready.

МИРОВОЙ КЕЙС практика BDD, Джордж Динвидди, 2009

Три амиго

Подход «трёх амиго» предлагает обсуждать каждую историю втроём: представитель бизнеса, разработчик и тестировщик. Каждый видит свои риски: бизнес — ценность, разработчик — реализацию, тестировщик — крайние случаи.

Короткая встреча втроём до разработки находит большинство вопросов, которые иначе всплыли бы на тестировании.

ПЛОХО

Трёхчасовой груминг

Первое чтение историй на встрече, 7 историй за сессию.

25 вопросов к PM посреди спринта.

ХОРОШО

Ритм с подготовкой

Техуточнение, асинхронные вопросы, час на спорное.

24 готовых истории за две недели, 6 вопросов посреди спринта.

14.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Время на уточнение	до 10% ёмкости команды
Груминг	45–60 минут, раз в неделю
Неготовых историй в спринте	до 10%
Вопросов к PM посреди спринта	единицы

14.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Первое чтение на встрече	Долго, поверхностно	Асинхронная подготовка
Без техлида и QA	Риски находятся поздно	Три амиго
Груминг раз в спринт	Перегрузка встречи	Еженедельно и коротко
Оценка без понимания	Неверные сроки	Сначала вопросы

ПРИМЕР

Saas

Команда B2B-сервиса добавила в трекер поле «вопросы к истории», обязательное для заполнения разработчиками за день до груминга. Средняя длительность груминга упала с 90 до 40 минут, а точность оценок выросла: отклонение факта от оценки сократилось с 60 до 25%.

14.7 Вопросы для самопроверки

- Что происходит на груминге и кто ведёт каждое действие?
- Зачем отдельное техническое уточнение для крупных фич?
- Как организовать недельный ритм уточнения?
- Как посчитать эффективность груминга?
- Что такое «три амиго»?

ПРАКТИКА 14 после главы 14

Перестройте груминг команды.

ЧТО СДЕЛАТЬ

01	Посчитайте эффективность текущего груминга по формуле (14.1).	☐
02	Введите асинхронный этап: истории и вопросы за день до встречи.	☐
03	Для крупной фичи проведите техуточнение с техлидом и QA.	☐
04	Сократите встречу до 60 минут на спорное и оценку.	☐
05	Через месяц сравните метрики.	☐

РЕЗУЛЬТАТ Попросите команду оценить новый формат на ретро.

15 Методологии: Scrum, Kanban, Scrumban

После неё вы понимаете принципы Scrum, Kanban и их гибрида, выбираете подход под тип работы команды и не путаете ритуалы с результатом.

15.1 Сравнение · 15.2 Scrum: суть · 15.3 Kanban: суть · 15.4 Кейс «Чека»: разные команды — разные подходы · 15.5 Бенчмарки · 15.6 Анти-паттерны · 15.7 Вопросы для самопроверки

Методология — договорённость команды о том, как организовать работу. Scrum строится вокруг коротких итераций с фиксированной целью. Kanban — вокруг непрерывного потока с ограничением незавершённой работы. Scrumban соединяет ритм Scrum с управлением потоком Kanban. Ни один подход не лучше вообще; важно, подходит ли он к типу работы и решает ли проблемы команды.

15.1 Сравнение

	SCRUM	KANBAN	SCRUMBAN
Единица планирования	Спринт 1–4 недели	Непрерывный поток	Короткие циклы планирования + поток
Роли	Product Owner, Scrum Master, разработчики	Не обязательны	Гибко
Изменения посреди цикла	Нежелательны	В любой момент	Через лимиты и приоритет
Ключевой механизм	Цель спринта, инкремент	Лимиты WIP, визуализация потока	Лимиты WIP + регулярные события
Метрики	Скорость, выполнение цели	Время цикла, пропускная способность	Обе
Подходит	Новая разработка с понятным горизонтом	Поддержка, операционка, поток заявок	Смешанная работа

15.2 Scrum: суть

События	Планирование спринта, ежедневный стендап, обзор, ретроспектива — внутри спринта.
Артефакты	Бэклог продукта, бэклог спринта, инкремент.
Цель спринта	Одна формулировка, ради которой команда работает две недели.
Определение готовности	Общие критерии, когда работа считается завершённой.

15.3 Kanban: суть

Kanban не предписывает ролей и событий. Его основа — визуализировать поток, ограничить незавершённую работу (WIP) на каждом этапе и улучшать поток по метрикам. Ограничение WIP — главный механизм: когда этап заполнен, команда не берёт новое, а помогает завершить начатое.

ФОРМУЛА 15.1 · ЗАКОН ЛИТТЛА

$$CT = \frac{WIP}{TH} \tag{15.1}$$

где

- CT — среднее время цикла, дн.
- WIP — среднее число элементов в работе, шт.
- TH — пропускная способность, шт./дн.

Команда поддержки «Чека»: WIP = 24, пропускная способность 2 задачи в день → CT = 12 дней. После лимита WIP = 10 при той же пропускной способности → 5 дней

15.4 Кейс «Чека»: разные команды — разные подходы

Все четыре команды «Чека» формально работали по Scrum, но у команды интеграций половина работы была срочными обращениями партнёров, и спринты постоянно ломались. Ника и Егор разобрали тип работы каждой команды.

КОМАНДА	ТИП РАБОТЫ	ПОДХОД	ПОЧЕМУ
Остатки	Новая фича с горизонтом 9 недель	Scrum, 2 недели	Цель спринта и ритм пилотов
Касса	Фичи + стабильность	Scrum, 2 недели	Плановая работа преобладает
Интеграции	50% срочных заявок партнёров	Kanban с классами обслуживания	Поток непредсказуем
Платформа	Инфраструктура + запросы команд	Scrumban	Плановые проекты и поток запросов

Через два месяца время ответа партнёрам у команды интеграций сократилось с 9 до 3 дней, а у остальных команд перестали «ломаться» спринты: срочные запросы больше не перебрасывались на них.

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Внедрять методологию ради методологии: «все команды по Scrum», потому что так принято. Начните с типа работы и проблем команды, затем выберите подход, который их решает, и проверьте по метрикам через 2–3 месяца.

КЛАССИКА — Кен Швабер и Джефф Сазерленд, 1995; Scrum Guide 2020

Scrum

Авторы представили Scrum на конференции OOPSLA в 1995 году, опираясь на статью Такеучи и Нонаки о «новой игре в разработку продуктов» 1986 года. Руководство по Scrum несколько раз сокращалось; редакция 2020 года занимает около 13 страниц.

Scrum — минимальный фреймворк. Всё, что сверх него (оценки в story points, диаграммы сгорания), — практики, а не обязательная часть.

МИРОВОЙ КЕЙС — Toyota, Тайити Оно, 1950-е; Дэвид Андерсон, 2010

Канбан из производства

Канбан появился в производственной системе Toyota как система карточек, которые сигнализируют о потребности в деталях: производить только то, что нужно следующему этапу.

Дэвид Андерсон адаптировал метод для разработки ПО в книге «Kanban» 2010 года, сделав акцент на ограничении незавершённой работы и эволюционных изменениях процесса.

ПЛОХО

Один подход на всех

Команда интеграций в Scrum, спринты ломаются срочными заявками.

Партнёры ждут 9 дней, команда недовольна.

ХОРОШО

Подход под тип работы

Kanban с классами обслуживания для потока заявок.

Партнёры ждут 3 дня, спринты других команд стабильны.

15.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Длина спринта	1–2 недели в продуктовой разработке
Лимит WIP	примерно число людей на этапе или меньше
Срок проверки нового подхода	2–3 месяца
Доля срочной работы, при которой Scrum ломается	от 30–40%

15.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Методология ради ритуалов	Встречи без пользы	От проблем команды
Kanban без лимитов WIP	Просто доска задач	Лимиты на этапах
Scrum с постоянными вбросами	Цели спринта не работают	Kanban или буфер
Нет метрик	Нельзя понять, стало ли лучше	Время цикла, скорость

ПРИМЕР

Медиа

Редакция цифрового издания пыталась работать по Scrum: две недели — слишком долго для новостей. Переход на Kanban с лимитом в 3 материала на редактора и классом «срочно» сократил время от идеи до публикации аналитических материалов с 8 до 4 дней.

15.7 Вопросы для самопроверки

- Чем Scrum отличается от Kanban по единице планирования и механизмам?
- Что такое лимит WIP и зачем он нужен?
- Как работает закон Литтла?
- Когда подходит Scrumban?
- Почему методологию выбирают по типу работы?

ПРАКТИКА 15 после главы 15

Проверьте, подходит ли методология вашей команде.

ЧТО СДЕЛАТЬ

01	Посчитайте долю плановой и срочной работы за 2 месяца.	☐
02	Измерьте WIP и пропускную способность, посчитайте время цикла по формуле (15.1).	☐
03	Сравните тип работы с таблицей подходов.	☐
04	Предложите одно изменение: лимит WIP, буфер, длину цикла.	☐
05	Договоритесь о метриках и сроке проверки.	☐

РЕЗУЛЬТАТ Обсудите результат на ретроспективе через 2–3 месяца.

16 Циклы разработки: sprint и flow

После неё вы управляете ритмом поставки в спринтах и в потоке, используете метрики цикла для прогнозов, отделяете ритм планирования от ритма релизов и понимаете, когда спринт мешает частым поставкам.

16.1 Два режима · 16.2 Отделить планирование от релиза · 16.3 Метрики цикла · 16.4 Кейс «Чека»: спринты остались, релизы — каждый день · 16.5 Бенчмарки · 16.6 Анти-паттерны · 16.7 Вопросы для самопроверки

Ритм работы команды задаётся двумя циклами, которые часто путают: циклом планирования (когда решаем, что делать) и циклом поставки (когда выпускаем). В классическом Scrum они совпадают: две недели — планирование, две недели — релиз. Но зрелые команды отделяют их: планируют раз в две недели, а выпускают каждый день или по готовности.

16.1 Два режима

	СПРИНТ	ПОТОК (FLOW)
Ритм	Фиксированный: 1–2 недели	Непрерывный
Обязательство	Цель спринта	Время цикла и пропускная способность
Релиз	Часто в конце спринта	По готовности каждого элемента
Прогноз	По скорости	По истории времени цикла
Сильная сторона	Фокус, ритм, синхронизация	Скорость, гибкость
Риск	Ожидание конца спринта, «мини-водопад»	Потеря фокуса, если нет целей

16.2 Отделить планирование от релиза

Если команда выпускает раз в спринт, работа, завершённая в первый день спринта, ждёт релиза до двух недель. Это прямые потери: пользователи не получают ценность, а изменения копятся и релиз становится рискованнее. Отделение релиза от спринта — один из самых дешёвых способов ускорить поставку.

ФОРМУЛА 16.1 · СРЕДНЕЕ ОЖИДАНИЕ РЕЛИЗА

$$W \approx \frac{T_{\text{р}}}{2} \tag{16.1}$$

где

$T_{\text{р}}$ — интервал между релизами — дн.

W — среднее время, которое готовая работа ждёт выкатки — дн.

«Чек»: релиз раз в 14 дней → ожидание ≈ 7 дней на каждую задачу. При ежедневных релизах — меньше дня

16.3 Метрики цикла

Время цикла	От начала работы над элементом до готовности; основа прогноза в потоке.
Время поставки	От запроса до выпуска пользователю; включает ожидание до начала работы.
Пропускная способность	Сколько элементов завершается за неделю.
Возраст элемента в работе	Сколько дней элемент уже в работе; сигнал застрявших задач.
85-й процентиль	«85% задач мы завершаем за N дней» — честнее среднего.

16.4 Кейс «Чека»: спринты остались, релизы — каждый день

Команда кассы планировала спринтами по две недели и выпускала в конце спринта. Егор с командой автоматизировали сборку и выкатку на 5% точек (подробнее в главе 35), после

чего релиз отвязали от спринта: каждая готовая история уходила в прод в тот же день за флагом.

ПОКАЗАТЕЛЬ	РЕЛИЗ РАЗ В СПРИНТ	РЕЛИЗ ПО ГОТОВНОСТИ
Изменений в релизе	10–14	1–3
Ожидание релиза	≈ 7 дней	< 1 дня
Доля неудачных релизов	18%	6%
Время цикла, 85-й процентиль	12 дней	6 дней
Планирование	раз в 2 недели	раз в 2 недели

Спринты как ритм планирования и ретроспектив сохранились: команде нравилась цель на две недели. Изменился только момент выпуска — и это дало больше, чем любая смена методологии.

ПРАВИЛО

Не держите готовую работу ради ритуала. Если история прошла проверки, её можно выпустить за флагом в тот же день — независимо от того, когда закончится спринт.

КЛАССИКА Эрик Рис, «The Lean Startup», 2011; Дон Рейнертсен, 2009

Маленькие партии

Рис и Рейнертсен показывают, что уменьшение размера партии — одно из самых сильных средств ускорения: маленькие партии быстрее проходят через систему, ошибки находятся раньше, а обратная связь приходит быстрее.

Релиз раз в спринт — большая партия. Релиз по готовности — партия из одного элемента.

МИРОВОЙ КЕЙС Etsy, 2010-е

Тысячи деплоев в день

Маркетплейс Etsy рассказывал, что перешёл от редких болезненных релизов к десяткам деплоев в день. Любой инженер мог выкатить изменение в первый рабочий день — в рамках онбординга.

Ключом стали маленькие изменения, автоматические проверки и прозрачный мониторинг. Частые релизы оказались безопаснее редких.

ПЛОХО

Релиз = конец спринта

14 изменений в релизе, каждый пятый откатывают.

Готовая работа ждёт неделю.

ХОРОШО

Релиз по готовности

1–3 изменения, флаги, выкатка на 5%.

Неудачных релизов 6%, ожидание меньше дня.

16.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Интервал релизов у зрелых команд	ежедневно или по готовности
Изменений в релизе	1–5
Прогноз в потоке	85-й процентиль времени цикла
Возраст элемента, требующий внимания	выше 85-го процентиля

16.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Релиз только в конце спринта	Ожидание и большие партии	Релиз по готовности
Тестирование в последние дни спринта	«Мини-водопад»	Тестирование по ходу
Среднее время цикла как прогноз	Слишком оптимистично	85-й процентиль
Поток без целей	Потеря фокуса	Цели на цикл планирования

ПРИМЕР

Едтех

Онлайн-школа заметила, что 60% работы в спринтах завершается в последние три дня. Анализ показал: тестирование начиналось только после окончания разработки всех задач. Команда ввела лимит «не больше двух историй в разработке одновременно» — работа стала завершаться равномерно, а время цикла снизилось на 40%.

16.7 Вопросы для самопроверки

- Чем цикл планирования отличается от цикла поставки?
- Сколько в среднем готовая работа ждёт релиза при интервале T?
- Какие метрики цикла вы знаете?
- Почему 85-й процентиль лучше среднего для прогноза?
- Как отделить релиз от спринта?

ПРАКТИКА 16 после главы 16

Оцените потери от привязки релиза к спринту.

ЧТО СДЕЛАТЬ

01	Посчитайте ожидание релиза по формуле (16.1).	☐
02	Соберите время цикла за 2 месяца и найдите 85-й процентиль.	☐
03	Найдите элементы, «застрявшие» дольше процентиля.	☐
04	Обсудите с техлидом, что нужно для релиза по готовности.	☐
05	Начните с одной команды или одного типа изменений.	☐

РЕЗУЛЬТАТ Сравните долю неудачных релизов до и после.

17 Управление зависимостями

После неё вы выявляете зависимости между командами и внешними участниками заранее, визуализируете их, снижаете число через архитектуру и договорённости и управляете оставшимися так, чтобы они не срывали сроки.

17.1 Виды зависимостей · 17.2 Почему зависимости опасны · 17.3 Управление зависимостями · 17.4 Кейс «Чека»: третий спринт · 17.5 Бенчмарки · 17.6 Анти-паттерны · 17.7 Вопросы для самопроверки

Зависимость — ситуация, когда работа одной команды не может завершиться без работы другой: API платёжной команды, согласование юристов, интеграция партнёра. Каждая зависимость — очередь, в которой работа ждёт. Чем больше зависимостей в фиче, тем ниже вероятность уложиться в срок: даже если каждая надёжна, их сочетание рискованно.

17.1 Виды зависимостей

ВИД	ПРИМЕР В «ЧЕКЕ»	КАК СНИЖАТЬ
Техническая между командами	Списание остатков требует изменения API платежей	Контракт API, моки, совместная работа
Знания	Только Егор знает модуль синхронизации	Парная работа, документация
Внешняя	Сертификация кассового ПО у оператора	Ранний старт, буфер
Организационная	Согласование юристов на тексты договора	Регулярные слоты, шаблоны
Данные	Справочник товаров ведёт другая команда	Владелец данных, договорённость о формате

17.2 Почему зависимости опасны

ФОРМУЛА 17.1 · ВЕРОЯТНОСТЬ УСПЕТЬ ПРИ ЗАВИСИМОСТЯХ

$$P = \Pi\, p_{\text{i}} \tag{17.1}$$

где

p_{i} — вероятность, что i-я зависимость будет закрыта вовремя — доля

P — вероятность, что все зависимости будут закрыты вовремя — доля

«Остатки v1»: API платежей 0,85, сертификация 0,9, справочник 0,9, юристы 0,95 → P ≈ 0,85 · 0,9 · 0,9 · 0,95 ≈ 0,65. Даже при надёжных партнёрах шанс без задержки — две трети

17.3 Управление зависимостями

ПЯТЬ ШАГОВ

1. **Выявить**
   На техуточнении: кто ещё должен что-то сделать, чтобы фича заработала?
2. **Визуализировать**
   Доска зависимостей: кто, что, к какой дате, статус.
3. **Убрать**
   Можно ли сделать самим, через существующий API, с моком?
4. **Договориться**
   Дата, владелец, критерий готовности — письменно.
5. **Следить**
   Еженедельная сверка, эскалация при риске.

17.4 Кейс «Чека»: третий спринт

На третьем спринте списание остатков застряло: команде платежей нужно было изменить API, а у неё был свой квартальный план. Зависимость выявили ещё на техуточнении, но договорились устно. Ника разобрала ситуацию и на будущее ввела доску зависимостей.

ЗАВИСИМОСТЬ	ВЛАДЕЛЕЦ	ДАТА	СТАТУС	ДЕЙСТВИЕ
API платежей: поле остатков	Команда платежей	нед. 5	задержка 1 неделя	Мок для разработки, разработчик «Остатков» помог с PR
Сертификация кассы	Оператор + Егор	нед. 7	в срок	Подали заявку на неделе 2
Формат справочника	Команда кабинета	нед. 3	в срок	Контракт данных подписан
Тексты оферты «Склад+»	Юристы	нед. 8	в срок	Шаблон с прошлых тарифов

Задержку API компенсировали: разработка шла на моке, а разработчик «Остатков» сам сделал изменение в коде платежей, которое команда платежей проверила. Такой «внутренний опенсорс» стал правилом для небольших изменений в чужих сервисах.

ПРАВИЛО

Устная договорённость о зависимости — не договорённость. Запишите владельца, дату и критерий готовности, сверяйте статус каждую неделю и эскалируйте, как только риск появился, а не когда срок прошёл.

КЛАССИКА Мелвин Конвей, 1968

Закон Конвея

Конвей заметил, что организации проектируют системы, повторяющие структуру своих коммуникаций. Если две команды должны договариваться, в системе появится граница между их частями.

Обратное тоже верно: чтобы уменьшить зависимости, меняют структуру команд и архитектуру так, чтобы фича целиком делалась одной командой.

МИРОВОЙ КЕЙС PayPal и другие, 2010-е

Внутренний опенсорс

Ряд компаний практикует «inner source»: команды могут вносить изменения в код других команд через пул-реквесты, а владельцы проверяют и принимают их. Это снимает очередь «ждём, пока у них дойдут руки».

Практика требует хороших тестов и понятных правил ревью, но резко сокращает время ожидания по межкомандным зависимостям.

ПЛОХО

Устная договорённость

«Платежи обещали API к пятому спринту».

Задержка обнаружена в день, когда API нужен.

ХОРОШО

Доска зависимостей

Владелец, дата, критерий, еженедельная сверка, мок и помощь с PR.

Задержка в неделю не сдвинула срок.

17.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Внешних зависимостей на фичу	чем меньше, тем лучше; больше 3 — высокий риск
Сверка статуса	еженедельно
Старт внешних процессов (сертификация, юристы)	в первые 2 недели проекта
Фичи, делаемые одной командой целиком	цель — большинство

17.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Зависимости находятся в разработке	Срыв сроков	Выявлять на техуточнении
Устные договорённости	Неожиданные задержки	Письменно с датой
Ждать вместо действовать	Простой	Моки, помощь, внутренний опенсорс
Структура команд против архитектуры	Постоянные зависимости	Закон Конвея

ПРИМЕР

Маркетплейс

Крупный маркетплейс посчитал, что средняя фича зависит от 4,2 команд, а 60% задержек вызваны ожиданием. После реорганизации в команды по клиентским сценариям среднее число зависимостей снизилось до 1,8, время поставки — на треть.

17.7 Вопросы для самопроверки

- Какие виды зависимостей бывают?
- Почему несколько надёжных зависимостей вместе дают высокий риск?
- Из каких шагов состоит управление зависимостями?
- Что говорит закон Конвея?
- Что такое внутренний опенсорс?

ПРАКТИКА 17 после главы 17

Составьте карту зависимостей ближайшей фичи.

ЧТО СДЕЛАТЬ

01	Выпишите все зависимости по видам.	☐
02	Оцените вероятность закрытия каждой вовремя и посчитайте общую по формуле (17.1).	☐
03	Для каждой подумайте, можно ли её убрать.	☐
04	Оформите договорённости: владелец, дата, критерий.	☐
05	Заведите еженедельную сверку.	☐

РЕЗУЛЬТАТ Внешние процессы запускайте в первую же неделю.

18 Роль PM и взаимодействие с командой разработки

После неё вы выстраиваете с командой отношения партнёрства, а не заказчика и исполнителей, участвуете в ритуалах там, где добавляете ценность, принимаете решения быстро и защищаете команду от хаоса приоритетов.

18.1 Где продакт нужен
18.2 Защита от хаоса приоритетов
18.3 Кейс «Чека»: правила работы с командой
18.4 Бенчмарки
18.5 Анти-паттерны
18.6 Вопросы для самопроверки

Команда разработки — не подрядчик продакта. Лучшие решения появляются, когда инженеры понимают проблему и участвуют в поиске решения, а продакт понимает технические ограничения и цену решений. Задача продакта — дать контекст и цели, быстро отвечать на вопросы, защищать команду от постоянной смены приоритетов и честно говорить о том, что знает и чего нет.

18.1 Где продакт нужен

СОБЫТИЕ	РОЛЬ PM	НУЖЕН ЛИ
Планирование	Цель спринта, приоритеты, ответы на вопросы	обязательно
Груминг	Контекст историй, критерии	обязательно
Ежедневный стендап	Слушать, отвечать на блокеры	по желанию команды
Обзор (демо)	Принимать работу, приглашать стейкхолдеров	обязательно
Ретроспектива	Участник, не руководитель	обычно да
Техническое ревью кода	—	нет

18.2 Защита от хаоса приоритетов

Самое разрушительное для команды — постоянные вбросы «срочного» посреди спринта.

Каждое переключение стоит времени на вход в контекст, а незавершённая работа копится. Продакт — фильтр: он решает, что действительно срочно, а что подождёт до следующего планирования.

ФОРМУЛА 18.1 · ПОТЕРИ НА ПЕРЕКЛЮЧЕНИЯХ

$$L = n_{\text{вбр}} \cdot t_{\text{пер}} \cdot k \tag{18.1}$$

где

- $n_{\text{вбр}}$ — вбросов в спринт — шт.
- $t_{\text{пер}}$ — потери на переключение одного человека: уход из контекста и возврат — ч
- k — людей, которых затрагивает вброс — чел.
- L — потерянные часы за спринт — ч

Команда кассы до правил: 9 вбросов × 3 ч × 2 чел. = 54 ч за спринт — почти 7 человеко-дней из 80. После правила «срочно — только инциденты P1–P2 и решение PM»: 2 вброса → 12 ч

18.3 Кейс «Чека»: правила работы с командой

Ника и команда «Остатков» договорились о правилах на первой ретроспективе. Их записали на странице команды и пересматривали раз в квартал.

ПРАВИЛО	ЗАЧЕМ
Все запросы к команде — через PM	Один фильтр приоритетов
Срочно посреди спринта — только инциденты и решение PM с объяснением	Защита цели спринта
PM отвечает на вопросы в течение 4 рабочих часов	Не блокировать разработку
Разработчики раз в месяц на звонке с клиентом	Понимание проблемы из первых рук
Технический долг — 20% ёмкости каждого спринта	Не копить риски
Решения по реализации — за командой	Доверие и ответственность

Правило о звонках с клиентами оказалось неожиданно сильным. Разработчик, послушавший, как владелец кофейни вечером вручную пересчитывает молоко, сам предложил списание по рецептурам — ещё до того, как это всплыло на пилоте.

ПРАВИЛО

Приносите команде проблему и цель, а не готовое решение. Инженеры, понимающие, зачем делают работу, предлагают решения проще и дешевле, чем продакт может придумать сам.

КЛАССИКА — Марти Каган, «Inspired», 2017

Команды-миссионеры и команды-наёмники

Каган различает команды-наёмники, которые делают то, что им сказали, и команды-миссионеры, которые верят в продукт и чувствуют ответственность за результат. Миссионеры создают лучшие продукты.

Чтобы команда стала миссионерской, ей нужно видеть пользователей, понимать цели бизнеса и иметь полномочия решать, как достичь результата.

МИРОВОЙ КЕЙС — Google, проект «Аристотель», 2015

Психологическая безопасность

Google изучила 180 команд, чтобы понять, что отличает эффективные. Главным фактором оказалась психологическая безопасность — уверенность, что можно задать вопрос, признать ошибку или предложить идею без страха наказания.

Для продакта вывод практический: если команда не задаёт вопросов и не спорит с приоритетами, это не согласие, а опасный сигнал.

ПЛОХО

Продакт-заказчик

«Вот ТЗ, сделайте к пятнице», вбросы посреди спринта.

Команда исполняет, не предлагает, теряет 54 ч на переключениях.

ХОРОШО

Продакт-партнёр

Проблема и цель, фильтр срочного, ответы за 4 часа, клиенты на звонках.

Команда предлагает решения сама.

18.4 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Вбросов посреди спринта	0–2
Время ответа PM	до 4 рабочих часов

ПАРАМЕТР	ОРИЕНТИР
Доля ёмкости на техдолг	15–20%
Контакт инженеров с клиентами	хотя бы раз в месяц

18.5 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Готовое решение вместо проблемы	Слабые решения	Проблема и цель
Вбросы от всех	Переключения, срывы	Один фильтр — PM
Медленные ответы	Простой или догадки	SLA на ответы
Нет времени на техдолг	Накопление рисков	Фиксированная доля

ПРИМЕР

Телеком

В команде мобильного приложения оператора руководители разных отделов напрямую писали разработчикам с «маленькими просьбами». Введение единого входа через продакта сократило вбросы с 15 до 3 за спринт, а выполнение целей спринта выросло с 40 до 85%.

18.6 Вопросы для самопроверки

- На каких событиях продакт обязателен, а где по желанию команды?
- Как посчитать потери на переключениях?
- Какие правила защищают команду от хаоса приоритетов?
- Чем команда-миссионер отличается от команды-наёмника?
- Что такое психологическая безопасность и почему она важна?

ПРАКТИКА 18 после главы 18

Договоритесь с командой о правилах работы.

ЧТО СДЕЛАТЬ

01	Посчитайте вбросы за последние 2 спринта и потери по формуле (18.1).	☐
02	Предложите правила: вход запросов, срочное, время ответа, техдолг.	☐
03	Обсудите и запишите их на ретроспективе.	☐
04	Организуйте звонок инженеров с клиентом.	☐
05	Через квартал пересмотрите правила.	☐

РЕЗУЛЬТАТ Спросите команду на ретро, что вам стоит делать иначе.

19 Подходы к QA: shift-left, автоматизация, ручные проверки

После неё вы понимаете, как переносить качество на ранние этапы, где автоматизация окупается, а где нужны ручные и исследовательские проверки, и как продакт влияет на качество, не будучи тестировщиком.

19.1 Стоимость дефекта по этапам · 19.2 Практики shift-left · 19.3 Автоматизация и ручные проверки · 19.4 Кейс «Чека»: от регресса на три дня · 19.5 Бенчмарки · 19.6 Анти-паттерны · 19.7 Вопросы для самопроверки

Качество нельзя «добавить» в конце, проверив готовый продукт. Если требования неясны, дефект закладывается до первой строки кода, а тестирование лишь находит его — поздно и дорого. Современный подход к качеству — shift-left: перенос проверок как можно левее по процессу, к требованиям, дизайну и разработке. Тестировщик из «последнего фильтра» становится участником всего цикла.

19.1 Стоимость дефекта по этапам

ФОРМУЛА 19.1 · РОСТ СТОИМОСТИ ИСПРАВЛЕНИЯ

$$C_{\text{к}} \approx C_{\text{о}} \cdot m^{k} \tag{19.1}$$

где

- $C_{\text{о}}$ — стоимость исправления на этапе требований — ч
- m — множитель между этапами, на практике 3–5 — –
- k — этап: требования 0, разработка 1, тестирование 2, прод 3 — –

«Чек», дефект «офлайн-продажи дублируют списание»: в требованиях — 1 ч обсуждения; в разработке — 4 ч; в тестировании — 1,5 дня; в проде — 2 недели с исправлением данных у 300 точек и компенсациями

19.2 Практики shift-left

Три амиго на груминге	PM, разработчик, QA обсуждают историю и крайние случаи до разработки.
Критерии приёмки как тесты	Сценарии «Дано — Когда — Тогда» превращаются в автотесты.
Ревью кода и статический анализ	Ошибки находятся до сборки.
Тесты пишет разработчик	Модульные и интеграционные тесты — часть определения готовности.
Тестирование по ходу спринта	Каждая история проверяется сразу, а не в конце.

19.3 Автоматизация и ручные проверки

ПОДХОД	КОГДА НУЖЕН	ПРИМЕР В «ЧЕКЕ»
Автоматические регрессионные тесты	Повторяющиеся проверки, критичные сценарии	Продажа, возврат, списание, синхронизация
Ручные сценарные проверки	Новые сценарии, сложная настройка оборудования	Подключение нового сканера
Исследовательское тестирование	Поиск неожиданного, оценка удобства	Час «сломай кассу» перед релизом
Тестирование на реальном оборудовании	Аппаратные особенности	Ферма из 12 моделей планшетов и принтеров

19.4 Кейс «Чека»: от регресса на три дня

До перемен релиз в «Чеке» сопровождался трёхдневным ручным регрессом: 4 тестировщика проходили 620 сценариев. Алина предложила план на квартал — автоматизировать критичное, ручное оставить для нового и исследовательского.

ПОКАЗАТЕЛЬ	ДО	ЧЕРЕЗ КВАРТАЛ
Ручных регресс-сценариев	620	140
Автотестов критичных сценариев	45	410
Время регресса	3 дня	40 минут авто + 3 ч исследовательского
Дефектов, найденных в проде за квартал	34	12
Доля дефектов, найденных до тестирования (груминг, ревью)	10%	38%

Самый неожиданный результат — рост доли дефектов, найденных до тестирования. Алина и её команда стали участвовать в груминге и находили проблемы в требованиях: в 38% случаев дефект ловили ещё на уровне истории.

ПРАВИЛО

Продакт влияет на качество больше, чем кажется: ясные критерии приёмки, крайние случаи в истории и время на автотесты в плане — это вклад PM. Если в спринте нет времени на тесты, качество уже снижено.

КЛАССИКА У. Эдвардс Деминг, «Out of the Crisis», 1982

Качество встраивается, а не проверяется

Один из 14 принципов Деминга: перестаньте полагаться на массовый контроль для достижения качества; встраивайте качество в продукт с самого начала. Контроль в конце только сортирует брак, но не предотвращает его.

Shift-left в разработке ПО — прямое применение этого принципа: дефект дешевле не допустить, чем найти.

МИРОВОЙ КЕЙС Microsoft, 2014

Тестировщики внутри команд

Microsoft перестроила инженерный процесс, отказавшись от отдельной роли тестировщиков в ряде подразделений: качество стало ответственностью разработчиков, а специалисты по качеству — консультантами и создателями инструментов.

Переход был болезненным, но ускорил поставку. Важный вывод — роль QA меняется, а не исчезает: от ручных проверок к стратегии качества и автоматизации.

ПЛОХО

QA — последний фильтр

Три дня ручного регресса перед каждым релизом.

34 дефекта в проде за квартал.

ХОРОШО

Качество по всему циклу

QA на груминге, автотесты критичного, исследовательские сессии.

12 дефектов в проде, регресс за 40 минут.

19.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Автоматизация критичных сценариев	близко к 100%
Время автоматического регресса	до часа
Доля дефектов до тестирования	от 30%
Исследовательское тестирование	сессии по 60–90 минут на крупную фичу

19.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
QA только в конце	Дорогие дефекты	Shift-left
Автоматизировать всё	Хрупкие тесты, затраты	Критичное и повторяющееся
Нет времени на тесты в плане	Скрытое снижение качества	Тесты — часть готовности
Только ручной регресс	Медленные релизы	Автоматизация регресса

ПРИМЕР

Финтех

Банк ввёл правило: история не принимается в спринт без сценариев, согласованных с QA. За полгода дефекты в требованиях, найденные после релиза, сократились на 55%, а время на тестирование одной истории — на треть: тестировщики заранее знали, что проверять.

19.7 Вопросы для самопроверки

- Что такое shift-left?
- Как растёт стоимость дефекта по этапам?
- Когда нужна автоматизация, а когда ручные проверки?
- Как продакт влияет на качество?
- Как меняется роль QA в современных командах?

ПРАКТИКА 19 после главы 19

Оцените, где ваша команда находит дефекты.

ЧТО СДЕЛАТЬ

01	Соберите дефекты за квартал и отметьте этап обнаружения.	☐
02	Посчитайте стоимость типичного дефекта по этапам по формуле (19.1).	☐
03	Найдите 10 критичных сценариев без автотестов.	☐
04	Пригласите QA на груминг.	☐
05	Заложите время на автотесты в следующий спринт.	☐

РЕЗУЛЬТАТ Повторите подсчёт через квартал.

20 Типы тестов: unit, integration, end-to-end, нагрузочные

После неё вы понимаете, что проверяет каждый уровень тестов, почему нужна пирамида, а не «рожок мороженого», и задаёте правильные вопросы о тестах, когда принимаете решение о готовности к релизу.

20.1 Уровни тестов
20.2 Пирамида тестов
20.3 Кейс «Чека»: нагрузочный тест, который спас выставку
20.4 Бенчмарки
20.5 Анти-паттерны
20.6 Вопросы для самопроверки

Продакту не нужно писать тесты, но нужно понимать, что они проверяют. «Всё протестировано» может значить что угодно: от сотни модульных тестов, которые не видят интеграцию с платежами, до одного ручного прогона на тестовом стенде. Понимание уровней тестов позволяет задать правильные вопросы перед релизом и оценить риск.

20.1 Уровни тестов

УРОВЕНЬ	ЧТО ПРОВЕРЯЕТ	СКОРОСТЬ	ПРИМЕР В «ЧЕКЕ»
Модульные (unit)	Отдельную функцию или класс	миллисекунды	Расчёт остатка после продажи весового товара
Интеграционные	Взаимодействие компонентов, базы, очередей	секунды	Чек записывается в базу и уходит в очередь синхронизации
Контрактные	Соответствие API договорённости между сервисами	секунды	API платежей возвращает поле остатков в нужном формате
End-to-end	Сценарий пользователя через всю систему	минуты	Приёмка → продажа → остаток в кабинете
Нагрузочные	Поведение под нагрузкой	минуты — часы	10 000 позиций, 50 чеков в минуту на точку
Исследовательские и приёмочные	Удобство, неожиданное, соответствие цели	часы	Сессия «сломай кассу», пилот

20.2 Пирамида тестов

Пирамида — модель распределения тестов: много быстрых модульных в основании, меньше интеграционных в середине, немного медленных end-to-end наверху. Обратная форма — «рожок мороженого», где большинство проверок ручные и end-to-end, — даёт медленные и хрупкие тесты, которые часто падают без причины.

ФОРМУЛА 20.1 · ВРЕМЯ ПРОГОНА НАБОРА ТЕСТОВ

$$T = \Sigma\, n_{\mathrm{i}} \cdot t_{\mathrm{i}} \,/\, p \qquad (20.1)$$

где

n_{i} — число тестов уровня i — шт.

t_{i} — среднее время одного теста уровня i — с

p — параллельность прогона — потоков

```
«Чек»: 3 200 unit × 0,05 с + 420 интеграционных × 2 с + 90 e2e × 40 с = 160 + 840 +
3 600 = 4 600 с; при 8 потоках ≈ 10 минут. Если бы те же сценарии проверялись только
e2e — 3 710 × 40 с / 8 ≈ 5 часов
```

20.3 Кейс «Чека»: нагрузочный тест, который спас выставку

За три недели до выставки Егор настоял на нагрузочном тесте «Остатков», хотя все модульные и e2e-тесты проходили. Тест имитировал крупную точку: 10 000 позиций, 50 чеков в минуту, синхронизацию каждые 10 секунд.

СЦЕНАРИЙ	ЦЕЛЬ	ПЕРВЫЙ ПРОГОН	ПОСЛЕ ИСПРАВЛЕНИЙ
Время проведения чека	≤ 2,1 с	3,8 с на 10 000 позиций	1,9 с
Синхронизация остатков	≤ 10 с	до 45 с в пике	6 с
Потеря данных при обрыве связи	0	2 чека из 1 000	0
Потребление памяти кассой	≤ 500 МБ	1,2 ГБ	380 МБ

Без нагрузочного теста крупные точки получили бы кассу, которая тормозит в час пик, — ровно ту проблему, которая вызывает 19% ухода клиентов. Исправление заняло неделю и уложилось в буфер.

ПРАВИЛО

Перед релизом спросите не «всё протестировано?», а «какие уровни тестов покрывают критичные сценарии и проверена ли нагрузка, как у крупных клиентов?». Ответ покажет реальный риск.

КЛАССИКА — Майк Кон, «Succeeding with Agile», 2009

Пирамида тестов

Кон предложил пирамиду с тремя уровнями: модульные тесты в основании, сервисные в середине и тесты интерфейса наверху. Чем выше уровень, тем тесты медленнее, дороже и хрупче, поэтому их должно быть меньше.

Позже модель уточняли — добавляли контрактные тесты, «трофей тестов» для фронтенда, — но идея баланса быстрых и медленных проверок осталась.

МИРОВОЙ КЕЙС — многочисленные примеры электронной коммерции

Сбой в чёрную пятницу

Ежегодно в дни распродаж часть интернет-магазинов падает под нагрузкой: сайт недоступен часами, заказы теряются. Разбор таких случаев обычно показывает, что функциональные тесты проходили, а нагрузку, сравнимую с пиком, не моделировали.

Нагрузочное тестирование кажется дорогим, пока не сравнишь его с выручкой, потерянной за час простоя в пиковый день.

ПЛОХО

«Тесты зелёные»

Модульные и e2e проходят — выпускаем.

Касса тормозит у крупных точек в час пик.

ХОРОШО

Вопросы по уровням

Какие уровни покрывают критичное? Проверена ли нагрузка?

Нагрузочный тест нашёл 4 проблемы до релиза.

20.4 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Соотношение unit / интеграционных / e2e	примерно 70 / 20 / 10
Прогон тестов на изменение	до 10–15 минут

ПАРАМЕТР	ОРИЕНТИР
Нестабильных (flaky) тестов	до 1–2%
Нагрузочный тест	перед крупными релизами и пиками

20.5 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
«Рожок мороженого»	Медленно и хрупко	Пирамида
Нет нагрузочных тестов	Сбои в пике	Моделировать крупных клиентов
Нестабильные тесты игнорируют	Перестают верить тестам	Чинить или удалять
Покрытие кода как цель	Тесты ради процента	Покрытие критичных сценариев

ПРИМЕР

Доставка

Сервис доставки продуктов перед новогодним пиком провёл нагрузочный тест с утроенным трафиком. Оказалось, что сервис расчёта слотов доставки отвечал 12 секунд при двукратной нагрузке. Кэширование и масштабирование исправили проблему за неделю, а в пик сервис выдержал 2,7-кратный рост заказов.

20.6 Вопросы для самопроверки

- Что проверяет каждый уровень тестов?
- Что такое пирамида тестов и чем плох «рожок мороженого»?
- Как оценить время прогона набора тестов?
- Почему нагрузочный тест нужен, даже если функциональные проходят?
- Какие вопросы задать о тестах перед релизом?

ПРАКТИКА 20 после главы 20

Разберитесь, как устроены тесты вашего продукта.

ЧТО СДЕЛАТЬ

01	Попросите техлида показать число тестов по уровням.	☐
02	Оцените время прогона по формуле (20.1).	☐
03	Выпишите 5 критичных сценариев и проверьте, на каких уровнях они покрыты.	☐
04	Узнайте, когда последний раз проводили нагрузочный тест.	☐
05	Добавьте вопросы о тестах в чек-лист готовности к релизу.	☐

РЕЗУЛЬТАТ Если нагрузку не проверяли больше полугода — запланируйте тест.

21 Beta-тесты и тестирование на реальных пользователях

После неё вы планируете бету и пилот, выбираете участников, задаёте критерии выхода, собираете обратную связь системно и принимаете решение о широком запуске на данных реального использования.

21.1 Виды бета-тестов
21.2 Отбор участников
21.3 Критерии выхода из беты
21.4 Кейс «Чека»: три волны пилота
21.5 Бенчмарки
21.6 Анти-паттерны
21.7 Вопросы для самопроверки

Тестовый стенд никогда полностью не повторяет реальность: другие данные, другое оборудование, другие привычки пользователей. Бета-тест — выпуск функции ограниченному кругу реальных пользователей, чтобы найти то, что не видно в тестах, и проверить ценность до широкого запуска. Для B2B бета часто называется пилотом и проходит с конкретными клиентами.

21.1 Виды бета-тестов

ВИД	УЧАСТНИКИ	ЦЕЛЬ
Внутренний (dogfooding)	Сотрудники компании	Найти очевидные проблемы
Закрытая бета	Приглашённые клиенты	Качество и ценность на реальных данных
Открытая бета	Любой желающий	Масштаб, разнообразие, обратная связь
Пилот B2B	Несколько клиентов с договорённостью	Подтвердить ценность и готовность к продажам
Ранний доступ	Клиенты, которые хотят первыми	Лояльность, отзывы для маркетинга

21.2 Отбор участников

Разнообразие	Разные типы точек, размеры, оборудование, регионы.
Представительность	Похожи на целевой сегмент фичи, а не только на самых лояльных.
Готовность	Согласны сообщать о проблемах и выделить время на обратную связь.
Риск	Не берите в первые волны клиентов, для которых сбой критичен.

21.3 Критерии выхода из беты

ФОРМУЛА 21.1 · АКТИВАЦИЯ В БЕТЕ

$$A = \frac{n_{\text{акт}}}{n_{\text{вкл}}} \cdot 100\% \qquad (21.1)$$

где

$n_{\text{акт}}$ — участников, использующих функцию по целевому сценарию — шт.

$n_{\text{вкл}}$ — участников, у которых функция включена — шт.

A — доля активных — %

```
«Остатки», пилот 100 точек: 62 ведут остатки через 3 недели → A = 62%. Цель выхода — 50%: выполнена. Среди кафе — 48%, среди магазинов — 71%
```

21.4 Кейс «Чека»: три волны пилота

Пилот «Остатков» шёл волнами вместе со срезами разработки. Для каждой волны Ника и Алина заранее записали критерии перехода к следующей.

ВОЛНА	ТОЧЕК	КРИТЕРИЙ ПЕРЕХОДА	РЕЗУЛЬТАТ	ГЛАВНАЯ НАХОДКА
1	10 (дружественные)	0 критичных дефектов, 7 из 10 ведут остатки	8 из 10	Рецептуры для кофеен
2	30	Расхождение остатков ≤ 2%, обращения ≤ 3 на точку	1,4%, 2,1 обращения	Сканер не читает часть штрихкодов
3	100	Активация ≥ 50%, время чека ≤ 2,1 с	62%, 1,9 с	Кафе хуже осваивают без подсказок

Находка третьей волны — кафе ведут остатки реже магазинов — привела к изменению онбординга: для кафе добавили шаблоны рецептур популярных напитков. Это вошло в широкий запуск.

ПРАВИЛО

Записывайте критерии выхода из каждой волны до её начала: метрики качества, активации и обратной связи. Без критериев бета длится бесконечно или заканчивается по ощущению.

КЛАССИКА Microsoft, Пол Марис, 1988

Ешьте свою собачью еду

Руководитель в Microsoft разослал письмо с призывом «есть свою собачью еду» — использовать собственные продукты внутри компании. Выражение закрепилось как название внутреннего бета-тестирования.

Сотрудники — не настоящие пользователи, но они находят очевидные проблемы раньше клиентов и чувствуют боль продукта на себе.

МИРОВОЙ КЕЙС Gmail, 2004–2009

Бета как маркетинг

Google запустил Gmail в 2004 году как бету по приглашениям. Каждому пользователю давали несколько приглашений, и они стали желанными — их даже продавали. Статус «бета» продержался больше пяти лет.

Пример показывает, что бета может решать не только задачи качества, но и формировать спрос. Но для B2B-продукта бесконечная бета подрывает доверие: клиенты хотят знать, когда продукт будет «настоящим».

ПЛОХО

Бета без критериев

«Дадим 100 клиентам, посмотрим».

Через два месяца непонятно, готовы ли к запуску.

ХОРОШО

Волны с критериями

10 → 30 → 100 точек, у каждой волны метрики перехода.

Решение о запуске на данных, находки исправлены до широкого релиза.

21.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Первая волна	5–20 дружественных клиентов
Длительность волны	1–3 недели
Активация для выхода	задаётся под фичу, часто 40–60%
Канал обратной связи	отдельный чат или форма + звонки

21.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Только лояльные участники	Завышенная оценка	Разнообразие и представительность
Нет критериев выхода	Бесконечная бета	Метрики перехода
Обратная связь «как придётся»	Потеря находок	Канал и регулярные звонки
Критичные клиенты в первой волне	Репутационный риск	Дружественные сначала

ПРИМЕР

Saas

Сервис бухгалтерии запускал новый модуль отчётности для 200 клиентов в открытой бете. Без отдельного канала обратной связи жалобы приходили в общую поддержку и терялись. Во втором запуске завели чат беты и еженедельный звонок — число найденных проблем выросло в 3 раза, а широкий запуск прошёл без всплеска обращений.

21.7 Вопросы для самопроверки

- Какие виды бета-тестов вы знаете?
- Как отбирать участников беты?
- Как посчитать активацию в бете?
- Зачем нужны критерии выхода из каждой волны?
- Что такое dogfooding?

ПРАКТИКА 21 после главы 21

Спланируйте бету для ближайшей фичи.

ЧТО СДЕЛАТЬ

01	Выберите вид беты и разбейте её на 2–3 волны.	☐
02	Опишите критерии отбора участников каждой волны.	☐
03	Запишите критерии перехода: качество, активация, обратная связь.	☐
04	Настройте канал обратной связи и расписание звонков.	☐
05	Посчитайте активацию по формуле (21.1) после первой волны.	☐

РЕЗУЛЬТАТ Разделите результаты по сегментам — различия часто важнее среднего.

22 Критерии качества

После неё вы формулируете определение готовности для команды и критерии качества релиза, включаете в них нефункциональные требования и принимаете решение «выпускаем или нет» по заранее согласованным правилам, а не по настроению.

22.1 Три уровня критериев
22.2 Definition of Done «Чека»
22.3 Нефункциональные требования
22.4 Решение о релизе
22.5 Кейс «Чека»: спор за два дня до релиза
22.6 Бенчмарки
22.7 Анти-паттерны
22.8 Вопросы для самопроверки

«Готово» для разработчика, тестировщика и продакта часто значит разное: код написан; тесты прошли; пользователи получили ценность. Без общих критериев споры о готовности возникают в последний день перед релизом. Критерии качества — договорённость команды о том, что значит «сделано», и о минимальном уровне, ниже которого релиз не выходит.

22.1 Три уровня критериев

УРОВЕНЬ	ЧТО ОПРЕДЕЛЯЕТ	КТО ДОГОВАРИВАЕТСЯ
Критерии приёмки	Работает ли конкретная история	PM + команда + QA
Definition of Done	Общие требования ко всем историям	Команда
Критерии релиза	Готов ли релиз к пользователям	PM + техлид + QA

22.2 Definition of Done «Чека»

- Критерии приёмки выполнены и проверены QA.
- Модульные и интеграционные тесты написаны, прогон зелёный.
- Код прошёл ревью двух человек.

- [ ] Работает офлайн и восстанавливается после обрыва связи.
- [ ] События аналитики отправляются и проверены.
- [ ] Функция за флагом, флаг описан.
- [ ] Документация для поддержки обновлена.

22.3 Нефункциональные требования

КАТЕГОРИЯ	ТРЕБОВАНИЕ «ЧЕКА»	КАК ПРОВЕРЯЕМ
Производительность	Чек ≤ 2,1 с на 10 000 позиций	Нагрузочный тест
Надёжность	Нет потери чеков при обрыве связи	Тест с отключением сети
Совместимость	12 моделей планшетов и 6 принтеров	Ферма устройств
Безопасность	Данные кассы зашифрованы, роли кассир/владелец	Ревью, автоматическое сканирование
Соответствие закону	Требования к онлайн-кассам и ОФД	Сертификация
Доступность	Крупный шрифт, контраст на кассе	Чек-лист

22.4 Решение о релизе

ФОРМУЛА 22.1 · ИНДЕКС ГОТОВНОСТИ РЕЛИЗА

$$R = \frac{\Sigma\, w_i \cdot c_i}{\Sigma\, w_i} \qquad (22.1)$$

где

c_i — выполнен ли критерий: 1 или 0 —

w_i — вес: блокирующий критерий — 0 при невыполнении обнуляет индекс; остальные — 1–3 —

R — доля выполненных критериев доля

```
«Остатки v1» за неделю до выставки: все 6 блокирующих выполнены; из 8 остальных (вес 14) не выполнены «фото товара» (1) и «подсказки для кафе» (2) → R = 11 / 14 ≈ 0,79 при пороге 0,75 — выпускаем, два пункта в v1.1
```

22.5 Кейс «Чека»: спор за два дня до релиза

Раньше каждый релиз сопровождался спором: QA находил дефекты, продажи давили сроком, Егор пытался найти компромисс. Ника, Алина и Егор договорились о критериях релиза заранее: какие дефекты блокируют, какие нет.

СЕРЬЁЗНОСТЬ ДЕФЕКТА	ПРИМЕР	БЛОКИРУЕТ РЕЛИЗ
Критичный	Потеря чеков, неверная сумма, сбой кассы	да, всегда
Высокий	Неверный остаток у части товаров	да, если затрагивает больше 1% точек
Средний	Отчёт строится 30 секунд вместо 5	нет, исправление в течение 2 недель
Низкий	Опечатка, выравнивание	нет

За два дня до выставки нашли дефект: при возврате весового товара остаток восстанавливался с ошибкой округления. По таблице он был «высоким», но затрагивал менее 1% точек пилота. Релиз вышел, исправление — через 4 дня. Спор занял 10 минут вместо привычных двух часов.

ПРАВИЛО

Договаривайтесь о критериях релиза до начала работы над фичей, а не в день релиза. Под давлением сроков любые правила кажутся гибкими.

КЛАССИКА Scrum Guide, 2020

Определение готовности

Руководство по Scrum называет Definition of Done обязательством для инкремента: формальное описание состояния, при котором работа соответствует требуемому качеству. Если элемент не соответствует определению, его нельзя выпускать и даже показывать на обзоре.

Команда может ужесточать определение со временем: добавлять тесты, проверки безопасности, аналитику.

МИРОВОЙ КЕЙС Toyota, андон

Остановить конвейер

На заводах Toyota любой рабочий может потянуть шнур андон и остановить конвейер, если видит дефект. Это дорого в моменте, но дефект не уходит дальше и причину устраняют сразу.

Для delivery аналог — блокирующие критерии качества: любой член команды может остановить релиз при их нарушении, и это не обсуждается.

ПЛОХО

Спор в день релиза

QA против продаж, два часа обсуждения, решение по настроению.

Через месяц тот же спор.

ХОРОШО

Критерии заранее

Блокирующие и неблокирующие дефекты определены.

Решение за 10 минут, исправление по плану.

22.6 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Пунктов в Definition of Done	5–10
Блокирующих критериев релиза	3–6
Пересмотр DoD	раз в квартал на ретро
Срок исправления средних дефектов	до 2 недель

22.7 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Нет общего «готово»	Споры, недоделки	Definition of Done
Критерии в день релиза	Давление сроков	Заранее
Без нефункциональных требований	Тормоза и сбои	Скорость, надёжность, совместимость
Все дефекты блокируют	Релизы не выходят	Классы серьёзности

ПРИМЕР

Медицина

Сервис телемедицины включил в критерии релиза «время подключения к видеосвязи ≤ 5 с на 3G». Новая версия видеомодуля проходила все функциональные тесты, но подключалась за 9 секунд на медленной сети. Релиз остановили, оптимизация заняла неделю — а пациенты в регионах с плохой связью продолжили пользоваться сервисом без жалоб.

22.8 Вопросы для самопроверки

- Какие три уровня критериев качества вы знаете?
- Что входит в Definition of Done?
- Какие нефункциональные требования важны для вашего продукта?
- Как посчитать индекс готовности релиза?
- Почему критерии релиза согласуют заранее?

ПРАКТИКА 22 после главы 22

Сформулируйте критерии качества команды.

ЧТО СДЕЛАТЬ

01	Запишите Definition of Done из 5–10 пунктов.	☐
02	Выпишите нефункциональные требования и способы проверки.	☐
03	Определите классы серьёзности дефектов и какие блокируют релиз.	☐
04	Посчитайте индекс готовности по формуле (22.1) для ближайшего релиза.	☐
05	Согласуйте критерии с техлидом и QA.	☐

РЕЗУЛЬТАТ Повесьте критерии на странице команды и пересматривайте раз в квартал.

23 Подготовка к релизу: чек-листы, freeze, релиз-ноты

После неё вы готовите релиз по чек-листу, решаете, нужен ли code freeze, пишете релиз-ноты для разных аудиторий и проводите релиз так, чтобы поддержка, продажи и клиенты знали, что их ждёт.

23.1 Чек-лист готовности
23.2 Code freeze: когда нужен
23.3 Релиз-ноты для разных аудиторий
23.4 Кейс «Чека»: релиз «Остатков»
23.5 Бенчмарки
23.6 Анти-паттерны
23.7 Вопросы для самопроверки

Релиз — не момент нажатия кнопки, а процесс: готовность кода, данных, инфраструктуры, поддержки, документации, коммуникации. Большинство неудачных релизов проваливаются не из-за кода, а из-за забытого: поддержка не знала о новой функции, миграция данных не прогнана на копии прода, клиенты не предупреждены о смене интерфейса.

23.1 Чек-лист готовности

ОБЛАСТЬ	ПРОВЕРКА	ОТВЕТСТВЕННЫЙ
Код и тесты	Критерии релиза выполнены, регресс зелёный	Техлид, QA
Данные	Миграции проверены на копии прода, есть план отката	Техлид
Инфраструктура	Мониторинг и алерты на новые компоненты	Платформа
Флаги	Функция за флагом, план раскатки	Техлид, PM
Поддержка	Обучение, база знаний, шаблоны ответов	PM, руководитель поддержки
Продажи и маркетинг	Материалы, дата анонса, обещания клиентам	PM, маркетинг
Юридическое	Оферта, тексты, соответствие требованиям	PM, юристы
Коммуникация	Релиз-ноты, уведомления в продукте	PM

23.2 Code freeze: когда нужен

Code freeze — период перед релизом, когда в релизную ветку не вносят новых изменений, кроме исправлений. Он нужен, когда релиз большой, проверки ручные и долгие. При непрерывной поставке и автоматических проверках freeze становится не нужен: каждое изменение маленькое и проверяется сразу. Исключение — периоды повышенного риска: для «Чека» это декабрь, пик продаж у розницы.

ФОРМУЛА 23.1 · СТОИМОСТЬ CODE FREEZE

$$C = d \cdot N \cdot u \cdot h \tag{23.1}$$

где

- d — длительность freeze — дн.
- N — инженеров, чья работа задерживается — чел.
- u — доля работы, которая ждёт, а не переключается на полезное — доля
- h — стоимость инженерного дня — ₽

«Чек» раньше: 3 дня freeze раз в 2 недели × 32 инженера × 0,4 × 25 000 ₽ ≈ 960 000 ₽ на релиз ≈ 25 млн ₽ в год. После автоматизации — freeze только в декабре

23.3 Релиз-ноты для разных аудиторий

АУДИТОРИЯ	ЧТО ВАЖНО	ФОРМАТ
Клиенты	Что изменилось для них и как пользоваться	Короткий текст в продукте, письмо, видео
Поддержка	Что изменилось, частые вопросы, известные ограничения	Внутренняя статья, шаблоны ответов
Продажи	Ценность, кому продавать, что обещать нельзя	Одностраничник, демо
Команда и компания	Что вышло, метрики, кого поблагодарить	Пост во внутреннем канале

23.4 Кейс «Чека»: релиз «Остатков»

Для широкого релиза Ника собрала чек-лист из 34 пунктов. Самой трудоёмкой частью

оказалась не техника, а поддержка: 40 сотрудников нужно было научить отвечать на вопросы об остатках, рецептурах и сканерах.

ДЕЙСТВИЕ	СРОК ДО РЕЛИЗА	РЕЗУЛЬТАТ
Обучение поддержки: 2 сессии + запись	-10 дней	40 из 40 прошли тест
База знаний: 18 статей	-7 дней	Статьи открывались 2 300 раз за первый месяц
Демо для продаж	-7 дней	Сценарий выставки
Письмо клиентам о новой функции	день релиза	Открыли 41%
Подсказки в кассе при первом входе	день релиза	Активация пилота повторилась
Дежурство команды в первые 3 дня	день релиза	2 инцидента P3, ни одного P1

ПРАВИЛО

Поддержка узнаёт о релизе раньше клиентов — минимум за неделю, с обучением и базой знаний. Если первый вопрос клиента о новой функции застаёт поддержку врасплох, релиз не готов.

КЛАССИКА Атул Гаванде, «The Checklist Manifesto», 2009

Чек-лист спасает жизни

Хирург Атул Гаванде описал, как простые чек-листы в авиации и хирургии резко снижают число ошибок. Пилотная программа ВОЗ с хирургическим чек-листом из 19 пунктов снизила осложнения и смертность в восьми больницах примерно на треть.

Чек-листы работают не потому, что профессионалы не знают, что делать, а потому, что под давлением и в рутине забывают очевидное.

МИРОВОЙ КЕЙС многочисленные примеры SaaS

Релиз, о котором не предупредили

Обычная история B2B: компания меняет интерфейс, клиенты открывают систему утром и не находят привычных кнопок. Поддержка перегружена, ключевые клиенты пишут руководству, часть требует вернуть старую версию.

Те же изменения с предупреждением за неделю, коротким видео и переключателем «вернуться к старому виду» на месяц проходят почти незаметно.

ПЛОХО

Релиз = выкатка кода

Поддержка узнала из обращений клиентов.

Всплеск обращений, недовольство, продажи без материалов.

ХОРОШО

Релиз = процесс

Чек-лист из 34 пунктов, обучение поддержки, релиз-ноты по аудиториям.

Ни одного P1, базу знаний открыли 2 300 раз.

23.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Обучение поддержки	за 7–10 дней до релиза крупной фичи
Code freeze при непрерывной поставке	только в периоды повышенного риска
Дежурство команды после крупного релиза	2–5 дней
Пунктов в чек-листе крупного релиза	20–40

23.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Нет чек-листа	Забытые шаги	Чек-лист по областям
Freeze каждый релиз	Потери времени	Автоматизация, freeze по риску
Одни релиз-ноты для всех	Никому не подходят	По аудиториям
Поддержка узнаёт последней	Хаос обращений	Обучение заранее

ПРИМЕР

Едтех

Онлайн-школа выпустила новый плеер уроков в день начала учебного года без предупреждения. Обращений в поддержку стало в 6 раз больше. На следующем крупном релизе команда за неделю разослала видео, предупредила кураторов и оставила старый плеер по кнопке на две недели — всплеск обращений составил лишь 30%.

23.7 Вопросы для самопроверки

- Какие области входят в чек-лист готовности релиза?
- Когда нужен code freeze, а когда он лишний?
- Как оценить стоимость code freeze?
- Чем релиз-ноты для клиентов отличаются от нот для поддержки и продаж?
- Почему поддержка должна узнать о релизе первой?

ПРАКТИКА 23 после главы 23

Подготовьте чек-лист ближайшего крупного релиза.

ЧТО СДЕЛАТЬ

01	Выпишите пункты по 8 областям с ответственными.	☐
02	Посчитайте стоимость текущего freeze по формуле (23.1).	☐
03	Подготовьте релиз-ноты для 4 аудиторий.	☐
04	Запланируйте обучение поддержки за неделю.	☐
05	Организуйте дежурство команды на первые дни.	☐

РЕЗУЛЬТАТ После релиза добавьте в чек-лист то, что забыли.

24 Feature toggles и стратегии rollout

После неё вы используете флаги функций, чтобы отделить выкатку кода от включения функции, выбираете стратегию раскатки — канареечную, поэтапную, по сегментам, — и управляете жизненным циклом флагов, не превращая их в технический долг.

24.1 Виды флагов
24.2 Стратегии раскатки
24.3 Радиус поражения
24.4 Кейс «Чека»: раскатка «Остатков»
24.5 Бенчмарки
24.6 Анти-паттерны
24.7 Вопросы для самопроверки

Флаг функции — переключатель в коде, который позволяет включать и выключать функцию без нового релиза. Флаги разделяют два события: деплой (код в проде) и релиз (пользователи видят функцию). Это делает выпуск безопаснее: если что-то пошло не так, функцию выключают за секунды, а не откатывают релиз.

24.1 Виды флагов

ВИД	ЗАЧЕМ	СРОК ЖИЗНИ
Релизный	Спрятать незавершённую функцию, включить по готовности	дни — недели
Экспериментальный	A/B-тест вариантов	недели
Операционный (kill switch)	Быстро отключить тяжёлую функцию при проблемах	долго
Разрешающий	Функция для тарифа, клиента, роли	постоянно

24.2 Стратегии раскатки

Канареечная	Сначала на 1–5% трафика или точек; если метрики в норме — дальше.

Поэтапная	5% → 25% → 50% → 100% с проверкой на каждом шаге.
По сегментам	Сначала один тип клиентов, регион или тариф.
Синий/зелёный	Две версии окружения, переключение трафика целиком.
Тёмный запуск	Код работает в фоне без видимого интерфейса, чтобы проверить нагрузку.

24.3 Радиус поражения

ФОРМУЛА 24.1 · ОЖИДАЕМОЕ ЧИСЛО ЗАТРОНУТЫХ КЛИЕНТОВ

$$E = N \cdot s \cdot p \tag{24.1}$$

где

N — всего клиентов — шт.

s — доля, у которой функция включена — доля

p — вероятность критичной проблемы — доля

E — ожидаемое число затронутых — шт.

«Чек», раньше: релиз сразу на всех, p ≈ 0,18 → E ≈ 18 000 · 1 · 0,18 ≈ 3 240 точек под риском на каждый релиз. Канарейка на 5%: 18 000 · 0,05 · 0,18 ≈ 162, и выключение флагом за минуты

24.4 Кейс «Чека»: раскатка «Остатков»

После пилота Ника и Егор раскатывали «Остатки» по сегментам и долям, с автоматической остановкой, если защитные метрики выходили за пороги. Флаг был разрешающим по тарифу: функция включалась только для точек на тарифе с остатками.

ШАГ	ДОЛЯ	ДЛИТЕЛЬНОСТЬ	ПОРОГ ОСТАНОВКИ	РЕЗУЛЬТАТ
1	Магазины, 5%	3 дня	время чека > 2,1 с или ошибки синхронизации > 0,5%	в норме

ШАГ	ДОЛЯ	ДЛИТЕЛЬНОСТЬ	ПОРОГ ОСТАНОВКИ	РЕЗУЛЬТАТ
2	Магазины, 25% + кафе, 5%	4 дня	то же + обращения > 3 на 100 точек	кафе: 3,4 обращения → пауза на 2 дня, шаблоны рецептур
3	Все, 50%	5 дней	то же	в норме
4	Все, 100%	—	—	через 16 дней после старта

Пауза на втором шаге оказалась главной ценностью стратегии: кафе получили шаблоны рецептур до того, как функция включилась у тысяч точек. Без флагов это был бы хотфикс после массового недовольства.

ПРАВИЛО

У каждого релизного флага — владелец и дата удаления. Флаг, который живёт дольше месяца после полной раскатки, становится техническим долгом: две ветки кода, которые нужно поддерживать и тестировать.

КЛАССИКА Пит Ходжсон, статья на сайте Мартина Фаулера, 2017

Переключатели функций

Ходжсон систематизировал виды флагов по сроку жизни и динамике: релизные, экспериментальные, операционные и разрешающие. Каждый вид требует своего управления: релизные удаляют быстро, разрешающие живут долго и должны быть частью архитектуры.

Он же предупреждал: флаги — мощный инструмент, который быстро превращается в «ад флагов», если не следить за их числом и удалением.

МИРОВОЙ КЕЙС Knight Capital, 2012

Флаг, который не удалили

Торговая компания выкатила новый код на серверы, но на одном из восьми серверов осталась старая версия. Переиспользованный флаг активировал на нём давно неиспользуемый код, который начал совершать тысячи сделок в секунду.

За 45 минут компания потеряла около 440 млн долларов и вскоре была поглощена. Случай стал хрестоматийным примером того, почему флаги нужно удалять, а не переиспользовать.

ПЛОХО

Включить всем

Релиз на 18 000 точек сразу, откат — новый релиз.

Проблема кафе — у тысяч клиентов.

ХОРОШО

Флаг и раскатка

5% → 25% → 50% → 100% с порогами остановки.

Пауза на шаге 2, шаблоны рецептур до массового включения.

24.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Первая доля раскатки	1–5%
Время выключения флагом	секунды — минуты
Срок жизни релизного флага после 100%	до 2–4 недель
Активных релизных флагов на команду	единицы

24.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Релиз на всех сразу	Большой радиус поражения	Канарейка
Флаги без владельца и срока	Ад флагов, долг	Владелец и дата удаления
Переиспользование флагов	Неожиданное поведение	Новый флаг на новую функцию
Ручная остановка раскатки	Медленная реакция	Автоматические пороги

ПРИМЕР

Соцсеть

Социальная сеть выкатывала новый алгоритм ленты на 1% пользователей в одной стране. Через два часа метрика жалоб выросла в 4 раза — автоматическая остановка выключила флаг. Причина — ошибка в фильтре контента для одного языка. На 100% аудитории такая ошибка стала бы новостью в СМИ.

24.7 Вопросы для самопроверки

- Чем деплой отличается от релиза при использовании флагов?
- Какие виды флагов бывают и сколько они живут?
- Какие стратегии раскатки вы знаете?
- Как оценить радиус поражения релиза?
- Почему флаги нужно удалять?

ПРАКТИКА 24 после главы 24

Спланируйте раскатку ближайшей фичи за флагом.

ЧТО СДЕЛАТЬ

01	Определите вид флага, владельца и дату удаления.	☐
02	Выберите стратегию: доли, сегменты, длительность шагов.	☐
03	Задайте пороги автоматической остановки по защитным метрикам.	☐
04	Посчитайте радиус поражения по формуле (24.1) для текущего и нового подхода.	☐
05	Проверьте список старых флагов команды и удалите ненужные.	☐

РЕЗУЛЬТАТ Запишите, сколько флагов удалили, — это метрика гигиены.

25 A/B-тестирование и постепенный rollout

После неё вы различаете раскатку ради безопасности и эксперимент ради измерения эффекта, используете холдаут-группу, чтобы честно оценить вклад фичи в outcome, и проводите A/B-тесты вариантов внутри delivery.

25.1 Раскатка и эксперимент
25.2 Холдаут-группа
25.3 A/B внутри delivery
25.4 Когда эксперимент невозможен
25.5 Бенчмарки
25.6 Анти-паттерны
25.7 Вопросы для самопроверки

Постепенная раскатка из главы 24 защищает от сбоев, но не отвечает на вопрос «сработала ли фича». Если включать функцию по очереди, первые группы отличаются от последних не только фичей, но и временем, сезоном, составом. Чтобы измерить эффект, нужен контролируемый эксперимент: случайная часть пользователей получает функцию, другая — нет, и разница между ними объясняется фичей.

25.1 Раскатка и эксперимент

	ПОСТЕПЕННАЯ РАСКАТКА	A/B-ТЕСТ	ХОЛДАУТ
Цель	Безопасность	Сравнить варианты	Измерить долгий эффект фичи
Группы	Растущая доля	Случайные A и B	Небольшая группа без функции
Длительность	дни	1–4 недели	1–6 месяцев
Вопрос	Не сломали ли?	Какой вариант лучше?	Сколько фича даёт в outcome?

25.2 Холдаут-группа

Холдаут — случайная группа, у которой функция остаётся выключенной долгое время после полной раскатки. Он нужен для эффектов, которые проявляются медленно: удержание, отток, переход на тариф. Размер холдаута — компромисс: достаточно большой для статистики и достаточно маленький, чтобы не лишать многих клиентов ценности.

ФОРМУЛА 25.1 · ЭФФЕКТ ФИЧИ ПО ХОЛДАУТУ

$$\Delta q = q_{\text{хол}} - q_{\text{тест}} \tag{25.1}$$

где

$q_{\text{хол}}$ — месячный отток в холдауте — доля

$q_{\text{тест}}$ — месячный отток у точек с функцией — доля

Δq — снижение оттока благодаря функции — п.п.

«Остатки»: холдаут 10% подходящих точек (1 400) на 3 месяца. Отток: холдаут 2,3%, с функцией 1,9% → Δq = 0,4 п.п. (95% ДИ 0,1…0,7) — совпало с ожиданием из дерева целей

25.3 A/B внутри delivery

Во время раскатки «Остатков» выяснилось, что кафе осваивают функцию хуже магазинов. Команда сделала два варианта онбординга и проверила их A/B-тестом на 2 400 кафе, у которых включались остатки.

МЕТРИКА	A: БЕЗ ШАБЛОНОВ	B: ШАБЛОНЫ РЕЦЕПТУР	РАЗНИЦА
Ведут остатки через 3 недели	48%	61%	+13 п.п. (+9…+17)
Ввели рецептуры	22%	57%	+35 п.п.
Обращения на 100 точек	3,4	1,9	-1,5
Время чека (защитная)	1,9 с	1,9 с	0

ПРАВИЛО

Раскатка отвечает на вопрос «не сломали ли», эксперимент — «сработало ли». Для фичи, ради которой меняют дорожную карту, заложите холдаут заранее — иначе через полгода эффект не отделить от сезона и маркетинга.

25.4 Когда эксперимент невозможен

СИТУАЦИЯ	ЧТО ДЕЛАТЬ
Мало пользователей	Сравнение до/после с контролем по похожему сегменту
Сетевой эффект между группами	Рандомизация по регионам или клиентам целиком

СИТУАЦИЯ	ЧТО ДЕЛАТЬ
Юридические или этические ограничения	Поэтапный запуск по регионам и разность разностей
Функция обещана всем клиентам	Холдаут на время, а не навсегда; честная коммуникация

КЛАССИКА Рон Кохави и др., «Trustworthy Online Controlled Experiments», 2020

Холдауты в продуктовых компаниях

Авторы описывают долгосрочные холдауты как способ измерить накопленный эффект множества изменений и проверить, что краткосрочные выигрыши в тестах не исчезают со временем.

Они предупреждают о стоимости: группа без улучшений получает худший опыт, поэтому холдауты должны быть небольшими и ограниченными по времени.

МИРОВОЙ КЕЙС практика экспериментов в крупных продуктах

Новизна, которая проходит

Команды экспериментов в больших компаниях регулярно наблюдают эффект новизны: новый элемент привлекает внимание в первые дни, метрики растут, а через несколько недель возвращаются к исходным. Обратный эффект — сопротивление изменениям — сначала тянет метрики вниз.

Поэтому для решений о продуктовых изменениях смотрят на эффект после стабилизации, а для удержания — на холдауты в месяцах.

ПЛОХО

«До и после»

Отток весной упал — значит, остатки сработали.

Нельзя отделить сезон и маркетинг.

ХОРОШО

Холдаут

10% подходящих точек без функции 3 месяца.

Δq = 0,4 п.п. с интервалом — эффект доказан.

25.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Размер холдаута	5–10%

ПАРАМЕТР	ОРИЕНТИР
Длительность холдаута для удержания	2–6 месяцев
A/B вариантов онбординга	2–4 недели
Защитные метрики	в каждом эксперименте

25.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Раскатка как эксперимент	Смешение с временем	Случайные группы
Сравнение до/после	Сезон и маркетинг	Холдаут
Холдаут навсегда	Клиенты без ценности	Ограничить по времени
Решение по первым дням	Эффект новизны	Ждать стабилизации

ПРИМЕР

Подписки

Сервис доставки еды раскатил программу лояльности на всех и через квартал увидел рост повторных заказов на 8%. Аналитик заметил, что в тот же квартал конкурент поднял цены. Следующую программу запустили с холдаутом 5%: реальный эффект составил 2,5% — меньше, но доказанный.

25.7 Вопросы для самопроверки

- Чем постепенная раскатка отличается от эксперимента?
- Что такое холдаут и когда он нужен?
- Как оценить эффект фичи по холдауту?
- Что делать, если эксперимент невозможен?
- Что такое эффект новизны?

ПРАКТИКА 25 после главы 25

Заложите измерение эффекта в ближайший релиз.

ЧТО СДЕЛАТЬ

01	Определите, нужен ли холдаут, и выберите размер и срок.	☐
02	Если есть варианты решения — спроектируйте A/B-тест.	☐
03	Задайте защитные метрики.	☐
04	Опишите, как посчитаете эффект по формуле (25.1).	☐
05	Согласуйте холдаут с продажами и поддержкой.	☐

РЕЗУЛЬТАТ Запишите ожидаемый эффект до запуска, чтобы сравнить с фактом.

26 Go-to-market: координация с маркетингом и продажами

После неё вы готовите запуск фичи вместе с маркетингом, продажами и поддержкой, определяете, кому и как её продавать, синхронизируете сроки и обещания и измеряете результат запуска не только в продукте, но и в выручке.

26.1 Элементы GTM · 26.2 Синхронизация сроков · 26.3 Кейс «Чека»: выставка и после · 26.4 Бенчмарки · 26.5 Анти-паттерны · 26.6 Вопросы для самопроверки

Фича, о которой не знают клиенты, не приносит ценности. Фича, которую продают не тем клиентам, приносит разочарование. Go-to-market — план вывода функции на рынок: кому она нужна, как о ней узнают, как её продают, сколько она стоит и как поддержка помогает ею пользоваться. Для B2B-продукта GTM часто важнее самой разработки: без продаж «Склад+» не принесёт денег.

26.1 Элементы GTM

ЭЛЕМЕНТ	ВОПРОС	«ОСТАТКИ» И «СКЛАД+»
Сегмент	Кому продаём в первую очередь?	Малые магазины и кофейни с 100–1 000 позиций
Ценность	Какую проблему решаем в словах клиента?	«Знаете, что заканчивается, до того как спросит покупатель»
Цена и упаковка	Как монетизируем?	Тариф «Склад+» 3 390 ₽ (+900 ₽), 14 дней бесплатно
Каналы	Как узнают?	Выставка, письмо базе, подсказка в кассе, вебинар
Продажи	Кто и как продаёт?	Менеджеры по текущим клиентам + новый скрипт
Поддержка	Как помогаем освоить?	База знаний, шаблоны, онбординг-звонок для «Склад+»

ЭЛЕМЕНТ	ВОПРОС	«ОСТАТКИ» И «СКЛАД+»
Метрики	Как поймём, что запуск удался?	Переходы на «Склад+», активация, выручка

26.2 Синхронизация сроков

Главный риск GTM — расхождение: маркетинг анонсирует дату, которую разработка не гарантирует, продажи обещают функции из второй версии. Решение — единый календарь запуска и правило: внешние даты называет продакт после подтверждения техлидом, а обещать можно только то, что уже прошло бету.

ФОРМУЛА 26.1 · ОЖИДАЕМАЯ ВЫРУЧКА ЗАПУСКА

$$\Delta MRR = N_{\text{ц}} \cdot r \cdot \Delta p \tag{26.1}$$

где

- $N_{\text{ц}}$ — целевых клиентов — шт.
- r — доля, перешедших на новый тариф — доля
- Δp — прирост платежа — ₽/мес

```
«Склад+»: 18 000 · 0,15 · 900 ₽ ≈ 2,43 млн ₽ MRR через 12 месяцев. Факт через 3 месяца: 9% → 1,46 млн ₽ — на траектории
```

26.3 Кейс «Чека»: выставка и после

На выставке 1 марта стенд «Чека» показывал сценарий «принял — продал — увидел остаток — получил уведомление». Ника заранее договорилась с продажами о трёх правилах: показывать только то, что в бете; про импорт и сети говорить «во втором квартале»; каждую заявку отмечать источником.

ЭТАП	ДЕЙСТВИЕ	РЕЗУЛЬТАТ ЗА МЕСЯЦ
Выставка	Демо на стенде, сбор заявок	420 заявок, 96 новых точек
Письмо базе	Сегменты: магазины, кафе, сети	Открыли 41%, перешли на «Склад+» 3%
Подсказка в кассе	После 100-го чека для подходящих точек	Включили пробный период 11%

ЭТАП	ДЕЙСТВИЕ	РЕЗУЛЬТАТ ЗА МЕСЯЦ
Вебинар	«Как навести порядок на складе за вечер»	380 участников, 52 перехода
Звонки менеджеров	Крупные текущие клиенты	34 перехода, 6 запросов на сети

ВАЖНО · ТИПИЧНАЯ ОШИБКА

Отдавать GTM маркетингу «после релиза». К моменту релиза уже должны быть готовы сегменты, материалы, обучение продаж и поддержки, цены и метрики. Начинайте GTM одновременно с разработкой.

КЛАССИКА — Джеффри Мур, «Crossing the Chasm», 1991

Пересечь пропасть

Мур показал, что технологические продукты легко находят первых энтузиастов, но проваливаются при переходе к прагматичному большинству. Причина — разные ожидания: энтузиасты прощают недоделки, прагматики ждут законченного решения для своей задачи.

Его рецепт — начинать с узкого сегмента и дать ему полное решение. Для «Остатков» это малые магазины и кофейни, а не сети.

МИРОВОЙ КЕЙС — Cyberpunk 2077, 2020

Анонс раньше продукта

Игру анонсировали за много лет до выхода, маркетинговая кампания обещала масштабный мир. Дату выхода переносили несколько раз, а на консолях прошлого поколения игра вышла с серьёзными техническими проблемами.

Sony на время убрала игру из своего магазина и предлагала возврат денег. Разрыв между обещаниями маркетинга и готовностью продукта стоил компании репутации и денег.

ПЛОХО

Маркетинг после релиза

Функция вышла, о ней узнали из рассылки через месяц.

Продажи обещают импорт, которого нет.

ХОРОШО

GTM с разработкой

Сегмент, цена, материалы, правила обещаний — до релиза.

96 новых точек после выставки, 9% на «Склад+» за 3 месяца.

26.4 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Старт GTM	вместе с разработкой
Обучение продаж	за 1–2 недели до запуска
Внешние даты	только после подтверждения техлидом
Метрики запуска	продуктовые + выручка + источники

26.5 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
GTM после релиза	Потерянный эффект	С началом разработки
Обещания из будущих версий	Разочарование клиентов	Только то, что прошло бету
Продавать всем	Неудачные внедрения	Целевой сегмент
Нет метки источника	Не понять, что сработало	Источник у каждой заявки

ПРИМЕР

Saas

Сервис онлайн-записи выпустил функцию предоплаты и сначала предложил её всем. Внедрения у крупных клиник шли тяжело, были отказы. Второй запуск — только для салонов красоты с частыми неявками — дал 28% переходов на тариф с предоплатой за два месяца и хорошие отзывы, которые потом помогли продавать клиникам.

26.6 Вопросы для самопроверки

- [ ] Из каких элементов состоит GTM?
- [ ] Почему GTM начинают вместе с разработкой?
- [ ] Как оценить ожидаемую выручку запуска?
- [ ] Какие правила обещаний нужно согласовать с продажами?

- Что такое «пропасть» по Муру?

ПРАКТИКА 26 после главы 26

Подготовьте GTM-план для ближайшей фичи.

ЧТО СДЕЛАТЬ

01	Заполните таблицу из семи элементов.	☐
02	Посчитайте ожидаемую выручку по формуле (26.1).	☐
03	Согласуйте с продажами правила обещаний.	☐
04	Составьте единый календарь запуска.	☐
05	Определите метрики и метки источников.	☐

РЕЗУЛЬТАТ Через месяц после запуска сравните каналы по результату.

27 Настройка аналитики и метрик здоровья продукта

После неё вы закладываете аналитику в определение готовности, составляете план событий для фичи, строите дашборд здоровья продукта и отличаете метрики, которые нужно смотреть каждый день, от тех, что смотрят раз в месяц.

27.1 План событий · 27.2 Метрики здоровья · 27.3 Кейс «Чека»: аналитика в Definition of Done · 27.4 Бенчмарки · 27.5 Анти-паттерны · 27.6 Вопросы для самопроверки

Если аналитику добавляют «после релиза», первые недели — самые важные для понимания освоения — теряются. Если её делают без плана, через месяц выясняется, что события называются по-разному на кассе и в кабинете, а важного действия нет вовсе. Аналитика — часть фичи: она проектируется вместе с требованиями и проверяется QA, как любой другой функционал.

27.1 План событий

СОБЫТИЕ	КОГДА	ПАРАМЕТРЫ	ЗАЧЕМ
stock_enabled	Точка включила остатки	тариф, тип точки, источник	Освоение
item_created	Создан товар	способ: вручную / сканер / шаблон	Как заполняют справочник
receipt_created	Оформлена приёмка	число позиций, сканер	Регулярность
stock_decremented	Списание при продаже	тип товара, рецептура	Работает ли списание
low_stock_alert	Уведомление «заканчивается»	канал	Ценность
stock_mismatch	Расхождение при инвентаризации	размер расхождения	Качество данных

27.2 Метрики здоровья

Метрики здоровья показывают, работает ли продукт для пользователей прямо сейчас. Они отличаются от продуктовых: продуктовые отвечают «растём ли», здоровья — «не сломалось ли». Для кассы здоровье — это скорость и надёжность продаж.

МЕТРИКА	ЦЕЛЬ	ЧАСТОТА	«ЧЕК» ДО → ПОСЛЕ
Время чека, 95-й процентиль	≤ 2,5 с	в реальном времени	3,1 → 2,2 с
Доля успешных синхронизаций	≥ 99,9%	в реальном времени	99,3 → 99,92%
Сессии без падений (crash-free)	≥ 99,8%	ежедневно	99,2 → 99,85%
Доступность облака	≥ 99,9%	ежедневно	99,5 → 99,91%
Активные точки (North Star)	рост	еженедельно	14 200 → 15 900
Обращения на 1 000 точек	≤ 45	еженедельно	42 → 39

ФОРМУЛА 27.1 · ПРОЦЕНТИЛЬ ВМЕСТО СРЕДНЕГО

$$P_{95} = x_{(\lceil 0{,}95 \cdot n \rceil)} \tag{27.1}$$

где

$x_{(k)}$ — k-е значение в упорядоченном ряду — с

n — число наблюдений — шт.

P_{95} — значение, быстрее которого проходят 95% чеков — с

«Чек», крупные точки до оптимизации: среднее время чека 1,6 с выглядело хорошо, а P95 = 3,1 с — каждый двадцатый чек в час пик проходил дольше трёх секунд, и именно на это жаловались кассиры

27.3 Кейс «Чека»: аналитика в Definition of Done

В прошлых фичах события добавляли после релиза, и для двух из них так и не удалось посчитать освоение. Для «Остатков» Ника добавила в DoD пункт «события отправляются и проверены QA», а Ян составил план событий до начала разработки. На второй день пилота уже было видно, что кафе почти не используют сканер, — это стало первым сигналом о проблеме с рецептурами.

ПРАВИЛО

План событий — часть спецификации, а проверка событий — часть Definition of Done. Фича без аналитики не готова, даже если работает.

КЛАССИКА Эрик Рис, «The Lean Startup», 2011

Метрики тщеславия

Рис различает метрики тщеславия — общие числа, которые растут сами собой и приятно выглядят в отчётах (регистрации за всё время, просмотры), — и действенные метрики, связанные с конкретными решениями и когортами.

Дашборд здоровья состоит из действенных метрик: если они ухудшаются, понятно, что делать.

МИРОВОЙ КЕЙС Google, «Site Reliability Engineering», 2016

Четыре золотых сигнала

Инженеры Google описали четыре сигнала, которые нужно отслеживать у любого сервиса: задержка (latency), трафик, ошибки и насыщение ресурсов. Если следить только за ними, большинство проблем видны вовремя.

Для продакта это хороший минимум метрик здоровья: насколько быстро, сколько используют, как часто ломается и близко ли к пределу.

ПЛОХО

Аналитика потом

События добавили через месяц после релиза.

Первый месяц освоения не виден, у двух фич освоение не посчитать.

ХОРОШО

Аналитика в DoD

План событий до разработки, проверка QA.

На второй день пилота видно, что кафе не используют сканер.

27.4 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Событий на фичу	5–10 ключевых
Метрики здоровья	4–8, в реальном времени или ежедневно

ПАРАМЕТР	ОРИЕНТИР
Процентили времени	P50, P95, P99
Crash-free сессий	от 99,5%, хорошо от 99,8%

27.5 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Аналитика после релиза	Потерянные данные	План событий в спецификации
Средние вместо процентилей	Скрытые проблемы	P95 и P99
Метрики тщеславия	Ложное спокойствие	Действенные метрики
Разные названия событий	Нельзя сравнить	Словарь событий

ПРИМЕР

Финтех

Платёжное приложение показывало на дашборде среднее время платежа 1,2 с. Переход на P99 выявил, что 1% платежей проходит дольше 15 секунд — у клиентов одного банка-эмитента. Проблему на стороне интеграции исправили за неделю, а жалобы на «зависшие платежи» сократились на 70%.

27.6 Вопросы для самопроверки

- Почему аналитика — часть фичи, а не дополнение?
- Что входит в план событий?
- Чем метрики здоровья отличаются от продуктовых?
- Почему процентиль лучше среднего для времени?
- Что такое четыре золотых сигнала?

ПРАКТИКА 27 после главы 27

Подготовьте аналитику и метрики здоровья для фичи.

ЧТО СДЕЛАТЬ

01	Составьте план событий с параметрами.	☐
02	Добавьте проверку событий в Definition of Done.	☐
03	Выберите 4–8 метрик здоровья продукта.	☐
04	Замените средние на процентили по формуле (27.1).	☐
05	Определите частоту просмотра каждой метрики.	☐

РЕЗУЛЬТАТ Проверьте события на тестовом стенде до релиза вместе с QA.

28 Мониторинг ошибок и инцидентов: Sentry, Datadog и другие

После неё вы понимаете, как устроены мониторинг ошибок, метрик и логов, участвуете в управлении инцидентами в роли продакта — оценка влияния, коммуникация с клиентами, приоритеты — и проводите разбор после инцидента без поиска виноватых.

28.1 Инструменты мониторинга
28.2 Уровни серьёзности
28.3 Роль продакта в инциденте
28.4 Кейс «Чека»: утро 23-го дня
28.5 Бенчмарки
28.6 Анти-паттерны
28.7 Вопросы для самопроверки

Инцидент — ситуация, когда продукт не работает так, как должен, для заметной части пользователей. Для кассы инцидент — это кафе, которое не может пробить чек в утренний час пик. Продакт не чинит инцидент, но отвечает за то, чтобы его влияние было понятно, клиенты получили честную информацию, а выводы превратились в изменения продукта и процесса.

28.1 Инструменты мониторинга

КЛАСС	ПРИМЕРЫ	ЧТО ПОКАЗЫВАЕТ
Отслеживание ошибок	Sentry, Firebase Crashlytics	Исключения и падения приложений с контекстом и числом затронутых
Метрики и APM	Datadog, Grafana + Prometheus, New Relic	Задержки, ошибки, нагрузка, трассировки запросов
Логи	ELK, Loki, облачные логи	Подробности событий для расследования
Синтетический мониторинг	Проверки сценариев по расписанию	Работает ли ключевой путь прямо сейчас
Дежурства и алерты	PagerDuty, Opsgenie и аналоги	Кто и когда реагирует

Часть зарубежных сервисов в России недоступна или ограничена, поэтому многие команды используют открытые решения на своих серверах: Sentry в собственной

установке, Grafana, Prometheus, ELK. Принципы от этого не меняются.

28.2 Уровни серьёзности

УРОВЕНЬ	«ЧЕК»	РЕАКЦИЯ
P1	Касса не проводит чеки у ≥ 5% точек	Немедленно, круглосуточно, статус клиентам за 15 минут
P2	Сбой функции у части точек: синхронизация, отчёты	В течение часа в рабочее время
P3	Ухудшение без потери функции	В течение дня
P4	Косметика	По плану

28.3 Роль продакта в инциденте

PM ВО ВРЕМЯ ИНЦИДЕНТА

1. **Оценить влияние**
   Сколько клиентов, какие сегменты, что именно не работает.
2. **Коммуникация**
   Статус для поддержки и клиентов: что случилось, что делаем, когда следующее обновление.
3. **Решения о компромиссах**
   Выключить функцию флагом? Откатить релиз? Обойтись без отчётов до утра?
4. **Не мешать**
   Не требовать статус у тех, кто чинит; один канал для обновлений.
5. **После**
   Разбор, выводы в бэклог, компенсации при необходимости.

28.4 Кейс «Чека»: утро 23-го дня

Через 23 дня после полной раскатки «Остатков» в 6:40 утра алерт показал: у 1 200 точек синхронизация остатков отставала на 20 минут, а у части касс замедлилось проведение чеков. Ночная миграция справочника заблокировала таблицу дольше расчётного. Это был

P1: точки открывались.

ВРЕМЯ	ДЕЙСТВИЕ
6:40	Алерт: P95 времени чека 4,8 с, отставание синхронизации
6:48	Дежурный инженер подтвердил P1, открыт канал инцидента
6:55	Ника: оценка — 1 200 точек, в основном кофейни, открытие в 7:00
7:02	Статус клиентам и поддержке: «Касса работает медленнее, чеки проводятся, чиним»
7:05	Решение: выключить операционным флагом списание остатков, чеки ускорились
7:20	Миграция остановлена, таблица разблокирована
7:30	Списание включено, синхронизация догнала очередь
7:40	Статус «решено», MTTR — 60 минут

ФОРМУЛА 28.1 · ВЛИЯНИЕ ИНЦИДЕНТА

$$I = N_{\text{зат}} \cdot t \cdot v \tag{28.1}$$

где

- $N_{\text{зат}}$ — затронутых клиентов — шт.
- t — длительность — ч
- v — стоимость часа проблемы для клиента или компании — ₽/ч
- I — оценка ущерба — ₽

```
1 200 точек × 1 ч × ≈ 150 ₽ (потерянные продажи из-за очередей) ≈ 180 тыс. ₽ у
клиентов. Раньше похожий инцидент длился 4 часа: ≈ 720 тыс. ₽ и десятки уходов
```

ПРАВИЛО

Первое сообщение клиентам — в течение 15 минут, даже если причина неизвестна: что не работает, что делаем, когда следующее обновление. Молчание во время инцидента вредит доверию больше, чем сам сбой.

КЛАССИКА John Allspaw, Etsy, 2012; Google SRE, 2016

Разбор без поиска виноватых

Олспоу описал практику безобвинительных разборов: цель — понять, как система и процесс позволили ошибке случиться, а не найти человека, который «нажал не ту кнопку». Если людей наказывают, они перестают рассказывать правду.

Google SRE закрепила формат: хронология, влияние, корневые причины, что сработало, что нет, действия с владельцами и сроками.

МИРОВОЙ КЕЙС Amazon S3, 28 февраля 2017

Опечатка, остановившая интернет

Инженер при плановом обслуживании ввёл команду с ошибкой и удалил больше серверов, чем планировалось. Хранилище S3 в одном регионе было недоступно около четырёх часов, вместе с ним — множество сайтов и сервисов.

Публичный разбор Amazon не искал виноватого: компания изменила инструмент, чтобы он не позволял удалять мощность слишком быстро, и ускорила перезапуск систем. Это пример исправления процесса, а не человека.

ПЛОХО

Тишина и поиск виноватого

Клиенты узнают о проблеме от кассиров, поддержка не знает, что ответить.

Разбор: «кто запустил миграцию?»

ХОРОШО

Статус и разбор процесса

Сообщение за 15 минут, флаг выключен за 25, решено за 60.

Разбор: миграции — только с проверкой времени блокировки на копии.

28.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Первое сообщение клиентам о P1	до 15 минут
Обновления статуса	каждые 30–60 минут
Разбор после P1–P2	в течение 5 рабочих дней
MTTR для P1	до часа у зрелых команд

28.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Узнавать от клиентов	Долгая реакция	Алерты на метрики здоровья
Молчание во время инцидента	Потеря доверия	Статус за 15 минут
Поиск виноватого	Скрытие ошибок	Безобвинительный разбор
Разбор без действий	Повторение	Действия с владельцами

ПРИМЕР

Доставка

Сервис доставки за год сократил MTTR по P1 с 2,5 часа до 35 минут, введя три вещи: операционные флаги для тяжёлых функций, ротацию дежурных с понятными инструкциями и шаблон статуса для клиентов. Число инцидентов почти не изменилось, но жалобы на них — вдвое.

28.7 Вопросы для самопроверки

- Какие классы инструментов мониторинга вы знаете?
- Как определить уровни серьёзности инцидентов?
- Что делает продакт во время инцидента?
- Как оценить влияние инцидента?
- Почему разбор должен быть безобвинительным?

ПРАКТИКА 28 после главы 28

Подготовьте продуктовую часть управления инцидентами.

ЧТО СДЕЛАТЬ

01	Определите уровни серьёзности на языке влияния на клиентов.	☐
02	Подготовьте шаблон статуса для клиентов и поддержки.	☐
03	Выясните, какие операционные флаги есть для тяжёлых функций.	☐
04	Оцените влияние последнего инцидента по формуле (28.1).	☐
05	Проведите или прочитайте разбор последнего P1 и проверьте действия.	☐

РЕЗУЛЬТАТ Проверьте, знает ли поддержка, где смотреть статус инцидента.

29 Поддержка пользователей и работа с обратной связью

После неё вы превращаете обращения в поддержку в данные для продукта, строите канал обратной связи от клиентов к команде, отвечаете тем, кто сообщил о проблеме, и снижаете число обращений через изменения продукта, а не скрипты.

29.1 Обращения как данные
29.2 Цикл обратной связи
29.3 Кейс «Чека»: первый месяц «Остатков»
29.4 Бенчмарки
29.5 Анти-паттерны
29.6 Вопросы для самопроверки

Поддержка — самый честный источник информации о продукте: люди пишут туда, когда что-то не работает или непонятно. Но в большинстве компаний эта информация теряется: обращения закрываются, а повторяющиеся проблемы остаются в продукте. Задача продакта — построить мост между поддержкой и командой, чтобы каждая частая проблема становилась задачей, а клиенты видели, что их услышали.

29.1 Обращения как данные

ПОЛЕ	ЗАЧЕМ
Категория (функция, сценарий)	Где проблема
Тип: ошибка, вопрос, просьба, жалоба	Что делать
Сегмент клиента	Кого затрагивает
Ссылка на задачу в трекере	Связь с работой команды
Время решения	Нагрузка на поддержку

ФОРМУЛА 29.1 · НАГРУЗКА ОБРАЩЕНИЙ НА ФУНКЦИЮ

$$T = \frac{n_{\text{обр}}}{N_{\text{польз}}} \cdot 1\,000 \qquad (29.1)$$

где

$n_{\text{обр}}$ — обращений по функции за месяц — шт.

$N_{\text{польз}}$ — клиентов, использующих функцию — шт.

T — обращений на 1 000 клиентов — шт.

«Остатки», первый месяц: 520 обращений на 7 900 точек → T ≈ 66. Через два месяца после изменений: 230 на 9 300 → ≈ 25. По кассе в целом — 39

29.2 Цикл обратной связи

ОТ ОБРАЩЕНИЯ К ИЗМЕНЕНИЮ

1. **Категоризация**
   Каждое обращение размечается по функции и типу.
2. **Еженедельный обзор**
   PM и руководитель поддержки смотрят топ-10 категорий, 30 минут.
3. **Задачи**
   Частые проблемы становятся задачами с приоритетом и ссылкой на обращения.
4. **Исправление**
   Изменение продукта, а не новый скрипт поддержки.
5. **Замыкание**
   Клиентам, сообщившим о проблеме, — сообщение «исправили».

29.3 Кейс «Чека»: первый месяц «Остатков»

В первый месяц после полной раскатки обращения по остаткам составили 66 на 1 000 точек — больше, чем по кассе в целом. Ника и руководитель поддержки разобрали 520 обращений и нашли, что пять категорий дают 78% нагрузки.

КАТЕГОРИЯ	ДОЛЯ	ИЗМЕНЕНИЕ ПРОДУКТА	ЭФФЕКТ
«Остаток ушёл в минус»	24%	Подсказка при первом минусе и настройка запрета продажи	-80% обращений категории
Сканер не читает штрихкод	19%	Ручной ввод кода из того же окна, список поддерживаемых сканеров	-65%
Как ввести рецептуру	16%	Шаблоны рецептур (из A/B) для всех кафе	-75%
Расхождение после инвентаризации	11%	Отчёт «откуда расхождение» по операциям	-50%
Где отчёт по остаткам	8%	Отчёт на главном экране кабинета	-90%

Каждому из 380 клиентов, писавших по этим темам, поддержка отправила сообщение «мы исправили то, о чём вы писали». Ответили благодарностью 64 клиента, а NPS среди получивших сообщение был на 11 пунктов выше, чем у остальных писавших в поддержку.

ПРАВИЛО

Если на один вопрос поддержка отвечает больше 20 раз в месяц, это задача для продукта, а не для базы знаний. Лучший ответ поддержки — тот, который больше не понадобится.

КЛАССИКА Билл Прайс и Дэвид Джаффе, «The Best Service is No Service», 2008

Лучший сервис — отсутствие сервиса

Бывший руководитель службы клиентов Amazon Билл Прайс показал, что большинство обращений вызвано дефектами продукта и процессов. Компании, которые устраняют причины обращений, а не ускоряют ответы, снижают и затраты, и недовольство клиентов.

Он предложил считать каждое обращение «дефектом» и вести учёт его причин, как на производстве.

МИРОВОЙ КЕЙС Basecamp, Stripe и другие

Инженеры на линии поддержки

Ряд компаний регулярно ставит разработчиков и продактов отвечать клиентам в поддержке — по несколько часов в месяц. Инженер, который сам ответил на двадцатый одинаковый вопрос, чинит причину быстрее, чем после отчёта.

Практика создаёт эмпатию и сокращает путь от проблемы клиента до изменения кода.

ПЛОХО

Больше скриптов

Поддержка получает новую статью на каждый частый вопрос.

Обращения не снижаются, клиенты раздражены.

ХОРОШО

Изменения продукта

Топ-5 причин превращены в задачи, клиентам сообщили об исправлении.

Обращения по остаткам 66 → 25 на 1 000 точек.

29.4 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Доля размеченных обращений	от 90%
Обзор с поддержкой	еженедельно, 30 минут
Порог «задача для продукта»	от 20 одинаковых обращений в месяц
Замыкание с клиентом	всем, кто сообщал

29.5 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Обращения без разметки	Нет данных	Категории и тип
Скрипты вместо исправлений	Нагрузка растёт	Задачи для продукта
Нет обратной связи клиентам	Ощущение, что не слышат	Сообщение «исправили»
Поддержка отдельно от команды	Длинный путь проблемы	Еженедельный обзор и смены инженеров

ПРИМЕР

Телеком

Оператор связи разметил обращения и обнаружил, что 18% связаны с непонятной детализацией счёта. Редизайн одного экрана сократил эти обращения на 60%, что при стоимости обращения 120 ₽ дало экономию около 40 млн ₽ в год.

29.6 Вопросы для самопроверки

- Какие поля нужны обращению, чтобы стать данными?
- Как посчитать нагрузку обращений на функцию?
- Из каких шагов состоит цикл обратной связи?
- Когда вопрос поддержки становится задачей для продукта?
- Почему важно сообщать клиентам об исправлениях?

ПРАКТИКА 29 после главы 29

Постройте цикл обратной связи с поддержкой.

ЧТО СДЕЛАТЬ

01	Договоритесь о разметке обращений по функциям и типам.	☐
02	Посчитайте нагрузку по формуле (29.1) для ключевых функций.	☐
03	Проведите обзор топ-10 категорий с поддержкой.	☐
04	Превратите 3 частые проблемы в задачи с приоритетом.	☐
05	После исправления сообщите клиентам.	☐

РЕЗУЛЬТАТ Запишитесь на смену в поддержке на полдня.

30 SLA, SLO и работа с техническим долгом

После неё вы различаете SLI, SLO и SLA, договариваетесь о целевом уровне надёжности, используете бюджет ошибок для решений «фичи или стабильность» и управляете техническим долгом как частью продуктовой работы.

30.1 SLI, SLO, SLA · 30.2 Бюджет ошибок · 30.3 Технический долг · 30.4 Кейс «Чека»: SLO и долг · 30.5 Бенчмарки · 30.6 Анти-паттерны · 30.7 Вопросы для самопроверки

Абсолютная надёжность недостижима и не нужна: каждая следующая «девятка» доступности стоит в разы дороже предыдущей, а пользователь не заметит разницы между 99,99% и 99,999%. Вопрос в том, какой уровень нужен клиентам, сколько он стоит и как балансировать надёжность со скоростью выпуска фич. Для этого есть SLO и бюджет ошибок.

30.1 SLI, SLO, SLA

ТЕРМИН	ЧТО ЭТО	«ЧЕК»
SLI	Индикатор уровня сервиса: что измеряем	Доля чеков, проведённых за ≤ 2,5 с
SLO	Целевое значение индикатора за период	99,9% чеков за 30 дней
SLA	Договорное обязательство с последствиями	«Склад+»: доступность 99,5%, иначе скидка 10% от месяца

SLA всегда слабее SLO: внутренняя цель должна быть строже договорной, чтобы был запас. Если SLO и SLA совпадают, каждое нарушение цели — нарушение договора и компенсация.

30.2 Бюджет ошибок

ФОРМУЛА 30.1 · БЮДЖЕТ ОШИБОК

$$B = (1 - SLO) \cdot T \tag{30.1}$$

где

SLO — целевая доступность — доля

T — период — мин

B — допустимое время недоступности за период — мин

```
SLO 99,9% за 30 дней: B = 0,001 · 43 200 ≈ 43 минуты в месяц. При прежних 99,5% — 216 минут. Инцидент 23-го дня (60 минут деградации) один исчерпал бюджет месяца
```

Бюджет ошибок превращает спор «фичи или стабильность» в правило. Пока бюджет есть, команда выпускает фичи с обычной скоростью. Если бюджет исчерпан, новые фичи замораживаются до восстановления, и команда занимается надёжностью. Решение не зависит от того, кто громче в споре.

30.3 Технический долг

Осознанный	Сознательное упрощение ради срока с планом вернуться: «импорт v2 переделаем».
Неосознанный	Решения, которые оказались неудачными со временем.
Устаревание	Библиотеки, версии ОС, неподдерживаемые компоненты.
Цена долга	Замедление разработки, инциденты, сложность онбординга.

30.4 Кейс «Чека»: SLO и долг

После инцидента 23-го дня Егор предложил ввести SLO и правило бюджета ошибок. Ника согласилась при условии, что техдолг будет виден в бэклоге с оценкой влияния, как любая фича. Команды завели реестр долга и выделили 20% ёмкости.

ЭЛЕМЕНТ ДОЛГА	ВЛИЯНИЕ	ОЦЕНКА	ПРИОРИТЕТ
Миграции без проверки блокировок	Инциденты P1	1 неделя	1
Синхронизация одним потоком	Задержки в пике у крупных точек	2 недели	2
Старая библиотека печати	Падения на 2 моделях принтеров, 0,3% сессий	1 неделя	3
Нет контрактных тестов с платежами	Риск при изменениях API	1 неделя	4
Ручная сборка кабинета	Медленные релизы	3 дня	5

За квартал закрыли первые четыре пункта. Доля сессий без падений выросла с 99,2 до 99,85%, а бюджет ошибок в следующие два месяца ни разу не был исчерпан — фичи выпускались без заморозок.

ПРАВИЛО

Технический долг приоритизируют по влиянию на пользователей и скорость команды, а не по «красоте кода». Долг без оценки влияния не попадает в план — так же, как фича без ценности.

КЛАССИКА Бетси Бейер и др., «Site Reliability Engineering», Google, 2016

Бюджет ошибок

Инженеры Google описали бюджет ошибок как способ согласовать интересы разработки (выпускать быстрее) и эксплуатации (не ломать). Обе стороны договариваются о SLO, а дальше решения принимаются по правилу, а не по спору.

Важная мысль книги: 100% — неверная цель почти для всего. Стремление к ней останавливает развитие продукта.

МИРОВОЙ КЕЙС Уорд Каннингем, 1992

Метафора долга

Автор вики придумал метафору технического долга, объясняя финансистам, зачем переписывать работающий код. Быстрое решение — как кредит: позволяет выпустить раньше, но по нему нужно платить проценты — замедлением будущей работы — и в какой-то момент вернуть основной долг.

Каннингем подчёркивал, что долг не всегда плох: кредит ради раннего выхода на рынок разумен, если его возвращают.

ПЛОХО

Спор «фичи или стабильность»

Каждый месяц техлид и продакт спорят, кто громче.

Долг копится, инциденты повторяются.

ХОРОШО

SLO и реестр долга

Бюджет 43 минуты, правило заморозки, 20% ёмкости на долг по влиянию.

Crash-free 99,85%, заморозок не потребовалось.

30.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
SLO доступности для B2B SaaS	99,9% и выше
SLA	мягче SLO на 0,2–0,5 п.п.
Ёмкость на техдолг	15–25%
Пересмотр SLO	раз в квартал

30.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Цель 100%	Остановка развития	Разумный SLO
SLA = SLO	Компенсации за каждое отклонение	Запас
Долг «когда будет время»	Время не появляется	Фиксированная доля ёмкости
Долг без влияния	Чинят красивое, а не важное	Оценка влияния

ПРИМЕР

Игры

Онлайн-игра обещала в SLA 99,95% доступности и платила компенсации внутриигровой валютой за каждое падение. Внутренний SLO был тем же. После введения SLO 99,97% и бюджета ошибок команда стала замечать проблемы до нарушения SLA, и компенсации за год сократились в 4 раза.

30.7 Вопросы для самопроверки

- Чем SLI отличается от SLO и SLA?
- Почему SLA должен быть мягче SLO?
- Как посчитать бюджет ошибок?
- Как бюджет ошибок помогает решать «фичи или стабильность»?
- Как приоритизировать технический долг?

ПРАКТИКА 30 после главы 30

Введите SLO и реестр долга для своего продукта.

ЧТО СДЕЛАТЬ

01	Выберите 1–2 SLI для ключевого сценария.	☐
02	Договоритесь о SLO и посчитайте бюджет ошибок по формуле (30.1).	☐
03	Сформулируйте правило при исчерпании бюджета.	☐
04	Заведите реестр долга с оценкой влияния.	☐
05	Выделите долю ёмкости на долг.	☐

РЕЗУЛЬТАТ Проверьте, как SLA в договорах соотносится с SLO.

31 Оценка достижения целей релиза

После неё вы проверяете, достиг ли релиз заявленных outcome-метрик, отделяете эффект фичи от внешних факторов, честно признаёте частичный успех и решаете, что делать дальше: развивать, исправлять или сворачивать.

31.1 Когда оценивать
31.2 Отделить эффект от шума
31.3 Кейс «Чека»: оценка через три месяца
31.4 Три решения после оценки
31.5 Бенчмарки
31.6 Анти-паттерны
31.7 Вопросы для самопроверки

Релиз — не финиш. Финиш — изменение поведения пользователей и результата бизнеса, ради которого фичу делали. Оценка целей релиза отвечает на три вопроса: изменились ли метрики, благодаря ли фиче, и что из этого следует для следующей работы. Без этой оценки команда не учится, а дорожная карта строится на предположениях, которые никто не проверял.

31.1 Когда оценивать

СРОК	ЧТО СМОТРИМ	«ОСТАТКИ»
1–2 недели	Качество и здоровье: ошибки, скорость, обращения	Инцидент 23-го дня, обращения 66 на 1 000
1 месяц	Освоение: включили, используют	34% подходящих точек ведут остатки
3 месяца	Outcome: целевое поведение и удержание	44% ведут остатки, отток у них 1,6%
6–12 месяцев	Impact: выручка, тариф	«Склад+» 9% за 3 месяца, прогноз 15% к году

31.2 Отделить эффект от шума

Метрика может вырасти без фичи — из-за сезона, рекламы, ухода конкурента — и упасть вопреки ей. Поэтому оценка опирается на контрольную группу: холдаут, A/B-тест или, если их нет, сравнение с похожим сегментом, который фичу не получил.

ФОРМУЛА 31.1 · РАЗНОСТЬ РАЗНОСТЕЙ

$$\Delta = (M_{\text{т,после}} - M_{\text{т,до}}) - (M_{\text{к,после}} - M_{\text{к,до}}) \tag{31.1}$$

где

$M_{\text{т}}$ — метрика в группе с фичей до и после —

$M_{\text{к}}$ — метрика в контрольной группе до и после —

Δ — эффект фичи за вычетом общего тренда —

```
Отток: с фичей 2,4 → 1,9%, холдаут 2,4 → 2,3% → Δ = -0,5 - (-0,1) = -0,4 п.п. Без контроля эффект завысили бы на 25%
```

31.3 Кейс «Чека»: оценка через три месяца

Через три месяца после полной раскатки Ника собрала команду, Яна и руководителя продаж и прошла по таблице целей из PRD. Часть целей выполнили, часть — нет, и это обсуждали так же спокойно, как успехи.

МЕТРИКА	ЦЕЛЬ	ФАКТ	СТАТУС	ВЫВОД
Доля точек, ведущих остатки	40%	44%	выполнено	Магазины 52%, кафе 36% после шаблонов
Отток точек с остатками	≤ 1,6%	1,6%	выполнено	Эффект -0,4 п.п. по холдауту
Переход на «Склад+»	15% к 12 мес.	9% к 3 мес.	на траектории	Сильнее всего — после вебинаров
Время чека, защитная	≤ 2,1 с	1,9 с	выполнено	—
Обращения на 1 000 точек	≤ 48	39 (по кассе)	выполнено	По остаткам 66 → 25
Сетевые клиенты	не цель v1	6 запросов, 2 ушли	сигнал	Сети — в план следующего квартала

ПРАВИЛО

Оценивайте релиз по метрикам, записанным до разработки, и не меняйте цели задним числом. Частичный успех — нормальный результат; важно, какие решения из него следуют.

31.4 Три решения после оценки

Развивать	Цели выполнены, есть сигнал роста: следующая версия, новые сегменты.
Исправлять	Освоение ниже цели, но причина понятна: онбординг, заметность, качество.
Сворачивать	Эффекта нет, а поддержка стоит денег: удалить или упростить.

КЛАССИКА — Эрик Рис, «The Lean Startup», 2011

Учиться на результатах

Цикл «создать — измерить — научиться» требует, чтобы измерение было связано с исходной гипотезой. Рис предупреждает о «театре успеха»: команды показывают рост общих чисел и не проверяют, подтвердилась ли гипотеза.

После каждой итерации нужно решить: продолжать, менять курс или остановиться.

МИРОВОЙ КЕЙС — Google, «весенние чистки» продуктов

Убрать то, что не работает

Google регулярно закрывает продукты и функции, которые не достигли целей: Google+, Reader, Inbox и многие другие. Решения часто критикуют пользователи, но компания объясняет их фокусом ресурсов.

Для продуктовой команды урок мельче, но тот же: функция без эффекта — не нейтральна, она стоит денег на поддержку и усложняет продукт.

ПЛОХО

Отчёт об успехе

«Фича выпущена, отток снизился» — без контроля и без провалов.

Следующий план строится на завышенном эффекте.

ХОРОШО

Честная оценка

Цели из PRD, эффект по холдауту, частичные результаты и сигналы.

Решения: развивать сети, исправлять онбординг кафе.

31.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Первая оценка освоения	через 1 месяц
Оценка outcome	через 3 месяца
Доля фич, полностью достигающих целей	часто меньше половины
Встреча по оценке	60 минут, команда + аналитик + бизнес

31.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Нет оценки после релиза	Команда не учится	Встреча через 1 и 3 месяца
Цели меняют задним числом	Самообман	Цели из PRD
Без контрольной группы	Завышенный эффект	Холдаут или разность разностей
Фичи не сворачивают	Растёт сложность	Решение «сворачивать»

ПРИМЕР

Едтех

Онлайн-школа ввела правило: через три месяца после релиза каждая фича получает одно из трёх решений. За год 4 из 15 функций свернули, ещё 5 переделали. Время разработчиков на поддержку старых функций снизилось на 20%.

31.7 Вопросы для самопроверки

- Какие сроки оценки релиза и что смотреть на каждом?
- Как отделить эффект фичи от общего тренда?
- Как посчитать разность разностей?
- Какие три решения можно принять после оценки?
- Почему цели нельзя менять задним числом?

ПРАКТИКА 31 после главы 31

Оцените последний крупный релиз.

ЧТО СДЕЛАТЬ

01	Найдите цели и метрики, записанные до разработки.	☐
02	Соберите факт на 1 и 3 месяца.	☐
03	Посчитайте эффект по формуле (31.1) или по холдауту.	☐
04	Отметьте выполненные, частичные и невыполненные цели.	☐
05	Примите одно из трёх решений и запишите его.	☐

РЕЗУЛЬТАТ Добавьте выводы в журнал решений PRD.

32 Сравнение план/факт по метрикам и срокам

После неё вы сравниваете запланированные и фактические сроки, объём и затраты, находите систематические причины отклонений и улучшаете планирование, а не ищете виноватых за срыв.

32.1 Что сравнивать · 32.2 Точность оценок · 32.3 Причины отклонений · 32.4 Кейс «Чека»: год план/факт · 32.5 Бенчмарки · 32.6 Анти-паттерны · 32.7 Вопросы для самопроверки

План всегда расходится с фактом. Вопрос не в том, было ли расхождение, а в том, повторяется ли оно и почему. Если каждая фича опаздывает на 30%, это не невезение, а систематическая ошибка оценки, которую можно исправить. Сравнение план/факт превращает опыт прошлых релизов в точность будущих.

32.1 Что сравнивать

ИЗМЕРЕНИЕ	ПЛАН «ОСТАТКОВ»	ФАКТ	ОТКЛОНЕНИЕ
Срок v1	9 недель	9 недель (Must)	0
Объём v1	150 SP	158 SP (рецептуры добавили)	+5%
Should в v1	24 SP	12 SP	-50%
Срок до 100% раскатки	2 недели	16 дней	+14%
Затраты команды	9 недель × 8 чел.	9,5 недели × 8 чел.	+6%
Инциденты P1 в первый месяц	0	1	+1

32.2 Точность оценок

ФОРМУЛА 32.1 · КОЭФФИЦИЕНТ ТОЧНОСТИ ОЦЕНОК

$$k = \frac{\Sigma F_i}{\Sigma P_i} \tag{32.1}$$

где

F_i — фактическая длительность задачи или фичи — дн.

P_i — плановая длительность — дн.

k — во сколько раз факт в среднем больше плана — –

```
«Чек», 12 фич прошлого года: план 410 дней, факт 590 → k ≈ 1,44. После перехода на
прогноз по истории и вертикальные срезы, 8 фич: план 230, факт 255 → k ≈ 1,11
```

Коэффициент выше 1,3 говорит о систематической недооценке. Лечение — не «оценивайте честнее», а изменения процесса: учитывать ожидания и зависимости, использовать историю скорости, резать фичи на маленькие срезы, оставлять буфер.

32.3 Причины отклонений

ПРИЧИНА	ДОЛЯ ОТКЛОНЕНИЙ «ЧЕКА» ЗА ГОД	ЧТО ИЗМЕНИЛИ
Зависимости от других команд	34%	Доска зависимостей, внутренний опенсорс
Неясные требования	22%	Три амиго, сценарии крайних случаев
Вбросы посреди спринта	18%	Фильтр через PM
Технические сюрпризы	16%	Техуточнение, спайки
Расширение объёма	10%	Журнал решений, MoSCoW

32.4 Кейс «Чека»: год план/факт

В конце года Ника и Егор свели план/факт по всем фичам четырёх команд. Главный результат был не в цифрах «Остатков», а в тренде: точность оценок выросла, а время от

идеи до прода снизилось с 74 до 34 дней — почти до цели года.

ПОКАЗАТЕЛЬ	НАЧАЛО ГОДА	КОНЕЦ ГОДА	ЦЕЛЬ
Время от идеи до прода	74 дня	34 дня	30 дней
Коэффициент точности оценок	1,44	1,11	≤ 1,2
Доля неудачных релизов	18%	6%	≤ 10%
Отток точек	2,4%	1,9%	1,8%
Фич, выпущенных в обещанный срок	42%	81%	—

ПРАВИЛО

Разбирайте отклонения по причинам, а не по людям. Если одна причина даёт больше четверти отклонений, это задача на улучшение процесса в следующем квартале.

КЛАССИКА Бент Фливбьерг, «How Big Things Get Done», 2023

Прогнозирование по эталонному классу

Исследователь крупных проектов собрал данные тысяч проектов — от мостов до IT-систем — и показал, что абсолютное большинство выходят за бюджет и сроки. Лучший способ прогноза — «эталонный класс»: взять распределение отклонений похожих проектов и скорректировать свою оценку.

Для продуктовой команды эталонный класс — собственная история фич.

МИРОВОЙ КЕЙС исследования IT-проектов

Систематическое опоздание

Многолетние отраслевые обзоры IT-проектов регулярно показывают, что значительная часть проектов выходит за сроки или бюджет, а крупные проекты делают это чаще мелких.

Устойчивость результатов говорит о системной причине — оптимизме планирования и больших партиях работы, — а не о некомпетентности отдельных команд.

ПЛОХО

«Кто сорвал срок?»

Разбор каждого опоздания как личной неудачи.

Оценки становятся «с запасом», а точность не растёт.

ХОРОШО

Причины и тренд

Отклонения по причинам, коэффициент точности по году.

k 1,44 → 1,11, в срок 42 → 81% фич.

32.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Коэффициент точности	1,0–1,2 — хороший уровень
Разбор план/факт	после каждой крупной фичи + годовой
Фичи в обещанный срок	от 80%
Причин в разборе	3–5 с долями

32.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Поиск виноватых	Скрытые оценки	Причины процесса
Только сроки	Не видно объёма и эффекта	Сроки, объём, затраты, метрики
Нет истории	Те же ошибки	Эталонный класс
«Оценивайте точнее»	Ничего не меняется	Изменения процесса

ПРИМЕР

Госсектор

Команда цифрового госсервиса год записывала план и факт по каждой задаче. Оказалось, что задачи с внешними согласованиями опаздывают в среднем в 2,3 раза, а без них — в 1,1. Внешние согласования стали запускать в первый день и закладывать отдельный буфер — доля сорванных сроков снизилась вдвое.

32.7 Вопросы для самопроверки

- Какие измерения сравнивают в план/факт?
- Как посчитать коэффициент точности оценок?
- Какие причины отклонений встречаются чаще всего?
- Что такое прогноз по эталонному классу?
- Почему отклонения разбирают по причинам, а не по людям?

ПРАКТИКА 32 после главы 32

Сведите план/факт по последним фичам.

ЧТО СДЕЛАТЬ

01	Соберите план и факт по 5–10 фичам: сроки, объём, затраты.	☐
02	Посчитайте коэффициент точности по формуле (32.1).	☐
03	Разметьте причины отклонений и посчитайте доли.	☐
04	Выберите одну причину для улучшения в следующем квартале.	☐
05	Скорректируйте прогноз ближайшей фичи коэффициентом.	☐

РЕЗУЛЬТАТ Повторите сведение через квартал и сравните коэффициент.

33 Retrospective команды

После неё вы проводите ретроспективы, после которых меняется работа команды, выбираете формат под ситуацию, превращаете выводы в конкретные действия с владельцами и проверяете, что действия выполнены.

33.1 Структура ретроспективы · 33.2 Форматы · 33.3 Выполнение действий · 33.4 Кейс «Чека»: ретро после релиза «Остатков» · 33.5 Бенчмарки · 33.6 Анти-паттерны · 33.7 Вопросы для самопроверки

Ретроспектива — регулярная встреча, на которой команда смотрит на свой процесс и договаривается, что улучшить. Её ценность — не в обсуждении, а в изменениях. Если на каждой ретро звучат одни и те же проблемы, а действия не выполняются, команда перестаёт верить в формат, и встреча превращается в ритуал жалоб.

33.1 Структура ретроспективы

РЕТРО ЗА 60 МИНУТ

1. **Настрой, 5 мин**
   Цель встречи, правило: обсуждаем процесс, а не людей.
2. **Проверка действий, 5 мин**
   Что из прошлых договорённостей выполнено и помогло ли.
3. **Сбор данных, 15 мин**
   Факты и ощущения: что помогало, что мешало. Метрики спринта.
4. **Поиск причин, 15 мин**
   Группировка, голосование, «пять почему» для главной темы.
5. **Действия, 15 мин**
   1–3 конкретных действия с владельцем и сроком.

РЕТРО ЗА 60 МИНУТ

6 **Закрытие, 5 мин**
Короткая оценка встречи.

33.2 Форматы

ФОРМАТ	СУТЬ	КОГДА
Хорошо / плохо / изменить	Три колонки стикеров	Обычный спринт
Начать / прекратить / продолжить	Действия, а не наблюдения	Когда нужны решения
Хронология	Восстановить события по датам	После релиза или инцидента
Парусник	Ветер, якоря, рифы, остров	Свежий взгляд, длинный период
Данные спринта	Метрики цикла, вбросы, возвраты	Когда спорят без фактов

33.3 Выполнение действий

ФОРМУЛА 33.1 · ДОЛЯ ВЫПОЛНЕННЫХ ДЕЙСТВИЙ РЕТРО

$$A = \frac{n_{\text{вып}}}{n_{\text{дог}}} \cdot 100\% \qquad (33.1)$$

где

$n_{\text{вып}}$ — выполненных действий к следующей ретро — шт.

$n_{\text{дог}}$ — договорённых действий — шт.

Команда «Остатков» в начале года: 7 действий за ретро, выполнено 2 → 29%. После правила «не больше трёх действий, у каждого владелец»: 3 из 3 в среднем 2,6 → 87%

33.4 Кейс «Чека»: ретро после релиза «Остатков»

После полной раскатки команда провела расширенную ретроспективу в формате хронологии: восстановили 16 недель проекта по датам и отметили на линии моменты, когда «стало легче» и «стало тяжело».

МОМЕНТ	ЧТО ПОМОГЛО ИЛИ ПОМЕШАЛО	ДЕЙСТВИЕ
Неделя 2: первый срез у пилота	Ранняя обратная связь — рецептуры	Пилот на первом срезе — стандарт для крупных фич
Неделя 5: задержка API платежей	Устная договорённость	Доска зависимостей для всех команд
Неделя 7: нагрузочный тест	Нашли 4 проблемы до релиза	Нагрузочный тест в критериях релиза
Неделя 11: пауза раскатки для кафе	Автоматические пороги	Шаблон раскатки с порогами
День 23: инцидент миграции	Нет проверки блокировок	Миграции только с проверкой на копии прода

ПРАВИЛО

Не больше трёх действий за ретро, у каждого — владелец и срок. Первые пять минут следующей ретро — проверка, выполнены ли они. Невыполненное действие либо переносят с причиной, либо закрывают.

КЛАССИКА Норм Керт, «Project Retrospectives», 2001

Главная директива ретроспективы

Керт сформулировал установку для ретроспектив: «Независимо от того, что мы обнаружим, мы понимаем и искренне верим, что каждый сделал всё, что мог, с учётом своих знаний, навыков, доступных ресурсов и ситуации».

Директива создаёт безопасность: можно говорить о проблемах, не опасаясь, что разговор станет поиском виноватого.

МИРОВОЙ КЕЙС «Agile Retrospectives», Эстер Дерби и Диана Ларсен, 2006

Ретроспективы в больших масштабах

Авторы описали пятишаговую структуру ретроспективы — настрой, сбор данных, выводы, решения, закрытие — и десятки упражнений для каждого шага. Книга стала стандартом для фасилитаторов.

Главный совет — менять форматы: одинаковая ретро каждые две недели надоедает, и люди перестают думать.

ПЛОХО

Ритуал жалоб

Семь действий без владельцев, выполнено два.

Через месяц те же проблемы.

ХОРОШО

Ретро с изменениями

До трёх действий с владельцами, проверка в начале следующей.

87% выполнения, проблемы не повторяются.

33.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Частота	раз в спринт или раз в 2 недели
Длительность	45–90 минут
Действий за ретро	1–3
Выполнение действий	от 70%

33.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Много действий	Не выполняются	До трёх
Нет проверки действий	Ритуал	Первые 5 минут — проверка
Руководитель ведёт и оценивает	Молчание	Нейтральный фасилитатор
Один формат всегда	Скука	Менять форматы

ПРИМЕР

Маркетплейс

Команда маркетплейса годами проводила ретро без данных, и каждый раз обсуждали «слишком много встреч». Когда на ретро вывели метрику — 11 часов встреч на человека в неделю, — команда за одну встречу отменила три регулярных совещания и сократила стендап. Через спринт метрика упала до 6 часов.

33.7 Вопросы для самопроверки

- Из каких шагов состоит ретроспектива?
- Какие форматы бывают и когда их использовать?
- Как посчитать долю выполненных действий?
- Что такое главная директива ретроспективы?
- Почему действий должно быть не больше трёх?

ПРАКТИКА 33 после главы 33

Проведите ретроспективу, которая меняет работу.

ЧТО СДЕЛАТЬ

01	Начните с проверки действий прошлой ретро.	☐
02	Принесите 2–3 метрики спринта.	☐
03	Выберите формат под ситуацию.	☐
04	Договоритесь о 1–3 действиях с владельцами и сроками.	☐
05	Посчитайте долю выполненных действий по формуле (33.1) через месяц.	☐

РЕЗУЛЬТАТ Попросите кого-то из команды провести следующую ретро.

34 Выявление инсайтов для будущих итераций

После неё вы собираете выводы из метрик, обращений, разборов и ретроспектив в единый поток инсайтов, превращаете их в гипотезы для следующих итераций и замыкаете цикл между delivery и discovery.

34.1 Источники инсайтов после релиза
34.2 От инсайта к гипотезе
34.3 Ценность инсайта
34.4 Кейс «Чека»: реестр инсайтов
34.5 Бенчмарки
34.6 Анти-паттерны
34.7 Вопросы для самопроверки

Каждый релиз приносит знания: как пользователи на самом деле используют функцию, где они спотыкаются, чего им не хватает, что в процессе сработало, а что нет. Если эти знания остаются в разных отчётах — дашборде аналитика, тикетах поддержки, заметках ретро, — следующая итерация начинается с нуля. Задача продакта — собрать их в одном месте и превратить в решения.

34.1 Источники инсайтов после релиза

ИСТОЧНИК	ЧТО ДАЁТ	«ОСТАТКИ»
Метрики и когорты	Кто и как использует, где отваливается	Кафе осваивают хуже магазинов
Обращения в поддержку	Проблемы и вопросы	«Минус», сканер, рецептуры
Интервью с активными и ушедшими	Почему так	Сети хотят остатки по всем точкам
Продажи	Возражения и запросы	«Склад+» продаётся после вебинаров
Разборы инцидентов	Риски платформы	Миграции, синхронизация
Ретроспективы	Процесс	Пилот на первом срезе работает

34.2 От инсайта к гипотезе

```
Наблюдение: 6 сетевых клиентов просили остатки по всем точкам, 2 ушли
Данные: сети — 12% точек и 21% MRR; отток сетей 1,4% против 1,9% у одиночных
Инсайт: сетям нужна сводная картина и перемещения, без них они уходят к складским системам
Гипотеза: если дать сводные остатки и перемещения, отток сетей снизится до 1,0%
 и 30% сетей перейдут на «Склад+» за 6 месяцев
Проверка: интервью с 8 сетями + прототип сводного отчёта за 2 недели
```

34.3 Ценность инсайта

ФОРМУЛА 34.1 · ОЖИДАЕМАЯ ЦЕННОСТЬ ГИПОТЕЗЫ

$$EV = p \cdot \Delta MRR \cdot T - C \tag{34.1}$$

где

p	—	вероятность, что гипотеза верна	доля
ΔMRR	—	прирост MRR при успехе	₽/мес
T	—	горизонт оценки	мес
C	—	стоимость проверки и разработки	₽

```
Сети: p = 0,5; ΔMRR ≈ 0,4 п.п. оттока × 9,4 млн ₽ MRR сетей + переходы ≈ 0,6 млн ₽/мес;
T = 12; C ≈ 4,8 млн ₽ (6 недель команды) → EV ≈ 0,5 · 7,2 - 4,8 = -1,2 млн. Проверка за
2 недели (0,6 млн) нужна, чтобы поднять p или отказаться
```

Расчёт показал, что браться за сети «вслепую» невыгодно, а дешёвая проверка — выгодна. Две недели интервью и прототипа подняли уверенность: 7 из 8 сетей подтвердили проблему, 5 согласились на пилот. С p = 0,8 ожидаемая ценность стала положительной — около 1 млн ₽ за год.

34.4 Кейс «Чека»: реестр инсайтов

Ника завела единый реестр инсайтов после релиза: каждая запись — наблюдение, источник, данные, гипотеза и решение. Раз в две недели она разбирала его с командой и Яном.

ИНСАЙТ	ИСТОЧНИК	РЕШЕНИЕ
Сети уходят без сводных остатков	Продажи, интервью, отток	Discovery 2 недели → в план квартала
Кафе нужны рецептуры при подключении	A/B, поддержка	Сделано: шаблоны по умолчанию
«Склад+» лучше продаётся после обучения	Продажи, источники	Еженедельные вебинары
Уведомление «заканчивается» открывают 70%, но заказывают у поставщика вручную	Метрики	Гипотеза: заказ поставщику из уведомления
Пилот на первом срезе находит главные проблемы	Ретро	Стандарт для всех команд

ПРАВИЛО

Каждый инсайт после релиза получает решение: в discovery, в бэклог, в процесс или «не делаем, потому что». Инсайт без решения — потерянное знание.

КЛАССИКА Эрик Рис, 2011

Создать — измерить — научиться

Цикл обратной связи Риса замыкает delivery и discovery: каждая поставка — эксперимент, а измерения — источник новых гипотез. Скорость обучения важнее скорости поставки.

Команда, которая выпускает часто, но не учится, просто быстрее делает ошибки.

МИРОВОЙ КЕЙС Slack, 2013

Функция из поведения пользователей

Slack вырос из внутреннего инструмента общения игровой студии Tiny Speck. Игра провалилась, а команда заметила, что её чат — самая ценная часть работы, и превратила его в продукт.

Инсайт пришёл не из исследования рынка, а из наблюдения за собственным использованием — так же, как многие инсайты после релиза приходят из поведения, а не из ожиданий.

ПЛОХО

Знания по углам

Метрики у аналитика, обращения в поддержке, выводы ретро в заметках.

Сети уходят, команда узнаёт через квартал.

ХОРОШО

Реестр инсайтов

Наблюдение, данные, гипотеза, решение, разбор раз в 2 недели.

Сети — проверены и в плане следующего квартала.

34.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Разбор реестра инсайтов	раз в 2 недели
Инсайтов без решения	0
Проверка крупной гипотезы	1–4 недели
Доля плана квартала из инсайтов после релизов	заметная часть

34.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Знания в разных местах	Потери	Единый реестр
Сразу в разработку	Дорогие ошибки	Проверка гипотезы
Только метрики	Нет «почему»	Интервью и поддержка
Инсайты без решения	Повторение проблем	Решение по каждому

ПРИМЕР

Saas

Сервис онлайн-документооборота заметил, что 30% клиентов скачивают документы и подписывают их вне сервиса. Это наблюдение из метрик привело к интервью, а затем к встроенной электронной подписи, которая стала главным драйвером перехода на платный тариф.

34.7 Вопросы для самопроверки

- Какие источники инсайтов есть после релиза?
- Как превратить наблюдение в гипотезу?
- Как оценить ожидаемую ценность гипотезы?
- Зачем нужен реестр инсайтов?
- Почему скорость обучения важнее скорости поставки?

ПРАКТИКА 34 после главы 34

Соберите инсайты после последнего релиза.

ЧТО СДЕЛАТЬ

01	Пройдите по 6 источникам и выпишите наблюдения.	☐
02	Для 3 главных сформулируйте инсайт и гипотезу.	☐
03	Оцените ожидаемую ценность по формуле (34.1).	☐
04	Примите решение по каждому инсайту.	☐
05	Заведите реестр и назначьте ритм разбора.	☐

РЕЗУЛЬТАТ Покажите реестр команде — часть инсайтов окажется их.

35 Автоматизация CI/CD и тестирования

После неё вы понимаете, как устроен конвейер непрерывной интеграции и поставки, какие этапы в нём нужны, где автоматизация окупается быстрее всего и как продакт помогает найти для неё время и аргументы.

35.1 Этапы конвейера
35.2 Окупаемость автоматизации
35.3 Кейс «Чека»: квартал на конвейер
35.4 Бенчмарки
35.5 Анти-паттерны
35.6 Вопросы для самопроверки

CI/CD — конвейер, который автоматически собирает, проверяет и выкатывает изменения. Непрерывная интеграция (CI) означает, что каждое изменение сразу объединяется с общим кодом и проходит тесты. Непрерывная поставка (CD) — что любое прошедшее проверки изменение можно выпустить одной кнопкой или автоматически. Без конвейера частые маленькие релизы невозможны: каждый требует часов ручной работы.

35.1 Этапы конвейера

ЭТАП	ЧТО ПРОИСХОДИТ	«ЧЕК» ДО	«ЧЕК» ПОСЛЕ
Сборка	Компиляция, зависимости	вручную у Егора	автоматически на каждое изменение, 4 мин
Статические проверки	Линтеры, безопасность	нет	2 мин
Тесты	Unit, интеграционные, контрактные	частично, 25 мин	10 мин параллельно
Сборка артефакта	Образ, подписанная сборка кассы	вручную	автоматически
Стенд	Выкатка на тестовое окружение	раз в неделю	на каждое изменение
E2E и регресс	Ключевые сценарии	3 дня вручную	40 мин

ЭТАП	ЧТО ПРОИСХОДИТ	«ЧЕК» ДО	«ЧЕК» ПОСЛЕ
Выкатка в прод	Канарейка → раскатка	ручная, ночью	кнопка, днём, 5% → 100%

35.2 Окупаемость автоматизации

ФОРМУЛА 35.1 · СРОК ОКУПАЕМОСТИ АВТОМАТИЗАЦИИ

$$T = \frac{C_{\text{авт}}}{n \cdot (t_{\text{руч}} - t_{\text{авт}}) \cdot h} \tag{35.1}$$

где

$C_{\text{авт}}$	—	затраты на автоматизацию	₽
n	—	число выполнений в месяц	раз
$t_{\text{руч}}, t_{\text{авт}}$	—	время одного выполнения вручную и автоматически	ч
h	—	стоимость часа	₽
T	—	срок окупаемости	мес

Регресс «Чека»: автоматизация 2 инженера × 8 недель ≈ 2 млн ₽; раньше 2 регресса в месяц × 96 ч (4 тестировщика × 3 дня) → сейчас 20 релизов × 1 ч. Экономия ≈ 192 - 20 = 172 ч × 3 000 ₽ ≈ 0,52 млн/мес → T ≈ 4 месяца, не считая снижения инцидентов

35.3 Кейс «Чека»: квартал на конвейер

Егор давно просил время на CI/CD, но проигрывал фичам в приоритизации. Ника помогла сформулировать аргумент в деньгах: стоимость задержки из-за ожидания релизов, стоимость ручного регресса и code freeze, потери от инцидентов. По WSJF автоматизация оказалась выше большинства фич, и на неё выделили 8 недель двух инженеров платформы.

МЕТРИКА DORA	ДО	ПОСЛЕ КВАРТАЛА	КОНЕЦ ГОДА
Частота деплоев	раз в 2 недели	2–3 раза в неделю	ежедневно
Время поставки изменения	9 дней	3 дня	1,5 дня
Доля неудачных изменений	18%	9%	6%
Время восстановления	4 часа	1,5 часа	1 час

ПРАВИЛО

Помогайте техлиду формулировать технические инвестиции на языке бизнеса: стоимость задержки, экономия часов, снижение инцидентов. Автоматизация, которую нельзя объяснить в деньгах и метриках, проигрывает фичам.

КЛАССИКА — Джез Хамбл и Дэвид Фарли, «Continuous Delivery», 2010

Непрерывная поставка

Авторы описали конвейер развёртывания как главный механизм надёжной поставки: каждое изменение проходит одинаковый автоматический путь от коммита до прода, а релиз становится скучным и безопасным событием.

Их принцип: если что-то болезненно, делайте это чаще — тогда придётся автоматизировать и процесс станет лёгким.

МИРОВОЙ КЕЙС — мобильные команды крупных приложений

Мобильные релизы раз в неделю

Многие крупные мобильные приложения выпускают обновления в магазины по фиксированному еженедельному или двухнедельному поезду: что готово и прошло проверки — едет, что нет — ждёт следующего. Функции внутри включаются флагами независимо от версии.

Для кассового приложения «Чека» принцип тот же: сборки уходят на точки поэтапно, а новые функции включаются флагами с сервера.

ПЛОХО

Ручной релиз

Сборка у одного человека, регресс 3 дня, выкатка ночью.

Релиз раз в две недели, каждый пятый — с откатом.

ХОРОШО

Конвейер

Автоматические сборка, тесты, стенд, канарейка.

Ежедневные деплои, 6% неудачных, восстановление за час.

35.4 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Время конвейера до стенда	до 15–20 минут

ПАРАМЕТР	ОРИЕНТИР
Окупаемость автоматизации регресса	обычно 3–9 месяцев
Ручных шагов в выкатке	0–1 (подтверждение)
Доля ёмкости платформы на конвейер	постоянная, не разовый проект

35.5 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Знание сборки у одного человека	Риск и очередь	Конвейер как код
Автоматизация без бизнес-аргумента	Проигрывает фичам	Деньги и метрики
Долгий конвейер	Обходят проверки	Параллельность, пирамида
Выкатка ночью вручную	Ошибки, усталость	Канарейка днём

ПРИМЕР

Финтех

Банковская команда считала, что регулирование не позволяет частые релизы. Аудит показал, что требования касались контроля изменений, а не частоты. Конвейер с автоматическим журналом проверок и подтверждением ответственного удовлетворил аудиторов, а частота релизов выросла с ежемесячной до еженедельной.

35.6 Вопросы для самопроверки

- Чем непрерывная интеграция отличается от непрерывной поставки?
- Какие этапы есть в конвейере?
- Как посчитать срок окупаемости автоматизации?
- Как продакт может помочь найти время на автоматизацию?
- Как изменились метрики DORA «Чека»?

ПРАКТИКА 35 после главы 35

Оцените конвейер поставки вашей команды.

ЧТО СДЕЛАТЬ

01	Попросите техлида нарисовать путь изменения от коммита до прода.	☐
02	Отметьте ручные шаги и их длительность.	☐
03	Посчитайте окупаемость автоматизации самого долгого шага по формуле (35.1).	☐
04	Сформулируйте аргумент в деньгах и метриках.	☐
05	Предложите автоматизацию в приоритизацию квартала.	☐

РЕЗУЛЬТАТ Снимайте метрики DORA ежемесячно, чтобы видеть эффект.

36 Налаживание процессов Continuous Delivery

После неё вы выстраиваете непрерывную поставку как систему практик — маленькие изменения, флаги, автоматические проверки, мониторинг и откат, — внедряете её поэтапно и понимаете, что меняется в работе продакта, когда релиз перестаёт быть событием.

36.1 Практики непрерывной поставки
36.2 Поэтапное внедрение
36.3 Скорость обучения
36.4 Кейс «Чека»: распространение на все команды
36.5 Бенчмарки
36.6 Анти-паттерны
36.7 Вопросы для самопроверки

Конвейер CI/CD — техническая основа. Непрерывная поставка — способ работы всей команды: изменения маленькие, любое изменение в основной ветке готово к выпуску, функции включаются флагами, качество проверяется автоматически, а если что-то пошло не так, откат занимает минуты. Для продакта это значит, что релиз перестаёт быть датой и становится решением «когда включить».

36.1 Практики непрерывной поставки

ПРАКТИКА	ЧТО ДАЁТ	ГДЕ В ГАЙДЕ
Маленькие вертикальные срезы	Меньше риск, быстрее обратная связь	Глава 11
Работа в основной ветке	Нет долгих слияний	—
Флаги функций	Деплой отделён от релиза	Глава 24
Автоматические проверки	Качество на каждом изменении	Главы 19–20, 35
Канареечная выкатка с порогами	Малый радиус поражения	Глава 24
Мониторинг и SLO	Быстрое обнаружение	Главы 27–30
Быстрый откат	Восстановление за минуты	Глава 28

36.2 Поэтапное внедрение

ОТ РЕЛИЗА РАЗ В СПРИНТ К ПОСТАВКЕ ПО ГОТОВНОСТИ

1. **Измерить**
   DORA и поток: точка отсчёта.
2. **Автоматизировать критичное**
   Сборка, регресс ключевых сценариев, выкатка.
3. **Ввести флаги**
   Незавершённое — за флагами, релиз по готовности.
4. **Уменьшить партии**
   Срезы, лимиты WIP, частые деплои.
5. **Канарейка и пороги**
   Автоматическая остановка по защитным метрикам.
6. **Распространить**
   Команда за командой, с наставниками из первой.

36.3 Скорость обучения

ФОРМУЛА 36.1 · ЧИСЛО ЦИКЛОВ ОБРАТНОЙ СВЯЗИ В ГОД

$$L = \frac{W}{t_{\text{пост}} + t_{\text{изм}}} \tag{36.1}$$

где

W	—	рабочих дней в году	дн.
$t_{\text{пост}}$	—	время от идеи до пользователя	дн.
$t_{\text{изм}}$	—	время до первых данных об эффекте	дн.
L	—	сколько раз за год можно проверить идею и изменить курс	раз

«Чек» в начале года: 248 / (74 + 30) ≈ 2,4 цикла на направление. В конце года: 248 / (34 + 14) ≈ 5,2. Вдвое больше попыток найти то, что работает

36.4 Кейс «Чека»: распространение на все команды

Команда кассы первой перешла на поставку по готовности. Затем Егор и Алина распространяли практики по одной команде в месяц, а инженер из первой команды две недели работал в каждой следующей как наставник.

КОМАНДА	МЕСЯЦ ПЕРЕХОДА	ДЕПЛОЕВ В НЕДЕЛЮ	НЕУДАЧНЫХ ИЗМЕНЕНИЙ
Касса	3	4–5	6%
Остатки	4	3–4	5%
Кабинет	5	5–7	4%
Интеграции	6	3	8% (внешние партнёры)

Изменилась и работа Ники. Вместо подготовки к «большому релизу» раз в две недели она решала, когда включать функции для сегментов, и смотрела на метрики каждый день. Релиз-ноты стали еженедельным дайджестом, а не событием.

ПРАВИЛО

Внедряйте непрерывную поставку по командам, начиная с той, где боль сильнее и техлид мотивирован. Первая команда становится примером и источником наставников для остальных.

КЛАССИКА — Джин Ким и др., «The DevOps Handbook», 2016

Три пути DevOps

Авторы описали три принципа: поток (быстрое движение работы от разработки к пользователям), обратная связь (быстрое обнаружение и исправление проблем) и непрерывное обучение (эксперименты и улучшение процесса).

Непрерывная поставка — реализация всех трёх: маленькие партии ускоряют поток, мониторинг и канарейки дают обратную связь, ретро и разборы — обучение.

МИРОВОЙ КЕЙС многочисленные трансформации DevOps

От квартальных релизов к ежедневным

Отчёты State of DevOps описывают типичный путь компаний: от квартальных релизов с длинной стабилизацией к еженедельным и затем ежедневным. Переход обычно занимает от полугода до двух лет и идёт командами, а не одним решением сверху.

Главное препятствие — не инструменты, а привычки: страх частых релизов, ручные согласования, большие фичи.

ПЛОХО

Внедрение приказом

«С понедельника все релизят каждый день».

Без автотестов и флагов — волна инцидентов и откат к старому.

ХОРОШО

Команда за командой

Первая команда, наставники, месяц на каждую следующую.

За полгода все 4 команды деплоят еженедельно и чаще.

36.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Срок перехода одной команды	1–3 месяца
Срок перехода компании	6–24 месяца
Деплоев в неделю у команды	от нескольких
Неудачных изменений	до 10–15%

36.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Частые релизы без автоматизации	Инциденты	Сначала проверки и флаги
Внедрение приказом	Сопротивление	Пример и наставники
Только инструменты	Старые привычки	Практики команды
Релиз-ноты на каждое изменение	Шум	Дайджест

ПРИМЕР

Медиа

Медиакомпания перешла на непрерывную поставку сайта и увидела побочный эффект: редакция стала сама предлагать эксперименты с подачей материалов, потому что изменения выходили за день, а не за месяц. Число экспериментов выросло с 4 до 30 в квартал.

36.7 Вопросы для самопроверки

- Чем непрерывная поставка отличается от конвейера CI/CD?
- Какие практики входят в непрерывную поставку?
- Как посчитать число циклов обратной связи в год?
- Как внедрять непрерывную поставку в нескольких командах?
- Что меняется в работе продакта?

ПРАКТИКА 36 после главы 36

Составьте план перехода к непрерывной поставке.

ЧТО СДЕЛАТЬ

01	Измерьте DORA и время от идеи до пользователя.	☐
02	Отметьте, какие из 7 практик уже есть.	☐
03	Посчитайте циклы обратной связи по формуле (36.1).	☐
04	Выберите первую команду и 2–3 шага на ближайший месяц.	☐
05	Договоритесь о наставничестве для следующих команд.	☐

РЕЗУЛЬТАТ Обсуждайте прогресс на ежемесячной встрече техлидов и продактов.

37 Управление несколькими командами и продуктами

После неё вы организуете delivery в нескольких командах: выстраиваете общий ритм планирования, синхронизируете зависимости и цели, распределяете ответственность между продактами и не превращаете координацию в бюрократию.

37.1 Структура команд · 37.2 Ритм синхронизации · 37.3 Нагрузка координации · 37.4 Кейс «Чека»: квартальное планирование · 37.5 Бенчмарки · 37.6 Анти-паттерны · 37.7 Вопросы для самопроверки

Когда команда одна, delivery — это её внутренний процесс. Когда команд четыре и больше, появляются новые задачи: общие цели, зависимости между командами, общая платформа, единые стандарты качества и релизов. Ошибка — решать их через слои менеджеров и согласований. Лучше — через структуру команд, общие правила и лёгкие регулярные синхронизации.

37.1 Структура команд

ТИП КОМАНДЫ	ЗА ЧТО ОТВЕЧАЕТ	«ЧЕК»
Потоковая (stream-aligned)	Сценарий или сегмент клиентов целиком	Касса, Остатки, Кабинет
Платформенная	Внутренние сервисы и инструменты для других команд	Платформа: CI/CD, мониторинг, синхронизация
Команда интеграций	Внешние системы и партнёры	Интеграции: эквайринг, ОФД, маркетплейсы
Помогающая (enabling)	Временно учит другие команды	Наставники непрерывной поставки

КЛАССИКА Мэттью Скелтон и Мануэль Паис, «Team Topologies», 2019

Топологии команд

Авторы предложили четыре типа команд — потоковые, платформенные, помогающие и команды сложных подсистем — и три режима взаимодействия: совместная работа, сервис и обучение.

Главная идея — снизить когнитивную нагрузку команд: потоковая команда должна делать фичу целиком, опираясь на платформу как на сервис, а не ждать других.

37.2 Ритм синхронизации

СОБЫТИЕ	ЧАСТОТА	УЧАСТНИКИ	ЗАЧЕМ
Квартальное планирование	раз в квартал	продакты, техлиды, руководство	Цели, распределение тем, крупные зависимости
Синк продактов	еженедельно, 30 мин	продакты + руководитель продукта	Приоритеты, конфликты, общие решения
Синк техлидов	еженедельно, 30 мин	техлиды + платформа	Архитектура, зависимости, инциденты
Демо для всех	раз в 2 недели	все желающие	Видимость результатов
Обзор метрик	ежемесячно	продакты, аналитика	North Star и цели команд

37.3 Нагрузка координации

ФОРМУЛА 37.1 · ЧИСЛО КАНАЛОВ КОММУНИКАЦИИ

$$K = \frac{n(n-1)}{2} \tag{37.1}$$

где

n — число команд, которым нужно договариваться между собой — шт.

K — число пар, требующих координации — шт.

4 команды, каждая с каждой: K = 6 пар. 8 команд — 28 пар. Если же потоковые команды опираются на платформу как на сервис и договариваются только с ней, при 7 потоковых командах каналов 7, а не 28

37.4 Кейс «Чека»: квартальное планирование

К концу года «Чек» планировал рост до шести команд: добавлялись команда сетей и команда подключения новых точек. Ника стала ведущим продактом и провела первое общее квартальное планирование: полдня, все продакты и техлиды, одна доска с целями и зависимостями.

КОМАНДА	ЦЕЛЬ КВАРТАЛА	ЗАВИСИМОСТИ
Сети (новая)	Сводные остатки и перемещения: отток сетей 1,4 → 1,0%	Остатки (модель данных), Платформа (синхронизация)
Остатки	Заказ поставщику из уведомления	—
Касса	SLO 99,9% без заморозок	Платформа
Кабинет	Отчёт по прибыльности для «Склад+»	Остатки
Подключение (новая)	Время до первого чека 3 → 1 день	Интеграции
Платформа	Многопоточная синхронизация	—

На доске сразу стало видно, что модель данных остатков нужна трём командам. Её владельцем назначили команду «Остатков», а изменения других команд — через внутренний опенсорс с ревью владельца. Планирование заняло 4 часа вместо прошлогодних двух недель переписки.

ПРАВИЛО

У каждого общего компонента и набора данных — одна команда-владелец. Другие команды вносят изменения через её ревью, а не ждут в очереди и не правят без спроса.

МИРОВОЙ КЕЙС SAFe, LeSS, Nexus

Масштабирование фреймворками

Для координации многих команд существуют фреймворки: SAFe с инкрементами планирования на 8–12 недель и большим планированием всех команд, LeSS с одним бэклогом на несколько команд, Nexus как расширение Scrum.

Фреймворки помогают крупным организациям, но несут риск бюрократии. Многие компании начинают с лёгкого ритма — квартальное планирование, синки, демо — и усложняют только при необходимости.

ПЛОХО

Координация перепиской

Каждая команда планирует сама, зависимости выясняются в разработке.

Две недели согласований, три команды правят одну модель данных.

ХОРОШО

Общий ритм и владельцы

Полдня квартального планирования, одна доска, владельцы компонентов.

Зависимости видны до старта, правила изменений понятны.

37.5 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Квартальное планирование	полдня — день
Синки продактов и техлидов	по 30 минут в неделю
Размер команды	5–9 человек
Время на координацию у продакта	до 15–20%

37.6 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Слои согласований	Медленные решения	Ритм и правила
Общий компонент без владельца	Конфликты	Одна команда-владелец
Каждая с каждой	Квадратичный рост координации	Платформа как сервис
Тяжёлый фреймворк сразу	Бюрократия	Лёгкий ритм, усложнять по необходимости

ПРИМЕР

Финтех

Финтех-компания выросла с 3 до 12 команд за год и ввела SAFe целиком. Через полгода продакты тратили 40% времени на встречи. Компания оставила только квартальное планирование и еженедельные синки, а остальные события сделала необязательными — время на встречи сократилось вдвое без роста срывов.

37.7 Вопросы для самопроверки

- Какие типы команд описывают «Топологии команд»?
- Какие события нужны для синхронизации нескольких команд?
- Как растёт число каналов коммуникации с числом команд?
- Зачем каждому общему компоненту владелец?
- Чем рискуют тяжёлые фреймворки масштабирования?

ПРАКТИКА 37 после главы 37

Наладьте координацию между командами.

ЧТО СДЕЛАТЬ

01	Определите тип каждой команды.	☐
02	Посчитайте каналы координации по формуле (37.1) и подумайте, как их сократить.	☐
03	Назначьте владельцев общих компонентов и данных.	☐
04	Проведите квартальное планирование на одной доске.	☐
05	Введите еженедельные синки продактов и техлидов.	☐

РЕЗУЛЬТАТ Через квартал спросите продактов, сколько времени уходит на координацию.

38 Баланс discovery и delivery в долгосрочной перспективе

После неё вы распределяете ёмкость команды между исследованием, поставкой, техническим долгом и поддержкой, не даёте delivery вытеснить discovery и строите продуктовую систему, которая надёжно поставляет то, что стоит поставлять.

38.1 Распределение ёмкости · 38.2 Метрики обоих потоков · 38.3 Кейс «Чека»: итоги года · 38.4 Бенчмарки · 38.5 Анти-паттерны · 38.6 Вопросы для самопроверки

Под давлением сроков delivery всегда побеждает: исследование можно отложить, а релиз — нет. Через год такой команда быстро и надёжно выпускает фичи, которые никому не нужны, — то, с чего начинался этот гайд. Обратная крайность — бесконечное исследование без поставки. Баланс не устанавливается сам, его задают явно: долями ёмкости, ритмом и метриками обоих потоков.

38.1 Распределение ёмкости

Поток	Доля «Чека»	Что входит
Новые возможности (delivery)	55%	Фичи из проверенных гипотез
Discovery	15%	Интервью, прототипы, эксперименты, фальшивые двери
Технический долг и платформа	20%	Реестр долга, конвейер, надёжность
Поддержка и инциденты	10%	Ошибки, обращения, дежурства

ФОРМУЛА 38.1 · ДОЛЯ ПРОВЕРЕННЫХ ИДЕЙ В DELIVERY

$$V = \frac{n_{\text{пров}}}{n_{\text{дел}}} \cdot 100\% \qquad (38.1)$$

где

$n_{\text{пров}}$ — фич в delivery, прошедших проверку ценности — шт.

$n_{\text{дел}}$ — всего фич, взятых в delivery — шт.

V — доля проверенных — %

```
«Чек»: два года назад 5 из 24 → 21%. В этом году 11 из 13 → 85%. Доля фич, которыми
пользуются ≥ 20% целевых точек: 38% → 77%
```

38.2 Метрики обоих потоков

ПОТОК	МЕТРИКА	«ЧЕК» В НАЧАЛЕ ГОДА → В КОНЦЕ
Discovery	Гипотез проверено за квартал	2 → 9
Discovery	Отказано до разработки	0 → 4 за квартал
Delivery	Время от идеи до прода	74 → 34 дня
Delivery	Доля неудачных изменений	18 → 6%
Результат	Отток точек	2,4 → 1,9%
Результат	MRR	44,8 → 54,6 млн ₽

38.3 Кейс «Чека»: итоги года

В конце года Ника подготовила для руководства итог: что изменилось в поставке и что это дало бизнесу. Отток не дошёл до цели 1,8% — остановился на 1,9%, — а MRR вырос до 54,6 млн ₽ при цели 58. Главный вклад продукта — «Остатки» и надёжность — был доказан холдаутом; недобор объяснялся медленным стартом продаж «Склад+» и уходом части сетей до появления сводных остатков.

Ещё важнее были изменения системы. Четыре команды деплоили ежедневно или несколько раз в неделю, фичи проходили проверку ценности, а каждый релиз заканчивался оценкой по метрикам. На следующий год цель поставили не только по оттоку, но и по скорости обучения: не меньше 10 проверенных гипотез в квартал.

ПРАВИЛО

Задайте доли ёмкости на discovery, delivery, техдолг и поддержку явно и пересматривайте их раз в квартал. Если discovery систематически получает меньше половины своей доли, через полгода команда начнёт поставлять непроверенное.

КЛАССИКА — Тереза Торрес, 2021; Марти Каган, 2020

Продуктовая система

Торрес описывает зрелую продуктовую команду как ту, что одновременно и непрерывно ведёт discovery и delivery: еженедельные контакты с клиентами, быстрые эксперименты и регулярная поставка. Каган добавляет, что продуктовая команда отвечает за результат, а не за выпуск.

Обе модели сходятся: ни поток discovery, ни поток delivery не ценны сами по себе — ценна их связка.

МИРОВОЙ КЕЙС — Чарльз О'Райли и Майкл Ташман, «амбидекстрия», 2004

Инновации и эксплуатация

Исследователи организаций показали, что успешные компании умеют одновременно эффективно эксплуатировать текущий бизнес и искать новые возможности. Те, кто делает только одно, либо стагнируют, либо не могут масштабироваться.

Баланс discovery и delivery — та же амбидекстрия на уровне продуктовой команды.

ПЛОХО

Delivery вытесняет всё

Discovery «когда будет время», долг «потом».

Быстрая поставка ненужного, рост инцидентов.

ХОРОШО

Явные доли и метрики

55 / 15 / 20 / 10, метрики обоих потоков, пересмотр раз в квартал.

85% проверенных фич, 6% неудачных изменений.

38.4 Бенчмарки

ПАРАМЕТР	ОРИЕНТИР
Доля discovery	10–20%
Доля техдолга и платформы	15–25%

ПАРАМЕТР	ОРИЕНТИР
Доля проверенных фич в delivery	от 70%
Пересмотр долей	раз в квартал

38.5 Анти-паттерны

АНТИ-ПАТТЕРН	ЧЕМ ВРЕДИТ	ЧТО ДЕЛАТЬ
Discovery по остаточному принципу	Непроверенные фичи	Явная доля
Только метрики delivery	Скорость без ценности	Метрики обоих потоков
Цели года только по выручке	Нет стимула учиться	Цели по обучению
Доли без контроля	Тихое вытеснение	Квартальный пересмотр

ПРИМЕР

Saas

B2B-сервис после года «только delivery» увидел, что новые функции почти не влияют на удержание. Команда выделила 15% ёмкости на discovery и ввела метрику «гипотез проверено». Через год две из трёх крупных фич дали измеримый эффект, а удержание клиентов на 12 месяцев выросло на 6 п.п.

38.6 Вопросы для самопроверки

- Почему delivery вытесняет discovery под давлением сроков?
- Как распределить ёмкость между потоками?
- Как посчитать долю проверенных идей в delivery?
- Какие метрики нужны обоим потокам?
- Что такое амбидекстрия организации?

ПРАКТИКА 38 после главы 38

Настройте баланс потоков на следующий квартал.

ЧТО СДЕЛАТЬ

01	Посчитайте фактическое распределение ёмкости за прошлый квартал.	☐
02	Задайте целевые доли четырёх потоков.	☐
03	Посчитайте долю проверенных фич по формуле (38.1).	☐
04	Выберите метрики discovery и delivery.	☐
05	Запланируйте пересмотр через квартал.	☐

РЕЗУЛЬТАТ Покажите руководству метрики обоих потоков рядом с выручкой.

A Шаблоны документов

Готовые формы для ключевых этапов delivery. Копируйте, заполняйте, адаптируйте под свою команду.

A.1 Одностраничник фичи · A.2 История с критериями приёмки · A.3 Доска зависимостей · A.4 План раскатки · A.5 Статус инцидента · A.6 Разбор инцидента · A.7 Оценка релиза

A.1 Одностраничник фичи

```
Проблема: что не получается у клиентов и как мы это знаем (данные)
Связь с целью: путь по дереву целей до цели компании
Outcome: метрика · база · цель · срок
Защитные метрики: что не должно ухудшиться
Объём: Must · Should · Could · Won't
Срезы: 1… 2… 3… — каждый с пилотом
Зависимости: владелец · дата · критерий
Запуск: флаг · раскатка · холдаут · GTM
Журнал решений: дата · решение · кто · почему
```

A.2 История с критериями приёмки

```
Как [роль] / Когда [ситуация], я хочу [действие], чтобы [цель]
Критерии:
 Дано: [начальное состояние]  Когда: [действие]  Тогда: [наблюдаемый результат с числами]
 Крайние случаи: ноль · минус · офлайн · большие объёмы · права
Макет: ссылка на фрейм · Аналитика: события · Зависимости: …
```

A.3 Доска зависимостей

ЗАВИСИМОСТЬ	ОТ КОГО	ЧТО НУЖНО	ДАТА	КРИТЕРИЙ ГОТОВНОСТИ	СТАТУС	ПЛАН Б
					в срок / риск / задержка	

A.4 План раскатки

ШАГ	СЕГМЕНТ И ДОЛЯ	ДЛИТЕЛЬНОСТЬ	ПОРОГИ ОСТАНОВКИ	ОТВЕТСТВЕННЫЙ
1	5%	2–3 дня	ошибки · скорость · обращения	
2	25%	3–5 дней		
3	50%	3–5 дней		
4	100% (холдаут __%)	—		

A.5 Статус инцидента

```
[Время] Статус: расследуем / нашли причину / исправляем / решено
Что не работает: [на языке клиента]
Кого затрагивает: [сегмент, примерная доля]
Что делать клиентам сейчас: [обходной путь, если есть]
Следующее обновление: [время]
```

A.6 Разбор инцидента

```
Кратко: что произошло, влияние (клиенты × время), уровень
Хронология: обнаружение → реакция → восстановление
Причины: техническая · процессная · почему не поймали раньше
Что сработало / что нет
Действия: что · владелец · срок
```

A.7 Оценка релиза

МЕТРИКА	ЦЕЛЬ ИЗ PRD	ФАКТ	КОНТРОЛЬ / ХОЛДАУТ	ЭФФЕКТ	ВЫВОД
					развивать / исправлять / сворачивать

Б Чек-листы по этапам

Короткие проверки перед переходом к следующему этапу. Невыполненный пункт — не запрет двигаться дальше, а осознанный риск.

Б.1 Перед взятием фичи в delivery · Б.2 Перед спринтом · Б.3 Перед релизом · Б.4 В первую неделю после релиза · Б.5 Через 1 и 3 месяца · Б.6 Раз в квартал

Б.1 Перед взятием фичи в delivery

- [ ] Проблема доказана данными, а не только запросом одного клиента.
- [ ] Есть путь по дереву целей до цели компании.
- [ ] Outcome-метрика, база, цель и срок записаны.
- [ ] Ценность проверена: интервью, прототип, эксперимент.
- [ ] Уровень документации выбран по риску.

Б.2 Перед спринтом

- [ ] Цель спринта сформулирована одной фразой.
- [ ] Истории соответствуют критериям готовности.
- [ ] Загрузка не выше 80% реальной ёмкости.
- [ ] Зависимости известны, владельцы назначены.
- [ ] Время на техдолг заложено.

Б.3 Перед релизом

- [ ] Критерии релиза выполнены, блокирующих дефектов нет.

- [ ] Нефункциональные требования проверены, включая нагрузку.
- [ ] События аналитики проверены QA.
- [ ] Функция за флагом, план раскатки с порогами готов.
- [ ] Поддержка обучена, база знаний обновлена.
- [ ] Продажи знают, что можно обещать.
- [ ] Миграции проверены на копии прода, есть план отката.

Б.4 В первую неделю после релиза

- [ ] Метрики здоровья в норме.
- [ ] Обращения размечены и разобраны.
- [ ] Дежурство команды организовано.
- [ ] Флаги раскатки двигаются по плану или остановлены с причиной.

Б.5 Через 1 и 3 месяца

- [ ] Освоение и outcome сравнены с целями.
- [ ] Эффект оценён по холдауту или контролю.
- [ ] Принято решение: развивать, исправлять, сворачивать.
- [ ] Инсайты занесены в реестр.
- [ ] Релизный флаг удалён.

Б.6 Раз в квартал

- [ ] Метрики DORA и потока сняты.
- [ ] План/факт по фичам сведён, коэффициент точности посчитан.
- [ ] Бэклог почищен.

- [ ] Доли ёмкости по потокам пересмотрены.
- [ ] SLO и реестр техдолга пересмотрены.

B Глоссарий

Термины гайда с короткими определениями. Английские названия — как их используют в командах и вакансиях.

A/B-тест	Эксперимент со случайным разделением пользователей на группы с разными вариантами.
Acceptance criteria (критерии приёмки)	Проверяемые условия, при которых история считается выполненной.
Backlog (бэклог)	Упорядоченный список работы команды.
Canary release (канареечный релиз)	Выкатка на небольшую долю пользователей перед полной раскаткой.
Change failure rate	Доля изменений, приведших к откату, хотфиксу или инциденту.
CI/CD	Непрерывная интеграция и поставка: автоматическая сборка, проверка и выкатка.
Code freeze	Период запрета новых изменений перед релизом.
Cost of delay (стоимость задержки)	Ценность, теряемая за каждую единицу времени задержки.
Definition of Done	Общие критерии завершённой работы.
Discovery	Поиск и проверка того, что стоит делать.
DORA-метрики	Частота деплоев, время поставки изменения, доля неудачных изменений, время восстановления.

Error budget (бюджет ошибок)	Допустимое время недоступности при заданном SLO.
Feature flag (флаг функции)	Переключатель, включающий функцию без нового релиза.
Flow efficiency (эффективность потока)	Доля активной работы во времени поставки.
Go-to-market (GTM)	План вывода функции на рынок.
Grooming (уточнение бэклога)	Регулярная подготовка историй к разработке.
Holdout (холдаут)	Группа, у которой функция долго остаётся выключенной для измерения эффекта.
Job story	История в формате «Когда… я хочу… чтобы…».
Kanban	Метод управления потоком с ограничением незавершённой работы.
Lead time	Время от запроса до поставки пользователю.
MoSCoW	Разделение объёма на Must, Should, Could, Won't.
MTTR	Среднее время восстановления после инцидента.
North Star	Главная метрика ценности продукта для клиента.
Outcome	Изменение поведения пользователей, ведущее к результату бизнеса.
Output	Выпущенный объём работы.
PRD	Документ требований к продукту.
RICE	Приоритизация: охват × влияние × уверенность / усилия.
Scrum	Фреймворк с короткими итерациями, ролями и событиями.

Scrumban	Гибрид ритма Scrum и управления потоком Kanban.
SLA	Договорное обязательство по уровню сервиса.
SLI	Индикатор уровня сервиса.
SLO	Целевое значение индикатора за период.
Spike (спайк)	Ограниченное по времени исследование технического вопроса.
User story	История в формате «Как… я хочу… чтобы…».
WIP-лимит	Ограничение числа элементов в работе на этапе.
WSJF	Стоимость задержки, делённая на размер работы.
Вертикальный срез	Небольшая часть фичи, проходящая через все слои и дающая ценность.
Дерево целей	Связь цели компании, рычагов, причин и инициатив.
Закон Литтла	Время цикла = незавершённая работа / пропускная способность.
Технический долг	Накопленные упрощения, замедляющие будущую работу.

КОНЕЦ ГАЙДА

Теперь проверьте себя

Тест по теоретическим основам и шаблоны к главам лежат на странице гайда.

- [ ] Пройдите тест по гайду — онлайн
- [ ] Возьмите шаблоны документов — с. 200
- [ ] Пройдите чек-листы по этапам — с. 203

Цены рекламы, тарифы и доступность сервисов указаны как ориентиры и устаревают быстрее остального текста. Числовая модель сквозного продукта вымышлена, но внутренне непротиворечива.